KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Approved by AICET, New Delhi & Affiliated to Anna University, Chennai
Accredited by NBA (CSE, ECE, EEE, Mech), NAAC,
Recognized by UGC with 2(f) &12(B) An ISO 9001:2005 Certified Institution
Namakkal - Trichy Main Road,
Tholurpatti (post), Thottiam, Trichy – 621215, Tamil Nadu

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

COURSE MATERIAL

Course Name with Code : **24AD401 / ARTIFICIAL INTELLIGENCE**

Course Faculty : **A. SANGEETHA**

Batch : **2024 – 2028**

Year / Semester / Section : **II / IV**

S.NO	TOPICS COVERED	PAGE NO
	UNIT – I INTRODUCTION	
1	Introduction to AI	1
2	Foundation of AI	3
3	History of AI	4
4	Future of Artificial Intelligence	6
5	Characteristics of Intelligent Agents	7
6	Agents and Environments	9
7	Structure of Agents	12
8	Nature of Environments	14
9	Typical Intelligent Agents	17
	UNIT – II PROBLEM SOLVING METHODS	
10	Problem solving Methods –Search Strategies	23
11	Uninformed – Informed Search	24
12	Heuristics Search	30
13	Local Search Algorithms and Optimization Problems	33
14	Searching with Partial Observations	34

15	Constraint Satisfaction Problems	36
16	Constraint Propagation – Backtracking Search	38
17	Game Playing	39
18	Optimal Decisions in Games	40
19	Alpha – Beta Pruning	43
UNIT – III KNOWLEDGE REPRESENTATION		
20	Predicate Logic	48
21	Propositional Logic	51
22	Agents Based on Professional Logic	54
23	First Order Logic – Unification	57
24	Forward Chaining – Backward Chaining – Resolution	62
25	Knowledge Representation	65
26	Ontological Engineering	66
27	Categories and Objects - Events	68
28	Mental Events and Mental Objects	69
29	Reasoning Systems for categories – Reasoning with Default Information	70
UNIT – IV PLANNING AND REASONING		
30	Classical Planning	74
31	Heuristic Planning	79
32	Hierarchical Planning	81
33	Analysis of Planning approach Uncertainty	84
34	Basic Probability Notation - Bayes Rule	89
35	Naïve Bayes Models	95
36	The Semantics of Bayesian Networks	101
37	Exact Inference in Bayesian Networks	107
38	Casual Networks	111
UNIT – V AI REAL-WORLD APPLICATIONS		
39	Fraud Detection in Banking	114
40	Fraud Detection in Finance	114
41	AI-Powered Voice Assistants	116
42	Object Detection I Autonomous Vehicles	118
43	AI in Self-Driving Cars	121

44	Automated Grading and Feedback Systems	124
45	AI in Music Composition and Creativity	127
46	Traffic Management and Smart Transportation	129
47	Case Study on AI – Assisted X-Ray Analysis	132
48	**Content Beyond Syllabus –** Expert Systems: Component and Architecture	

Course Faculty Signature **HOD**

UNIT-I

INTRODUCTION

Introduction to AI – Foundation and History of AI - Future of Artificial Intelligence – Characteristics of Intelligent Agents – Agents and Environments- Structure of Agents – Nature of Environments -Typical Intelligent Agents.

1.1 Introduction to AI

We have claimed that AI is exciting, but we have not said what it is. In Figure 1.1 we see eight definitions of AI, laid out along two dimensions. The definitions on top are concerned with thought processes and reasoning, whereas the ones on the bottom address behavior. The definitions on the left measure success in terms of fidelity to human performance, whereas the ones on the right measure against an ideal performance measure, called rationality. A system is rational if it does the "right thing," given what it knows. Historically, all four approaches to AI have been followed, each by different people with different methods. A human-centered approach must be in part an empirical science, in volving observations and hypotheses about human behavior. A rationalist1 approach involves a combination of mathematics and engineering. The various group have both disparaged and helped each other. Let us look at the four approaches in more detail.

Thinking Humanly	**Thinking Rationally**
"The exciting new effort to make computers think ... machines with minds, in the full and literal sense." (Haugeland, 1985) "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning ..." (Bellman, 1978)	"The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly	**Acting Rationally**
"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) "The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990)	"Computational Intelligence is the study of the design of intelligent agents." (Poole et al., 1998) "AI is concerned with intelligent behavior in artifacts." (Nilsson, 1998)

1.1.1 Acting humanly: The Turing Test approach

The Turing Test, proposed by Alan Turing (1950), was designed to provide a satisfactory operational definition of intelligence. A computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer. Chapter 26 discusses the details of the test and whether a computer would really be intelligent if it passed. For now, we note that programming a computer to pass a rigorously applied test provides plenty to work on. The computer would need to possess the following capabilities:

- **natural language processing** to enable it to communicate successfully in English;
- **knowledge representation** to store what it knows or hears;
- **automated reasoning** to use the stored information to answer questions and to draw new conclusions;
- **machine learning** to adapt to new circumstances and to detect and extrapolate patterns.

1.1.2 Thinking humanly: The cognitive modelling approach

If we are going to say that a given program thinks like a human, we must have some way of determining how humans think. We need to get inside the actual workings of human minds. There are three ways to do this: through introspection—trying to catch our own thoughts as they go by; through psychological experiments—observing a person in action; and through brain imaging—observing the brain in action. Once we have a sufficiently precise theory of the mind, it becomes possible to express the theory as a computer program. If the program's input–output behavior matches corresponding human behavior, that is evidence that some of the program's mechanisms could also be operating in humans.

In the early days of AI there was often confusion between the approaches: an author would argue that an algorithm performs well on a task and that it is therefore a good model of human performance, or vice versa. Modern authors separate the two kinds of claims; this distinction has allowed both AI and cognitive science to develop more rapidly. The two fields continue to fertilize each other, most notably in computer vision, which incorporates neurophysiological evidence into computational models.

1.1.3 Thinking rationally: The "laws of thought" approach

The Greek philosopher Aristotle was one of the first to attempt to codify "right thinking," that is, irrefutable reasoning processes. His syllogisms provided patterns for argument structures that always yielded correct conclusions when given correct premises—for example, "Socrates is a man; all men are mortal; therefore, Socrates is mortal." These laws of thought were supposed to govern the operation of the mind; their study initiated the field called logic.

There are two main obstacles to this approach. First, it is not easy to take informal knowledge and state it in the formal terms required by logical notation, particularly when the knowledge is less than 100% certain. Second, there is a big difference between solving a problem "in principle" and solving it in practice.

1.1.4 Acting rationally: The rational agent approach

An agent is just something that acts (agent comes from the Latin agere, to do). Of course, all computer programs do something, but computer agents are expected to do more: operate autonomously, perceive their environment, persist over a prolonged time period, adapt to change, and create and pursue goals. A rational agent is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome.

1.2 Foundation of AI

1. Philosophy

• Can formal rules be used to draw valid conclusions?

• How does the mind arise from a physical brain?

• Where does knowledge come from?

• How does knowledge lead to action?

Rationalism, Dualism, Materialism, Empiricism, Induction, Logical Positivism, Confirmation Theory.

2. Mathematics

- What are the formal rules to draw valid conclusions?
- What can be computed?
- How do we reason with uncertain information?

The main three fundamental areas are logic, computation and probability. Algorithm, incompleteness theorem, computable, tractability, NP completeness, Non deterministic polynomial and probability.

3. Economics

- How should we make decisions so as to maximize payoff?
- How should we do this when others may not go along?
- How should we do this when the payoff may be far in the future?
- Utility, Decision Theory, Game Theory, Operations Research.

4. Neuroscience

Neuroscience is the study of the nervous system, especially the brain. We are still a long way from understanding how cognitive processes actually work. The truly amazing conclusion is that a collection of simple cells can lead to thought, action, and consciousness or, brains cause minds. The only real alternative theory is mysticism: that minds operate in some mystical realm that is beyond physical science.

5. Psychology

- How do humans and animals think and act?

  Behaviourism, Cognitive psychology.
- The three key steps of a knowledge-based agent:

6. Computer engineering

- How can we build an efficient computer?

Operational computer and operational programmable computer. AI has pioneered many ideas that have made their way back to mainstream computer science, including time sharing, interactive interpreters, personal computers with windows and mice, rapid development environments, the linked list data type, automatic storage management, and key concepts of symbolic, functional, declarative, and object-oriented programming.

7. Linguistics

- How does language relate to thought?
- Verbal Behavior — behaviorist approach to language learning

Computational linguistics or natural language processing and knowledge representation.

8. Control theory and cybernetics

- How can artifacts operate under their own control? control Theory, Homeostatic and objective function.

1.3 History of AI

- Artificial Intelligence is not a new word and not a new technology for researchers. This technology is much older than you would imagine. Even there are the myths of Mechanical men in Ancient Greek and Egyptian Myths. Following are some milestones in the history of AI which defines the journey from the AI generation to till date development.

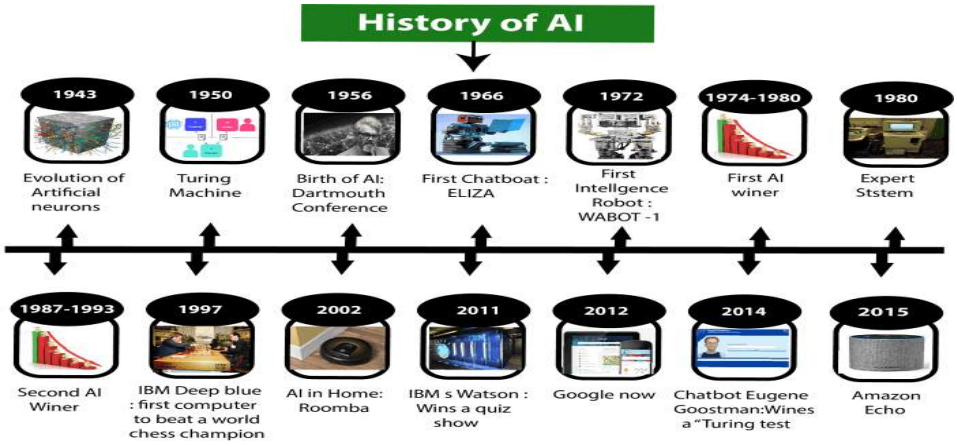


Figure 1.9 History of Artificial Intelligence

Maturation of Artificial Intelligence (1943-1952)

- **Year 1943:** The first work which is now recognized as AI was done by Warren McCulloch and Walter pits in 1943. They proposed a model of **artificial neurons**.

- **Year 1949:** Donald Hebb demonstrated an updating rule for modifying the connection strength between neurons. His rule is now called **Hebbian learning**.
- **Year 1950:** The Alan Turing who was an English mathematician and pioneered Machine learning in 1950. Alan Turing publishes **"Computing Machinery and Intelligence"** in which he proposed a test. The test can check the machine's ability to exhibit intelligent behavior equivalent to human intelligence, called a **Turing test**.

The birth of Artificial Intelligence (1952-1956)

- **Year 1955:** An Allen Newell and Herbert A. Simon created the "first artificial intelligence program"Which was named as **"Logic Theorist"**. This program had proved 38 of 52 Mathematics theorems, and find new and more elegant proofs for some theorems.
- **Year 1956:** The word "Artificial Intelligence" first adopted by American Computer scientist John McCarthy at the Dartmouth Conference. For the first time, AI coined as an academic field.
- At that time high-level computer languages such as FORTRAN, LISP, or COBOL were invented. And the enthusiasm for AI was very high at that time.

The golden years-Early enthusiasm (1956-1974)

- **Year 1966:** The researchers emphasized developing algorithms which can solve mathematical problems. Joseph Weizenbaum created the first chatbot in 1966, which was named as ELIZA.
- **Year 1972:** The first intelligent humanoid robot was built in Japan which was named as WABOT-1.

The first AI winter (1974-1980)

- The duration between years 1974 to 1980 was the first AI winter duration. AI winter refers to the time period where computer scientist dealt with a severe shortage of funding from government for AI researches.
- During AI winters, an interest of publicity on artificial intelligence was decreased.

A boom of AI (1980-1987)

- **Year 1980:** After AI winter duration, AI came back with "Expert System". Expert systems were programmed that emulate the decision-making ability of a human expert.
- In the Year 1980, the first national conference of the American Association of Artificial Intelligence **was held at Stanford University**.

The second AI winter (1987-1993)

- The duration between the years 1987 to 1993 was the second AI Winter duration.

- Again Investors and government stopped in funding for AI research as due to high cost but not efficient result. The expert system such as XCON was very cost effective.

The emergence of intelligent agents (1993-2011)

- **Year 1997:** In the year 1997, IBM Deep Blue beats world chess champion, Gary Kasparov, and became the first computer to beat a world chess champion.
- **Year 2002:** for the first time, AI entered the home in the form of Roomba, a vacuum cleaner.
- **Year 2006:** AI came in the Business world till the year 2006. Companies like Facebook, Twitter, and Netflix also started using AI.

Deep learning, big data and artificial general intelligence (2011-present)

- **Year 2011:** In the year 2011, IBM's Watson won jeopardy, a quiz show, where it had to solve the complex questions as well as riddles. Watson had proved that it could understand natural language and can solve tricky questions quickly.
- **Year 2012:** Google has launched an Android app feature "Google now", which was able to provide information to the user as a prediction.
- **Year 2014:** In the year 2014, Chatbot "Eugene Goostman" won a competition in the infamous "Turing test."
- **Year 2018:** The "Project Debater" from IBM debated on complex topics with two master debaters and also performed extremely well.
- Google has demonstrated an AI program "Duplex" which was a virtual assistant and which had taken hairdresser appointment on call, and lady on other side didn't notice that she was talking with the machine.
- Now AI has developed to a remarkable level. The concept of Deep learning, big data, and data science are now trending like a boom. Nowadays companies like Google, Facebook, IBM, and Amazon are working with AI and creating amazing devices. The future of Artificial Intelligence is inspiring and will come with high intelligence.

1.4 Future of Artificial Intelligence

Artificial Intelligence (AI) has rapidly evolved from being a theoretical concept in computer science to a powerful force shaping the present and future of humanity. Once confined to research labs, AI now plays a central role in everyday life—recommending what we watch, guiding medical diagnoses, enabling voice assistants, and even piloting self-driving cars. As technology continues to advance, the **future of AI promises a transformation deeper than the Industrial Revolution**, influencing economies, cultures, and human interactions at a global scale.

1.4.1 Expanding Capabilities of AI

Today's AI excels in narrow tasks—such as identifying faces, predicting consumer behavior, or playing complex games. However, the long-term vision is the rise of Artificial General Intelligence (AGI), where machines can learn, reason, and adapt across multiple fields just as humans do. Researchers are working on neuromorphic chips that mimic the human brain, advanced reinforcement learning models, and self-evolving algorithms. These innovations are expected to create systems capable of problem-solving in unfamiliar domains and exhibiting creative thinking beyond programmed instructions.

1.4.2 Integration into Human Life

In the future, AI will become an invisible partner in our daily routines. Smart homes will anticipate human needs—adjusting energy usage, providing health alerts, or assisting in learning activities. Transportation networks will rely on autonomous vehicles, reducing accidents and optimizing traffic. In healthcare, AI will analyze genetic information, predict potential diseases before symptoms appear, and assist doctors in personalized treatments. Education will also witness transformation through adaptive platforms that adjust learning paths to each student's pace and style.

1.4.3 Economic Transformation and Work

The future of AI is closely tied to the global economy. Industries such as manufacturing, agriculture, logistics, and finance are already benefiting from intelligent automation. Predictive maintenance in factories, precision farming, and algorithm-driven financial analysis are just the beginning. While there are concerns about job displacement, history suggests that technological revolutions create new categories of employment even as they retire old ones.

1.4.4 Ethical and Social Implications

With great power comes great responsibility. The rise of AI brings complex ethical questions: Who is accountable if an autonomous vehicle makes a fatal error? How do we prevent algorithmic bias from reinforcing social inequalities? How can privacy be protected in a world where AI systems rely on massive amounts of data?

1.4.5 Human-AI Partnerships

Rather than replacing humans, the future vision of AI lies in partnerships. AI can handle massive datasets, uncover hidden patterns, and provide accurate predictions, while humans bring values, creativity, empathy, and moral judgment. Together, this synergy could accelerate breakthroughs in medicine, climate solutions, and even space exploration.

1.5 Characteristics of Intelligent Agents

In Artificial Intelligence, the term **intelligent agent** is fundamental to understanding how machines can act purposefully in complex environments. An agent is broadly defined as anything that perceives its environment through sensors and acts upon it using actuators. However, what

distinguishes an **intelligent agent** from a simple program is its ability to act rationally and adaptively, without requiring continuous human direction.

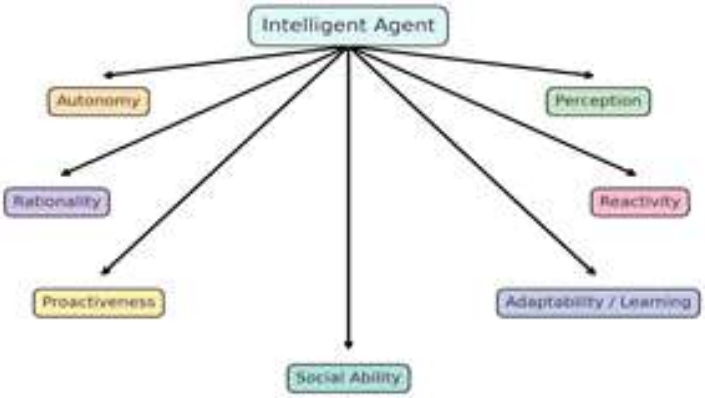


Figure:1.10 Characteristics of Intelligent Agents

1. **The Central Box: Intelligent Agent**

At the center of the framework is the intelligent agent itself. It is the decision-making entity that receives inputs from its environment, processes these inputs, and produces outputs in the form of actions. Unlike traditional programs that follow fixed instructions, intelligent agents evaluate their environment and select actions designed to achieve specific goals. The central box symbolizes this rational decision-maker that integrates all the surrounding characteristics into a unified system.

2. **Autonomy: Independence in Decision-Making**

The first defining feature of an intelligent agent is autonomy. An autonomous agent makes decisions based on its own knowledge and internal processes, rather than relying entirely on human commands. For example, a self-driving car does not require step-by-step instructions from a human driver; it independently interprets traffic signals, obstacles, and road conditions to navigate safely. Autonomy is crucial because it allows agents to function effectively in environments where human supervision is limited or impractical.

3. **Perception: Sensing the Environment**

Perception is the gateway through which agents connect with the world around them. Using sensors, intelligent agents collect data about their environment, whether physical or digital. A robot may use cameras, infrared detectors, and GPS for perception, while a software agent may rely on data from web servers or user input. The accuracy and richness of perception directly influence the quality of decisions. Without perception, an agent would lack the context needed to act rationally.

4. Rationality: Choosing the Best Action

At the core of intelligence is rationality—the ability to select actions that are expected to maximize success. A rational agent is not perfect or all-knowing, but it consistently makes reasonable choices based on the information it has. For example, a chess-playing program may

not always win, but it selects moves that maximize its chances of winning according to its evaluation function. Rationality ensures that agents act purposefully rather than randomly, aligning their behavior with predefined performance measures.

5. Reactivity: Responding to Changes

An intelligent agent must also demonstrate reactivity, meaning it can respond promptly to changes in its environment. Environments are rarely static, and agents must be capable of adjusting to new or unexpected events. For instance, an automated stock trading system reacts immediately to market fluctuations, while a self-driving vehicle reacts instantly to a pedestrian stepping onto the road. Reactivity ensures that agents remain relevant and effective in dynamic conditions.

6. Proactiveness: Taking Initiative Toward Goals

Beyond reacting, agents exhibit proactiveness—the ability to plan ahead and pursue goals independently. Proactive agents anticipate future needs and take action without waiting for external triggers. A digital assistant like Google Assistant or Siri is proactive when it reminds users about upcoming appointments or suggests faster travel routes. This quality gives agents foresight, enabling them to contribute to long-term objectives rather than simply responding to immediate events.

7. Adaptability / Learning: Improving from Experience

A hallmark of true intelligence is the capacity to learn from experience. Intelligent agents become more effective by analyzing their successes and failures and adjusting their behavior accordingly. A spam filter improves its accuracy over time by learning from user feedback, and a machine translation system enhances performance as it processes more data. Adaptability ensures that agents remain capable in changing environments and are not restricted to static knowledge.

8. Social Ability: Interacting with Humans or Other Agents

Finally, many intelligent agents require social ability. They often need to interact, negotiate, or collaborate with humans or other agents to achieve their goals. Social ability is critical in multi-agent systems, where multiple intelligent entities work together, such as in distributed robotics or traffic coordination. In human-agent interaction, this ability is seen in chatbots, virtual assistants, and collaborative robots that use natural language processing to communicate effectively.

1.6 Agents and Environments

An agent can be described as anything that is capable of perceiving its surrounding environment through sensors and interacting with it using actuators. This simple but powerful concept forms the foundation of artificial intelligence.

- A human agent uses sensory organs like eyes, ears, and skin as sensors and employs hands, legs, and speech as actuators to interact with the environment.
- A robotic agent may have cameras, microphones, or infrared sensors as its sensing units, and wheels, motors, or robotic arms as actuators.
- A software agent, on the other hand, perceives information in the form of files, keystrokes, or network packets, and acts by displaying outputs, sending data, or writing to storage.
- The environment is essentially everything that lies outside the agent. In principle, it could mean the whole universe, but in practice, it refers to the specific part of the world that influences the agent's perceptions and is affected by the agent's actions.
- The percept is the input received by the agent's sensors at a particular instant. A percept sequence is the complete history of all percepts the agent has experienced. The behavior of the agent is determined by its agent function, which maps each percept sequence to appropriate action

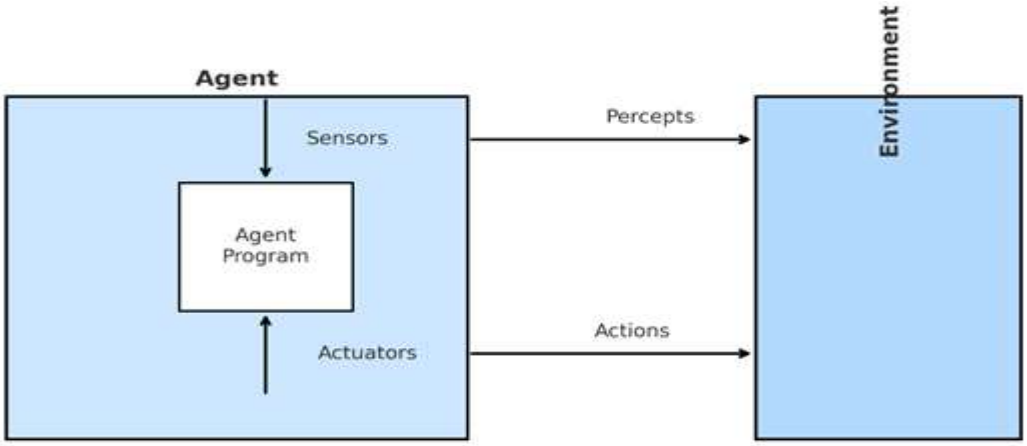


Figure:1.11 Agent Interaction with Environment via Sensors and Actuators

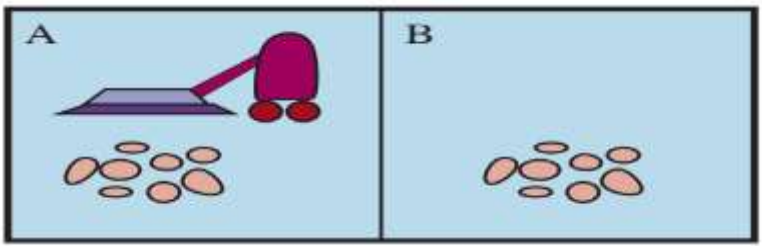


Figure:1.12 Simple Vacuum-Cleaner Agent Environment

The vacuum-cleaner world is a simplified model used to illustrate the fundamental concepts of intelligent agents. It consists of just two locations, labeled **A** and **B**, each of which can either be **clean** or **dirty**. The agent in this world is a vacuum cleaner equipped with three main abilities:

1. **Perception** – The agent can sense its current location (whether it is in square A or B) and detect whether that square is clean or dirty.

2. **Action** – The agent can perform three possible actions:
    - **Suck**: Clean the square it currently occupies.
    - **Move Left**: Shift from the right location (B) to the left location (A).
    - **Move Right**: Shift from the left location (A) to the right location (B).
3. **Decision-Making** – Based on its perception, the agent decides which action to take.

In the example shown in the diagram, the vacuum agent is located in **square A**, where the floor is dirty. The agent can perceive this situation and choose the action **Suck**, which removes the dirt. Once clean, it can move to square B if necessary to check and clean that location.

1.6.1 Performance measures

- Agents are judged by the consequences of their actions.
- A performance measure evaluates how well an agent achieves desirable outcomes in its environment.
- **Example:** For a vacuum-cleaner agent, measuring only "dirt collected" can encourage unintended behavior (spilling and re-cleaning dirt). A better measure is "keeping the floor clean" while minimizing wasted energy and noise.
- **Key idea:** Performance measures must reflect the actual goals we want the agent to achieve, not just approximate behaviours.

1.6.2 Rationality

- A rational agent is one that chooses actions expected to maximize performance, based on what it knows and perceives.
- Rationality depends on:
    1. The performance measure (what counts as success).
    2. The agent's prior knowledge about the environment.
    3. The set of available actions.
    4. The percept received so far.
- **Rationality ≠ Perfection:** An agent cannot predict every future event (omniscience is impossible). It acts rationally based on available evidence, even if outcomes turn out differently.
- Agents may also gather extra information (e.g., "look before crossing the road") to make better decisions.

1.6.3 Omniscience, Learning, and Autonomy

- **Omniscience:** Knowing the exact outcome of all actions (impossible in reality). Agents must act on expected, not guaranteed, results.

- **Learning:** Agents should improve their performance over time by learning from percept's and past experiences. Example: A vacuum agent that learns where dirt usually appears performs better than one with fixed rules.
- **Autonomy:** An agent should not depend only on the knowledge given by its designer. Instead, it should become more independent by learning and adapting.
- A rational, autonomous agent starts with some built-in knowledge but gradually relies more on its own learning, making it effective in diverse environments.

1.7 Structure of Agents

1. Agent Function vs. Agent Program

At the core of every intelligent agent lies the distinction between the agent function and the agent program.

- The agent function is an abstract, mathematical description that maps percepts (the inputs an agent receives from its environment) to actions (the outputs it performs). It represents the theoretical definition of how the agent should behave in any given situation.
- The agent program, in contrast, is the practical implementation of the agent function. It is the actual piece of software or set of instructions that runs on a physical device, such as a computer or robot. This program takes percepts as input, processes them according to its internal rules, and produces an appropriate action.

2. Agent Architecture

The agent architecture refers to the physical platform on which the agent program operates. This architecture includes the hardware components — such as sensors, actuators, processors, and memory — that allow the agent to interact with its environment.

A general definition can be expressed as:

Agent = Architecture + Program

Thus, the architecture handles three essential tasks:

1. Receiving percepts from sensors – Collecting information from the environment.
2. Running the program – Processing percepts and determining the best action.
3. Sending actions to actuators – Executing the chosen action in the environment.

3. Example: Table-Driven Agent

To illustrate the connection between agent function, program, and architecture, consider the table-driven agent.

This simple design works by storing a complete history of percepts and consulting a predefined table to decide on an action. For each possible sequence of percepts, the table

specifies the corresponding action. When the agent receives a new percept, it appends it to the history and looks up the sequence in the table to select the appropriate action.

The pseudocode representation is as follows:

function TABLE-DRIVEN-AGENT(percept) **returns** an action

persistent: percepts, a sequence, initially empty

table, a table of actions, indexed by percept sequences, initially fully specified

append percept to the end of percepts

action LOOKUP(percepts, table)

return action

1.7.1 Agent Programs

1. Agent Function vs. Agent Program

- To understand the structure of an intelligent agent, it is important to distinguish between the agent function and the agent program.
- An agent program is the software implementation that receives percepts from the agent's sensors and produces actions to be executed by its actuators. In other words, it is the practical mechanism by which the agent operates in the environment.
- An agent function, on the other hand, is a theoretical mapping from percept sequences to actions. It defines the ideal behavior of the agent by considering the complete history of all percepts the agent has ever experienced.
- If the current action depends only on the most recent percept, the program can be very simple. However, in most real-world environments, history matters. For example, the decision a taxi makes at an intersection may depend not only on the current traffic signal but also on where the passenger asked to go earlier. In such cases, the agent program requires memory to store percepts so that the full sequence can be considered when choosing actions.

2. Table-Driven Agents

One of the earliest and most straightforward ways to represent an agent function is through a table-driven agent.

- The basic idea is to maintain a large lookup table that lists every possible percept sequence and specifies the corresponding action.
- For example, in the vacuum world problem, the agent could consult its table to decide whether to move left, right, or clean, based on the sequence of percepts it has observed.

Consider the case of an automated taxi. If every frame from the taxi's camera had to be stored as a percept and mapped to an action, the resulting table would have more entries than the total

number of atoms in the universe. Similarly, in chess, the number of possible game states is estimated to exceed 10^{150}. A table containing an entry for every possible sequence of moves would be utterly impractical.

4. **Why Table-Driven Agents Still Matter**

   Although they are impossible to implement in practice, table-driven agents still play a valuable role in theory. They provide a precise definition of what an agent function should achieve: a complete mapping from percept sequences to actions.

5. AI's Real Challenge

- The true challenge in artificial intelligence is to build compact, efficient programs that approximate the agent function without resorting to enormous lookup tables.
- A useful analogy comes from mathematics. Before the invention of algorithms like Newton's method, the square root of a number was often calculated using large precomputed tables. This was effective but extremely wasteful in terms of space. Modern algorithms allow the same task to be done quickly and efficiently without requiring massive storage.
- In AI, the goal is similar: instead of building an impractical table for intelligent behavior, researchers seek to design general-purpose algorithms that can reason, learn, and act rationally in diverse environments.

1.8 Nature of Environments

Before designing rational agents, it is essential to understand task environments—the "problems" that agents are intended to solve. We begin by explaining how to specify a task environment, illustrated with several examples. Then, explore the different types and complexities of task environments, as these characteristics directly shape the design of an effective agent program.

1.8.1 Specifying the task environment

Task Environments and PEAS

When designing a rational agent, we must clearly define its **task environment**, which includes the **performance measure, environment, actuators, and sensors**—collectively known as the **PEAS description**. Specifying the task environment thoroughly is the first step in agent design. For example, consider a **vacuum-cleaner agent**, which has a simple task environment. A more complex example is an **automated taxi driver**.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

Figure:1.13 PEAS description of the task environment for an automated taxi driver.

1.Performance Measure

The agent's goals determine its success. For an automated taxi, these include:

- Reaching the correct destination safely and quickly
- Minimizing fuel consumption and wear
- Reducing cost and trip time
- Following traffic laws
- Ensuring passenger comfort
- Maximizing profits

Some goals may conflict, requiring the agent to make trade-offs.

2. Environment

The taxi must navigate a variety of roads (rural lanes, city streets, highways) with:

- Other vehicles, pedestrians, stray animals, roadworks, and police
- Weather conditions (snow, rain, etc.)
- Different driving conventions (right-hand vs. left-hand traffic)

  A more restricted environment simplifies agent design.

3. Actuators

The taxi's actuators enable it to interact with the world, including:

- Engine, accelerator, brakes, and steering
- Horn, turn signals, and displays
- Voice output for passenger interaction
- Communication with other vehicles

4. Sensors

Sensors provide information about the environment and the taxi itself:

- Cameras, lidar, and ultrasound for obstacle detection
- Speedometer and accelerometer for motion control

- Engine, fuel, and electrical system sensors
- GPS for navigation
- Touchscreen or voice input for passenger commands

1.8.2 Properties of task environments

Task environments in AI can vary widely, but they can be classified along a few key **dimensions**, which influence the design of the agent and the techniques used for implementation.

1. Fully Observable vs. Partially Observable

- **Fully observable:** The agent's sensors capture the complete state of the environment relevant to decision-making. No internal memory is needed.
- **Partially observable:** Some relevant information is missing or noisy. For example, a vacuum agent with only a local dirt sensor cannot see other squares, and a taxi cannot know other drivers' intentions.
- **Unobservable:** The agent has no sensors. Even then, some goals may still be achievable.

2. Single-Agent vs. Multiagent

- **Single-agent:** The agent operates alone, e.g., solving a crossword puzzle.
- **Multiagent:** Other agents affect the environment, e.g., chess or taxi driving.
    - **Competitive:** Agents have conflicting goals, e.g., chess.
    - **Cooperative:** Agents share some goals, e.g., avoiding collisions in traffic.

3. Deterministic vs. Nondeterministic

- **Deterministic:** The next state is fully determined by the current state and actions.
- **Nondeterministic:** Outcomes are uncertain, e.g., taxi driving with unpredictable traffic or mechanical failures.
- **Stochastic:** A type of nondeterministic environment where probabilities are known.

4. Episodic vs. Sequential

- **Episodic:** Each action is independent of previous actions, e.g., classifying parts on an assembly line.
- **Sequential:** Current actions affect future states, e.g., chess or taxi driving.

5. Static vs. Dynamic

- **Static:** The environment does not change while the agent decides.
- **Dynamic:** The environment changes continuously, requiring real-time decisions, e.g., taxi driving.
- **Semidynamic:** Environment is static, but performance measures change over time, e.g., timed chess.

6. Discrete vs. Continuous

- **Discrete:** Finite states, actions, and percepts, e.g., chess.
- **Continuous:** States, actions, or percepts vary smoothly over a range, e.g., taxi driving.

7. Known vs. Unknown

- **Known:** The agent knows the rules or laws of the environment.
- **Unknown:** The agent must learn how the environment works.

The most challenging environments are **partially observable, multiagent, nondeterministic, sequential, dynamic, continuous, and unknown**, such as driving in a new country with unfamiliar rules and passengers.

Examples of Task Environments :

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 1.14: Task environments and their characteristics.

1.9 Typical Intelligent Agents.

Intelligent agents are systems that perceive their environment through sensors and act upon it through actuators to achieve specific goals. The choice of agent depends on the complexity of the environment and the desired level of intelligence. Below are some commonly used types of intelligent agents:

1. **Simple Reflex Agents** – Act only on current percepts, ignoring history.
2. **Model-Based Reflex Agents** – Use internal memory of past percepts (world model).
3. **Goal-Based Agents** – Actions guided by specific goals.
4. **Utility-Based Agents** – Choose actions based on preferences/utility to maximize performance.

1.9.1 Simple Reflex Agents

- **Simple reflex agents** choose actions based **only on the current percept**, ignoring past percepts.

- Example: A vacuum agent that decides what to do based on its current location and whether it detects dirt.

Advantages

- Very small and simple programs compared to large lookup tables.
- Can be implemented with **if-then rules** or even **Boolean circuits**.
- Works well in **fully observable environments** where current percepts give complete information.

Condition–Action Rules

- Actions are triggered by conditions detected in the environment.
- Format: if condition then action
- Example:
    - Driving: if car-in-front-is-braking then initiate-braking
    - Vacuum: if location is dirty then suck

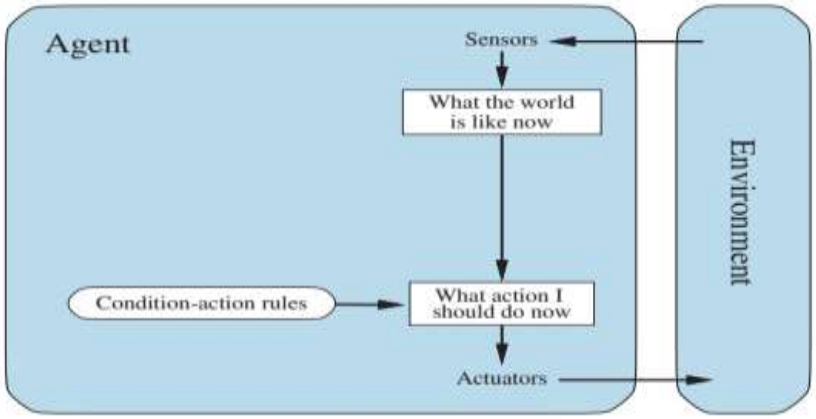


Figure 1.15 Schematic diagram of a simple reflex agent.

Implementation Structure

- General approach:
    1. **INTERPRET-INPUT:** Converts percept into abstract state.
    2. **RULE-MATCH:** Finds the first condition–action rule matching that state.
    3. Execute the corresponding action.

Randomization Trick

- In some cases, **random actions** can help avoid problems like infinite loops.
- Works on average but is **not always rational**; deterministic agents perform better in most cases.

Pseudocode:

```
function SIMPLE-REFLEX-AGENT(percept):
  state = INTERPRET-INPUT(percept)
  rule = RULE-MATCH(state, rules)
```

action = rule.ACTION

return action

1.9.2 Model-based reflex agents

1.Agent–Environment Interaction

- The agent interacts with the environment using sensors and actuators.
- Sensors detect the current percepts (e.g., location and dirt for a vacuum, brake lights for a car).
- Actuators perform actions (e.g., move, suck dirt, apply brakes).

2.Current State / Decision Process

- Represented by a rectangle in the diagram.
- This is where the agent interprets percepts and decides what action to take.
- The agent does not store past percepts; it only uses the current percept.

3.Condition–Action Rules

- Represented conceptually by ovals (background information).
- Each rule connects a condition to an action:
- The agent searches through the rule set and selects the first matching rule.

4. Flow of Information

1. Percept → captured by sensors.
2. INTERPRET-INPUT → abstracts the percept into a state.
3. RULE-MATCH → finds a condition–action rule matching the state.
4. Action → sent to actuators, affecting the environment.
5. Environment changes → new percepts are sensed → cycle repeats.

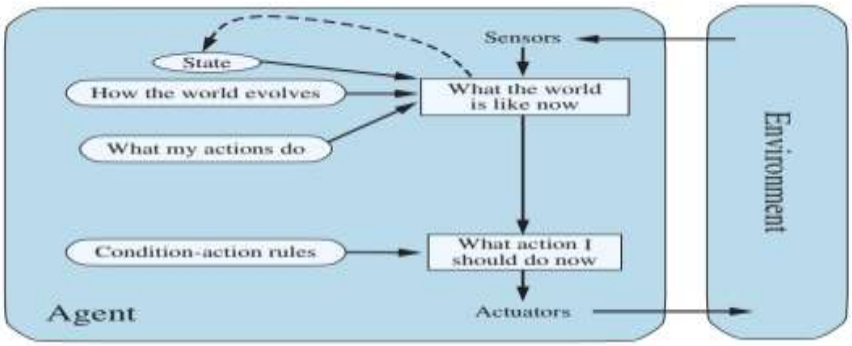


Figure: 1.16 Model-based reflex agents

1.9.3 Goal-Based Agents – Simplified Notes

1. Definition

- Goal-based agents use both the current state and goal information to decide actions.
- Goals describe desirable situations (e.g., reaching a destination).

2. How They Work

- Combine state information (like model-based reflex agents) with goals.
- Choose actions that move the agent toward the goal.
- Sometimes achieving a goal is simple (single action).
- Sometimes it requires planning a sequence of actions (search and planning algorithms).

3. Difference from Reflex Agents

- Reflex agents: action = immediate response to percept (no reasoning about future).
    - **Example:** Brakes when it sees brake lights.
- Goal-based agents: action = chosen because it predicts progress toward a goal.
    - **Example:** Brakes to avoid hitting a car (goal = safety).

4. Advantages

- **Flexible:** Goals can be changed without rewriting all rules.
- **Explicit reasoning:** Agent can consider consequences of actions before choosing.
- Handles complex environments better than simple reflex agents.

5. Structure (Conceptual)

- Sensors → perceive environment
- Model → keep track of world state and effects of actions
- Goals → represent desired outcomes
- Decision → select action predicted to achieve goal
- Actuators → perform action

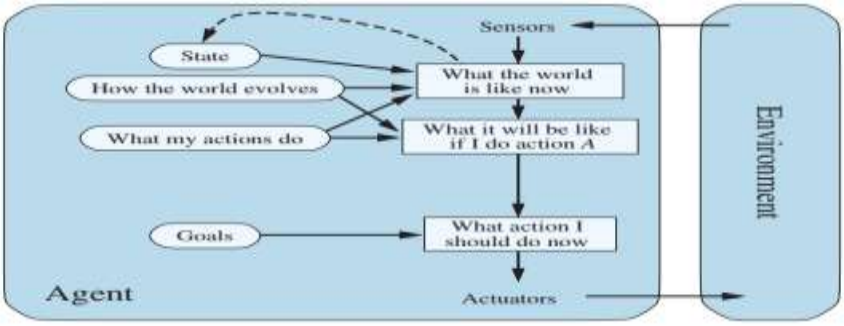


Figure:1.17 Goal-Based Agent with World Model

1.9.4 Utility-based agents

1.Utility and Utility Function

- **Utility:** A numeric measure of how desirable a state is (more precise than "happy").
- **Utility function:** Internal agent measure to evaluate how good each state is.
- Rational agents choose actions to maximize expected utility, aligning internal utility with external performance.

2. Advantages of Utility-Based Agents

- Handle conflicting goals (e.g., speed vs. safety).

- Manage uncertain outcomes when goals cannot be achieved with certainty.
- Can make flexible and informed decisions in partially observable or nondeterministic environments.

3. Expected Utility

- Expected utility = average utility of possible outcomes, weighted by their probabilities.
- Rational decision-making involves choosing the action with the highest expected utility.

4. Learning Agents

- Extend utility-based or other agents by adding a learning element.
- Performance element: Chooses actions (like a normal agent program).
- Learning element: Updates the performance element to improve behavior over time.
- Allows agents to adapt to unknown environments or unknown utility functions.

5. Model-Free Agents

- Can learn best actions in situations without learning a full model of the environment.

6. Structure of a Utility-Based Agent (Conceptual)

- Sensors → perceive environment
- Model → track world state and action effects
- Utility function → evaluate desirability of states
- Decision → choose action to maximize expected utility
- Actuators → perform action

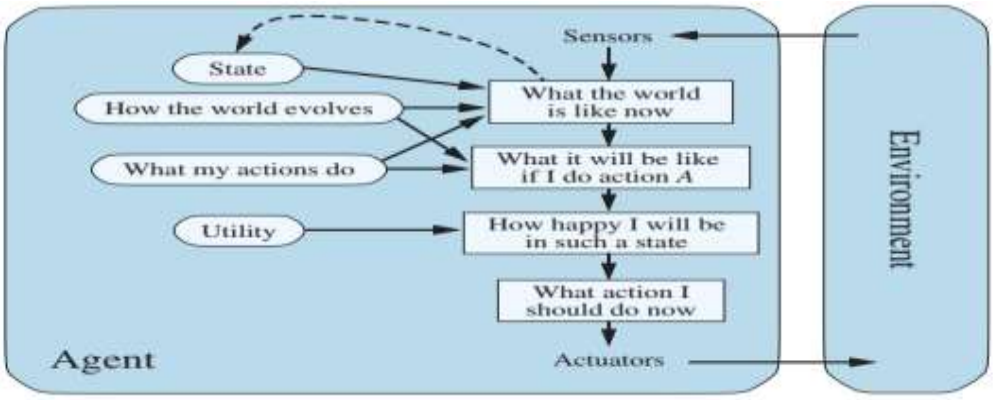


Figure 1.18: Utility-Based Agent with World Model

1.9.5 Learning Agents

1. Why Learning Agents?

Creating intelligent agents entirely by hand is a complex and labor-intensive task. Early discussions, dating back to Alan Turing (1950), highlighted the limitations of manually coding every possible behavior an agent might need. Real-world environments are dynamic, uncertain,

and often unpredictable. If an agent were required to respond to every situation through pre-programmed rules, the system would quickly become unmanageable.

2. Components of a Learning Agent

A learning agent generally consists of four main components that work together to ensure adaptation and improvement:

- Performance Element – This component is responsible for selecting actions in real time, based on the agent's current knowledge. It functions much like a standard agent program, making decisions and interacting with the environment.
- Learning Element – The learning element is the heart of the agent's adaptability. It improves the performance element by using experience and feedback. Whenever the agent encounters new situations, this element helps it adjust its strategies to achieve better results.
- Critic – The critic evaluates how well the agent is performing. It does so by comparing the agent's behavior against a predefined performance measure or standard. The critic's role is essential because without some form of evaluation, the agent cannot know whether its learning process is leading to improvement.
- Problem Generator – This component encourages the agent to try out new actions, even if they are not guaranteed to succeed. Exploration is crucial because relying only on past actions may prevent the discovery of better solutions. By suggesting novel behaviors, the problem generator expands the agent's knowledge and increases its ability to adapt in the long run.

3. How Learning Works

The process of learning involves continual interaction between the agent and its environment. The learning element updates the agent's internal components in response to new experiences. For example, the agent may gain new knowledge about:

- What its actions do (cause-and-effect relationships between actions and outcomes).
- How the world evolves (patterns of change in the environment over time).

A key part of this process is exploration. By experimenting with new strategies, the agent can identify approaches that were not obvious at first. Without exploration, the agent might remain stuck with suboptimal solutions.

UNIT-II
PROBLEM SOLVING METHODS

Problem solving Methods - Search Strategies- Uninformed - Informed - Heuristics - Local Search Algorithms and Optimization Problems - Searching with Partial Observations - Constraint Satisfaction Problems — Constraint Propagation - Backtracking Search – Game Playing – Optimal Decisions in Games – Alpha - Beta Pruning.

2.1 Search Strategies

Search is the fundamental technique through which agents explore the space of possible solutions. A search algorithm specifies how nodes (states) are generated, expanded, and evaluated. The performance of search strategies is usually measured in terms of completeness, optimality, time complexity, and space complexity.

Problem Space Search Terminology

Search is one of the fundamental techniques in artificial intelligence for problem solving. To formalize the search process, several key terms are used to describe the elements involved in defining and analyzing problems.

1. Problem Space

The problem space refers to the environment in which the search is carried out. It consists of:

- A set of all possible states that can exist in the problem.
- A set of operators that describe how one state can be transformed into another.

For example, in a puzzle game, every arrangement of the puzzle pieces is a state, and moving a piece is an operator. The entire set of states and operators together forms the problem space.

2. Problem Instance

A problem instance specifies the task to be solved. It is defined by two elements:

- The initial state, which represents the starting point.
- The goal state, which represents the desired outcome.

For example, in a navigation problem, the initial state might be the starting city, and the goal state might be the destination city.

3. Problem Space Graph

The problem space can be represented as a graph, where:

- Nodes correspond to problem states.
- Edges correspond to operators that transform one state into another.

This representation makes it possible to apply graph search algorithms to find a path from the initial state to the goal state.

4. Depth of a Problem

The depth refers to the length of the shortest path (measured in number of operators) from the initial state to the goal state. A shallow depth implies that the solution can be reached with fewer steps, whereas a greater depth indicates more complex problem-solving requirements.

5. Space Complexity

Space complexity measures the maximum number of nodes that must be stored in memory at any point during the search. This is a critical factor, since memory limitations often restrict the feasibility of certain algorithms.

6. Time Complexity

Time complexity measures the total number of nodes generated or expanded during the search. It indicates the computational effort required to reach a solution.

7. Admissibility

An algorithm is said to be admissible if it always guarantees to find an optimal solution whenever one exists. Admissibility ensures that the solution returned is not just any solution but the best possible one, according to the defined cost function.

8. Branching Factor

The branching factor refers to the average number of child nodes generated from a single node in the problem space graph. A higher branching factor indicates that the search space expands rapidly, which can significantly increase both time and space requirements.

Properties of Search Algorithms

To evaluate and compare different search algorithms, researchers consider several essential properties. These properties help determine the suitability of a given algorithm for a particular problem.

1. Completeness

A search algorithm is complete if it guarantees to find a solution whenever one exists. This property is particularly important in practical applications where missing a valid solution is unacceptable.

2. Optimality

A search algorithm is optimal if the solution it returns is guaranteed to be the best among all possible solutions, typically meaning the one with the lowest cost. For example, uniform-cost search is optimal because it always expands the node with the lowest path cost first. Optimality ensures that the solution is not just correct but also efficient in terms of resources.

3. Time Complexity

The time complexity of a search algorithm is the total computational time required to find a solution. It is usually measured by counting the number of nodes generated or expanded during the search process. Algorithms with high time complexity may be impractical for large problem spaces, even if they are complete and optimal.

4. Space Complexity

The space complexity measures the maximum memory required at any point during the search. In many algorithms, space complexity can be even more critical than time complexity. For example, breadth-first search may generate and store a huge number of nodes in memory, making it infeasible for deep or large problem spaces.

Types of search algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.

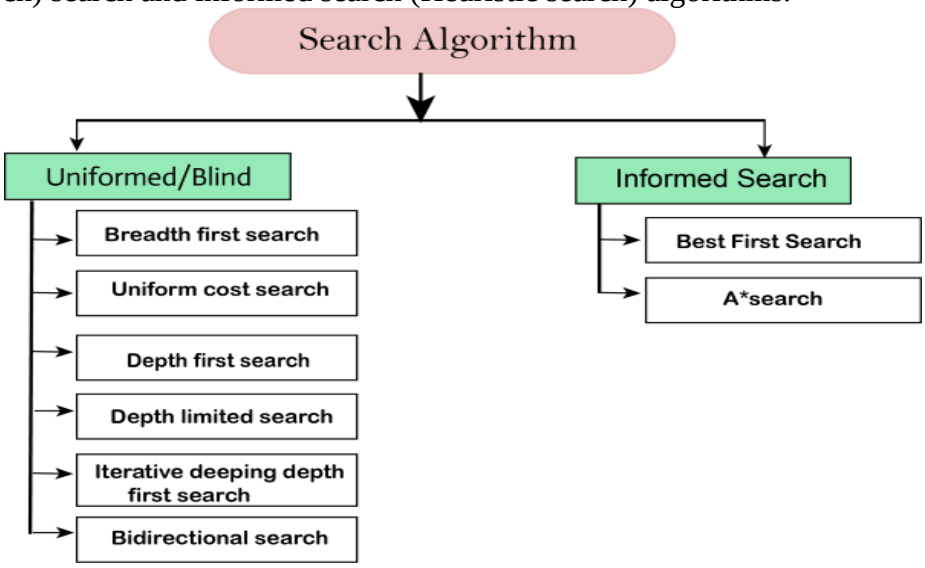


Figure 2.1 Search algorithms

2.2 Uninformed/Blind Search:

The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.

Uninformed Search Algorithms

Uninformed search is a class of general-purpose search algorithms which operates in brute force-way. Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree, so it is also called blind search.

Following are the various types of uninformed search algorithms:

1. Breadth-first Search
2. Depth-first Search
3. Depth-limited Search
4. Iterative deepening depth-first search
5. Uniform cost search
6. Bidirectional Search

1. Breadth-first Search:

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

Advantages:

- BFS will provide a solution if any solution exists.
- If there are more than one solutions for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

Disadvantages:

It requires lots of memory since each level of the tree must be saved into memory to expand the next level.

- BFS needs lots of time if the solution is far away from the root node.

Example:

In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

1. S---> A--->B---->C--->D---->G--->H--->E---->F---->I---->K

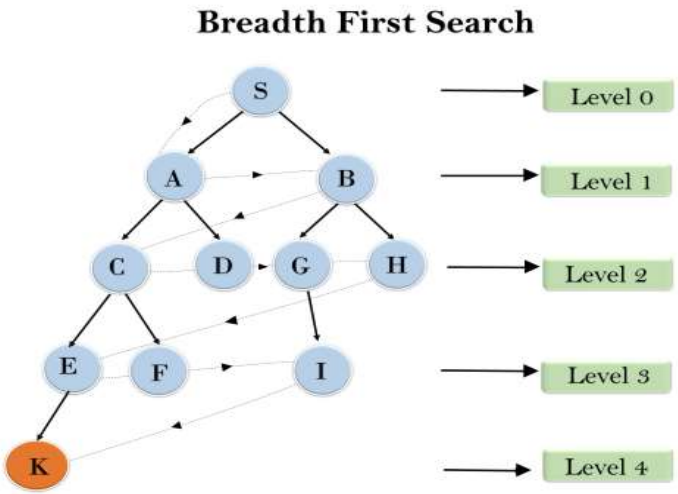


Figure 2.2 Breadth First Search

Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

$T(b) = 1+b^2+b^3+.....+ b^d= O(b^d)$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

2. Depth-first Search

- Depth-First Search (DFS) is a recursive technique used to explore tree or graph data structures.

- It is termed depth-first because it begins at the root node and explores a path as far as possible before backtracking to follow another path.
- The implementation of DFS relies on a **stack** data structure.
- The working principle of DFS resembles that of the Breadth-First Search (BFS) algorithm, though the exploration order differs.

Advantage:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

Disadvantage:

- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Example:

In the below search tree, we have shown the flow of depth-first search, and it will follow the order as: Root node--->Left node ----> right node.

It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.

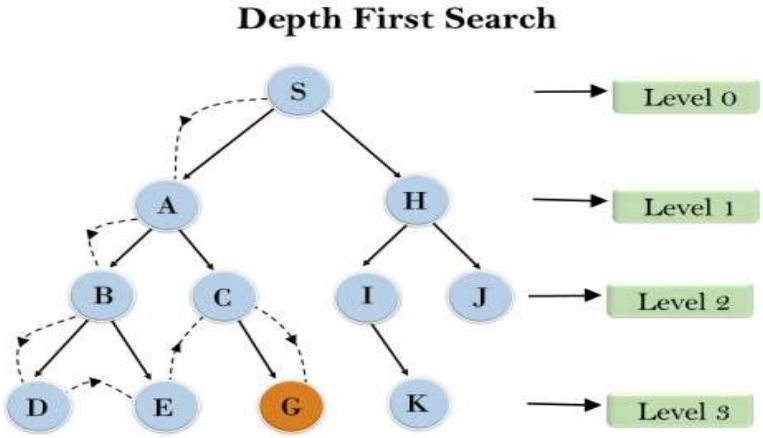


Figure 2.3 Depth First Search

Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$T(n)= 1+ n^2+ n^3 +.....+ n^m=O(n^m)$

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is O(bm).

3. Depth-Limited Search Algorithm:

A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be terminated with two Conditions of failure:

- Standard failure value: It indicates that problem does not have any solution.
- Cutoff failure value: It defines no solution for the problem within a given depth limit.

Advantages:

Depth-limited search is Memory efficient.

Disadvantages:

- Depth-limited search also has a disadvantage of incompleteness.

- It may not be optimal if the problem has more than one solution.

Example:

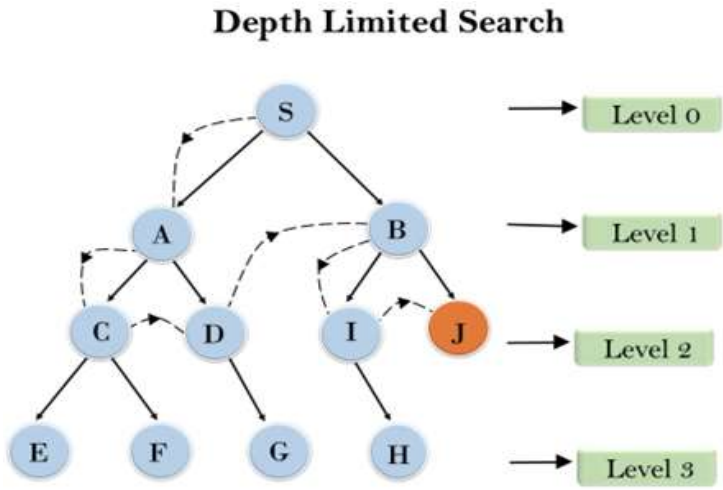


Figure 2.4 Depth Limited Search

Completeness: DLS search algorithm is complete if the solution is above the depth-limit.

Time Complexity: Time complexity of DLS algorithm is $O(b^{\ell})$.

Space Complexity: Space complexity of DLS algorithm is $O(b\times\ell)$.

Optimal: Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $\ell>d$.

4. Uniform-cost Search Algorithm:

Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph. This algorithm comes into play when a different cost is available for each edge. The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost.

Advantages:

- Uniform cost search is optimal because at every state the path with the least cost is chosen.

Disadvantages:

- It does not care about the number of steps involve in searching and only concerned about path cost. Due to which this algorithm may be stuck in an infinite loop.

Example:

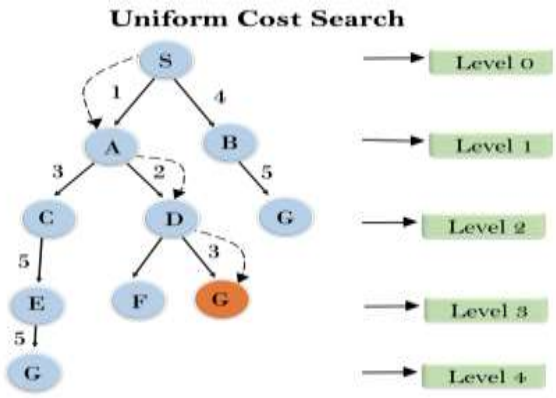


Figure 2.5 Uniform Cost Search

Completeness: Uniform-cost search is complete, such as if there is a solution, UCS will find it.

Time Complexity: Let C* is Cost of the optimal solution, and ε is each step to get closer to the goal node. Then the number of steps is = C*/ε+1. Here we have taken +1, as we start from state 0 and end to C*/ε. Hence, the worst-case time complexity of Uniform-cost search is$O(b^{1+[C^*/\varepsilon]})$/.

Space Complexity: The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $O(b^{1 + [C^*/\varepsilon]})$.

5. Iterative deepening depth-first Search:

The iterative deepening algorithm is a combination of DFS and BFS algorithms. This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

Advantages:

- Itcombines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency.

Disadvantages:

- The main drawback of IDDFS is that it repeats all the work of the previous phase.

Example:

Following tree structure is showing the iterative deepening depth-first search. IDDFS algorithm performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as:

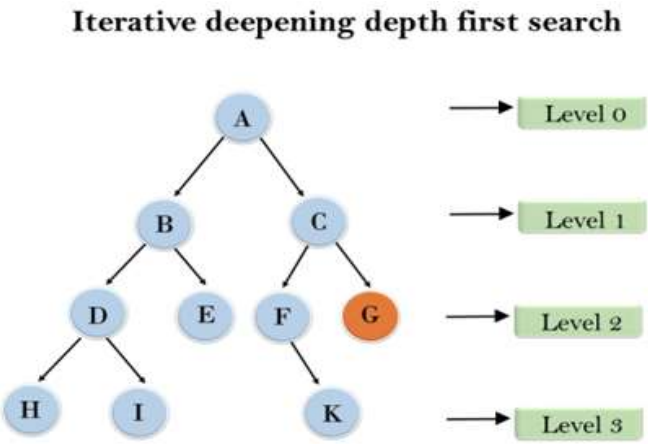


Figure 2.6 Iterative deepening depth first search

1'st Iteration-----> A
2'nd Iteration----> A, B, C
3'rd Iteration------>A, B, D, E, C, F, G
4'th Iteration------>A, B, D, H, I, E, C, F, K, G
In the fourth iteration, the algorithm will find the goal node.

Completeness: This algorithm is complete is ifthe branching factor is finite.

Time Complexity: Let's suppose b is the branching factor and depth is d then the worst-case time complexity is $O(b^d)$.

Space Complexity: The space complexity of IDDFS will be O(bd).

6. Bidirectional Search Algorithm:

Bidirectional search algorithm runs two simultaneous searches, one form initial state called as forward-search and other from goal node called as backward-search, to find the goal node. Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.

Advantages:

- Bidirectional search is fast.
- Bidirectional search requires less memory

Disadvantages:

- Implementation of the bidirectional search tree is difficult.
- In bidirectional search, one should know the goal state in advance.

Example:

In the below search tree, bidirectional search algorithm is applied. This algorithm divides one graph/tree into two sub-graphs. It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction.

The algorithm terminates at node 9 where two searches meet.

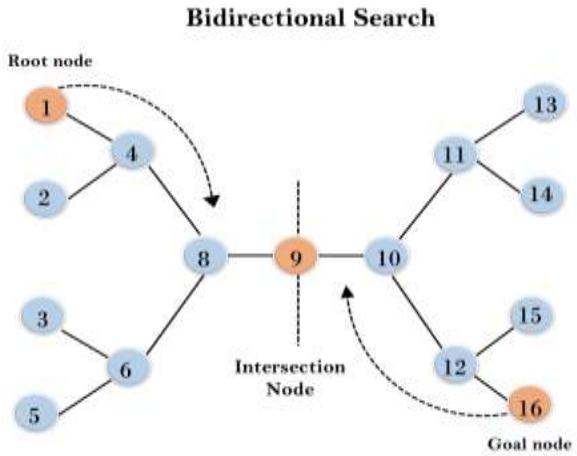


Figure 2.7 Bidirectional Search

Completeness: Bidirectional Search is complete if we use BFS in both searches.
Time Complexity: Time complexity of bidirectional search using BFS is $O(b^d)$.
Space Complexity: Space complexity of bidirectional search is $O(b^d)$.
Optimal: Bidirectional search is Optimal.

2.3 Informed Search

Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a Heuristic search.

1. Best first search (BFS)

This algorithm always chooses the path which appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It lets us to take the benefit of both algorithms. It uses the heuristic function and search. With the help of the best-first search, at each step, we can choose the most promising node.

Best first search algorithm:

Step 1: Place the starting node into the OPEN list.
Step 2: If the OPEN list is empty, Stop and return failure.
Step 3: Remove the node n from the OPEN list, which has the lowest value of h(n), and places it in the CLOSED list.
Step 4: Expand the node n, and generate the successors of node n.
Step 5: Check each successor of node n, and find whether any node is a goal node or not. If any successor node is the goal node, then return success and stop the search, else continue to next step.
Step 6: For each successor node, the algorithm checks for evaluation function f(n) and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both lists, then add it to the OPEN list.
Step 7: Return to Step 2.

2. A* Search Algorithm

A* search is the most commonly known form of best-first search. It uses the heuristic function h(n) and cost to reach the node n from the start state g(n). It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently.

It finds the shortest path through the search space using the heuristic function. This search algorithm expands fewer search tree and gives optimal results faster.

Algorithm of A* search:

Step 1: Place the starting node in the OPEN list.
Step 2: Check if the OPEN list is empty or not. If the list is empty, then return failure and stops.
Step 3: Select the node from the OPEN list which has the smallest value of the evaluation function (g+h). If node n is the goal node, then return success and stop, otherwise.

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list. If not, then compute the evaluation function for n' and place it into the Open list.

Step 5: Else, if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.

Step 6: Return to Step 2.

2.4 Heuristic Search

A heuristic is a technique that is used to solve a problem faster than the classic methods. These techniques are used to find the approximate solution of a problem when classical methods do not. Heuristics are said to be the problem-solving techniques that result in practical and quick solutions.

Why do we need heuristics?

- Heuristics are used in situations in which there is the requirement of a short-term solution.
- The heuristic method might not always provide us the finest solution, but it is assured that it helps us find a good solution in a reasonable time.
- Based on context, there can be different heuristic methods that correlate with the problem's scope.

Heuristic search techniques in AI (Artificial Intelligence)

Figure 2.8 Heuristic search techniques

We can perform the Heuristic techniques into two categories:

Direct Heuristic Search techniques in AI

It includes Blind Search, Uninformed Search, and Blind control strategy. These search techniques are not always possible as they require much memory and time. These techniques search the complete space for a solution and use the arbitrary ordering of operations.

Example:

AO*algorithm

AO* algorithm is a best first search algorithm. AO* algorithm uses the concept of AND-OR graphs to decompose any complex problem given into smaller set of problems which are further solved. AND-OR graphs are specialized graphs that are used in problems that can be broken down into sub problems where AND side of the graph represent a set of tasks that need to be done to achieve the main goal, whereas the or side of the graph represent the different ways of performing task to achieve the same main goal.

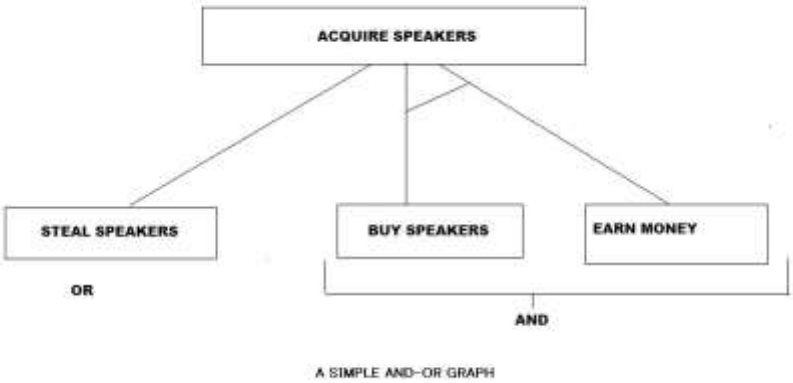


Figure 1.9 A Simple AND-OR Graph

In the above figure we can see an example of a simple AND-OR graph wherein, the acquisition of speakers can be broken into sub problems/tasks that could be performed to finish the main goal. The sub task is to either steal speakers which will directly helps us achieve the main goal "or" earn some money "and" buy speakers which helps us achieve the main goal.

Working of AO algorithm:

The AO* algorithm works on the formula given below:

f(n) = g(n) + h(n)

where,

- g(n): The actual cost of traversal from initial state to the current state.
- h(n): The estimated cost of traversal from the current state to the goal state.
- f(n): The actual cost of traversal from the initial state to the goal state.

Now, to get a better idea of the AO* algorithm let's take a look at an example.

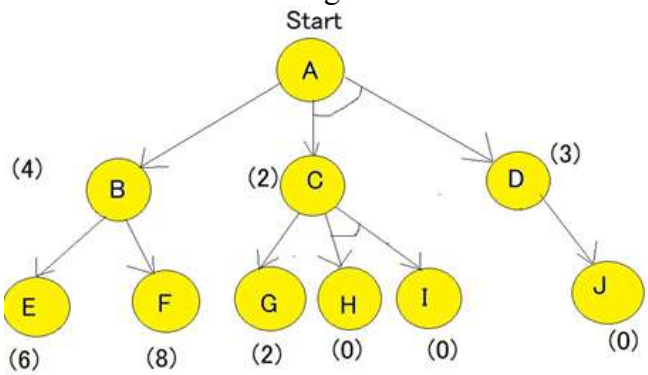


Figure 2.10 Heuristic value in each edge

Step-1

Starting from node A, we first calculate the best path.

f(A-B) = g(B) + h(B) = 1+4= 5 , where 1 is the default cost value of travelling from A to B and 4 is the estimated cost from B to Goal state.

f(A-C-D) = g(C) + h(C) + g(D) + h(D) = 1+2+1+3 = 7 , here we are calculating the path cost as both C and D because they have the AND-Arc. The default cost value of travelling from

A-C is 1, and from A-D is 1, but the heuristic value given for C and D are 2 and 3

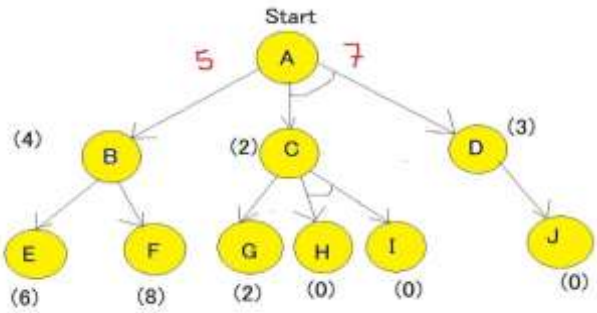


Figure 2.11 heuristic value

The minimum cost path is chosen i.e A-B.

Step-2

Using the same formula as step-1, the path is now calculated from the B node,

f(B-E) = 1 + 6 = 7, f(B-F) = 1 + 8 = 9

f(A-B) = g(B) + updated((h(B)) = 1+7=8

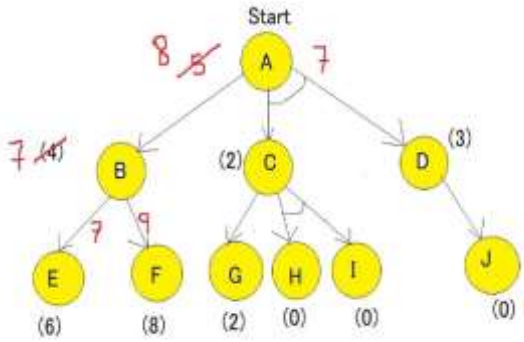


Figure 2.12 Change in heuristic

Step-3

Comparing path of f(A-B) and f(A-C-D) it is seen that f(A-C-D) is smaller. Hence f(A-C-D) needs to be explored.

Now the current node becomes C node and the cost of the path is calculated,

f(C-G) = 1+2 = 3

f(C-H-I) = 1+0+1+0 = 2

f(C-H-I) is chosen as minimum cost path, also there is no change in heuristic since it matches the actual cost. Heuristic of path of H and I are 0 and hence they are solved, but Path A-D also needs to be calculated, since it has an AND-arc.

f(D-J) = 1+0 = 1, hence heuristic of D needs to be updated to 1. And finally the f(A-C-D) needs to be updated.

f(A-C-D) = g(C) + h(C) + g(D) + updated((h(D)) = 1+2+1+1 =5.

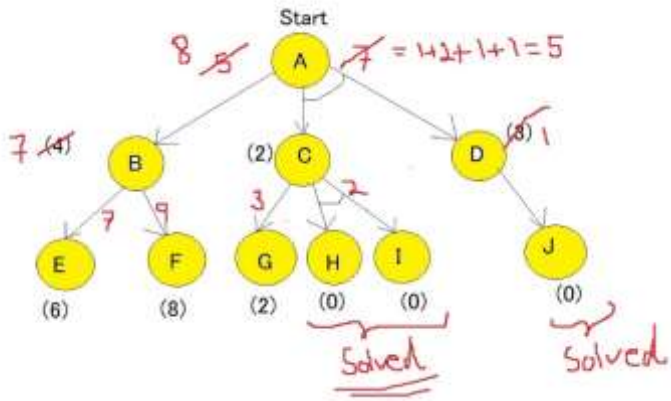


Figure 2.13 Heuristic solved path value f(A-C-D).

As we can see that the solved path is f(A-C-D).

Weak Heuristic Search techniques in AI

It includes Informed Search, Heuristic Search, and Heuristic control strategy. These techniques are helpful when they are applied properly to the right types of tasks. They usually require domain-specific information.

2.5 Local Search Algorithms and Optimization Problems

Local Search

Local search algorithms work differently from systematic search methods. Instead of exploring paths and storing visited states, they focus only on the current state and move to one of its neighboring states. This approach is not guaranteed to cover the whole search space, so it may miss solutions.

local search has two major advantages:

1. It requires very little memory.
2. It can handle very large or even infinite state spaces where systematic algorithms fail.

Optimization Problems

Local search is especially useful for solving **optimization problems**, where the task is to find the best state according to a given **objective function**.

To visualize this, imagine the search space as a **landscape**:

- Each state is a point in the landscape.
- The "height" (or elevation) represents the value of the objective function.
- The aim is to find the **highest peak (global maximum)** or the **lowest valley (global minimum)** depending on the problem.

1. Hill Climbing Algorithm

- It is a technique for optimizing the mathematical problems. Hill Climbing is widely used when a good heuristic is available.
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that. The steps of a simple hill-climbing algorithm are listed below:

Step 1: Evaluate the initial state. If it is the goal state, then return success and Stop.

Step 2: Loop Until a solution is found or there is no new operator left to apply.

Step 3: Select and apply an operator to the current state.

Step 4: Check new state:

If it is a goal state, then return to success and quit.

Else if it is better than the current state, then assign a new state as a current state.

Else if not better than the current state, then return to step2.

Step 5: Exit.

Example:

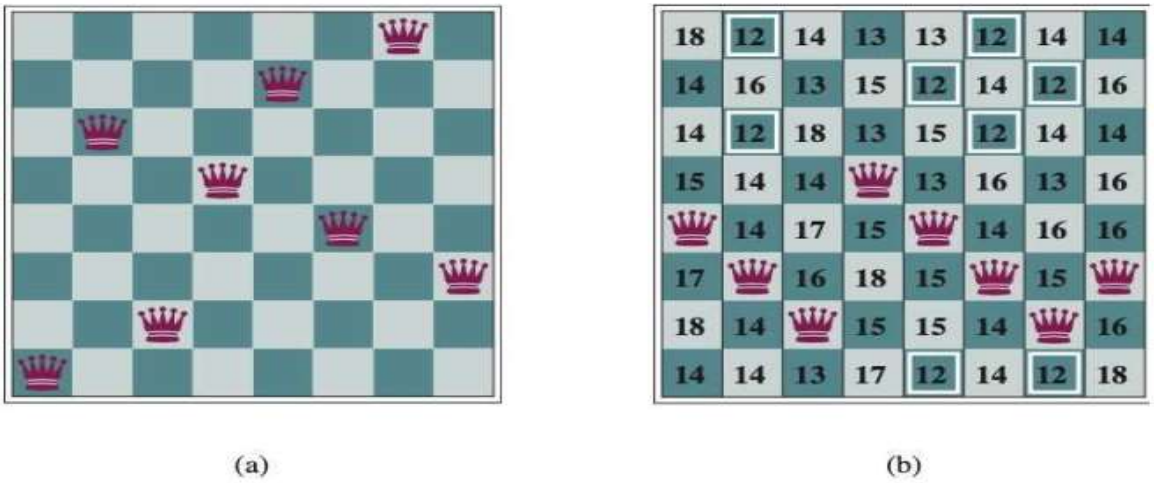


Figure: 2.9 Queen Problem and Heuristic Search for a Solution

(a) Place 8 queens on a chess board so that no queen attacks another. (A queen attacks any piece in the same row, column, or diagonal.) This

position is almost a solution, except for the two queens in the fourth and seventh columns that attack each other along the diagonal. (b) An 8-queens state with heuristic cost estimate *h*=17. The board shows the value of *h* for each possible successor obtained by moving a queen within its column. There are 8 moves that are tied for best, with *h*=12. The hill climbing algorithm will pick one of these.

2. Simulated annealing

Simulated annealing is an algorithm that uses a probabilistic approach to find a good solution to an optimization problem, especially when the problem has many potential solutions that are not the best, known as local optima. It's an improvement on simple hill-climbing algorithms, which can get stuck in these local optima.

Algorithm:

The algorithm's structure is similar to hill climbing, but with a key difference: it allows for "downhill" moves. Instead of always choosing the best move, it makes a random one.

1. **Initial State:** The algorithm starts with a random solution and a high "temperature" (T).
2. **Making a Move:** A new candidate solution is randomly generated by making a small change (a "move") to the current one.
3. **Evaluating the Move:** The change in cost, or "badness" of the move (ΔE), is calculated.
4. **Accepting the Move:**
   - If the move improves the solution (i.e., the cost decreases), it's always accepted.
   - If the move worsens the solution (i.e., the cost increases), it is accepted with a certain probability. This probability is determined by the formula eΔE/T.
5. **Cooling Down:** The "temperature" T is gradually lowered over time according to a defined **schedule**. This makes it less likely for the algorithm to accept bad moves as the process goes on.

- By allowing bad moves, especially at the beginning when the temperature is high, the algorithm can escape from **local optima** and explore different parts of the solution space. As the temperature decreases, the algorithm becomes more like a traditional hill climber, focusing on refining the solution it has found.

3. Local beam search

A potential weakness of local beam search is that its k states can become too similar and cluster together in one area of the search space. This can make the search inefficient, as it's essentially just a slower version of regular hill climbing.

Algorithm:

1. **Parallel Exploration:** At each step, the algorithm considers all the successors from its current k states. This creates a broader set of potential next moves.
2. **Selecting the Best:** From this large set of successors, the algorithm selects the **k best** states to continue the search from in the next iteration. All other states are discarded.
3. **Information Sharing:** Unlike a series of independent random-restart searches, local beam search links its parallel searches.
4. The algorithm effectively shares information, allowing the "most promising" states to influence the overall search. If one path is showing great progress (e.g., has successors with much better values).
5. The algorithm will focus more of its resources on that area by selecting its successors for the next step.
6. This allows it to quickly abandon less fruitful paths and concentrate on the most promising ones.

2.6 Search in Partially Observable Environments

1. Searching with no observation

In a **sensorless** or **conformant problem**, an agent operates in an environment where its percepts provide no information about the current state. This seems impossible at first, but solutions can be found by using a special type of search. Since the agent can't rely on

information from its senses, it must instead find a sequence of actions that guarantees a successful outcome, regardless of the initial state. This type of solution is known as a **coercive plan**.

Searching in Belief-State Space

To solve sensorless problems, we don't search in the space of physical states (which the agent doesn't know) but rather in the **belief-state space**. A **belief state** is the set of all possible physical states the agent could be in.

- **States:** The states in this search space are **subsets of the physical states**. For a problem with N physical states, there are up to 2N possible belief states.
- **Initial State:** The initial belief state is the set of all possible physical states, representing complete uncertainty.
- **Actions:** The available actions at a belief state are the actions that are legal in **at least one** of the physical states within that belief state. If an illegal action could be dangerous, a safer approach is to only allow actions that are legal in **all** physical states.
- **Transition Model:** When a deterministic action is taken, the new belief state is the set of all physical states that could be reached by applying that action to each of the states in the current belief state.
- **Goal Test:** The goal is necessarily achieved when **every** state in the current belief state satisfies the goal condition.
- **Efficiency:** Solving these problems is hard due to the immense size of the belief-state space. Algorithms can improve efficiency by pruning redundant belief states. If one belief state is a subset of another, the superset can be pruned because any solution for it will also work for the subset.

2. Partially Observable Environments

In a **partially observable** environment, the agent receives some, but not all, information about its state. The planning process for this type of problem involves three stages:

1. **Prediction:** The agent first predicts the new belief state after taking an action, just like in a sensorless problem. This predicted belief state represents all possible physical states that could result from the action.
2. **Possible Percepts:** Next, the agent identifies all possible percepts it could receive in the predicted belief state.
3. **Update:** For each possible percept, the agent updates its belief state, reducing its uncertainty. The new belief state is the subset of the predicted belief state that is consistent with the received percept. This is how the agent gains information over time.

3. Solving partially observable problems

To solve **partially observable problems**, where an agent receives some information but not a complete picture of its environment, the **AND–OR search algorithm** is a powerful tool. Instead of searching through physical states (which the agent doesn't know), it searches through **belief states**, where each belief state represents the set of all possible physical states.

The AND–OR algorithm is used to create a **conditional plan**, which is a sequence of actions that includes contingencies based on percepts.

- **AND Nodes:** These nodes represent the uncertainty of the environment. When an action is taken, there might be several possible outcomes. The algorithm must find a solution for **every** possible outcome.
- **OR Nodes:** These nodes represent a choice of action for the agent to make. At an OR node, the agent can choose one action from a set of options, and a solution is found if that action leads to a successful outcome.

Optimizing the Search

Standard AND–OR search can be inefficient because of the large number of belief states. To improve performance:

1. **Subset and Superset Pruning:** The algorithm can discard (prune) a belief state if it's a **superset** of a belief state that has already been explored. This is because a solution

that works for the larger, more uncertain state will also work for the smaller, more specific one.

2. **Incremental Search:** This approach focuses on solving the problem one physical state at a time. By finding a plan that works for a small subset of the belief state and incrementally expanding it to cover more states, it can quickly detect failures and achieve a significant speedup.

An agent for partially observable environments

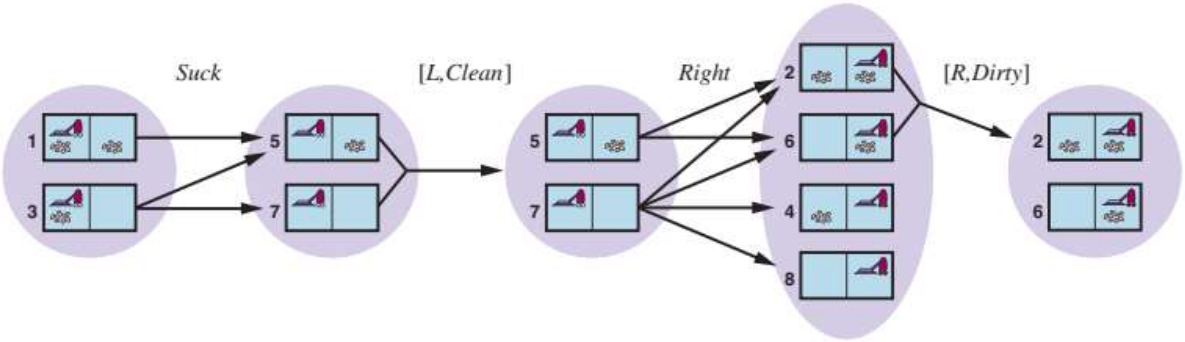


Figure:2.10 Two prediction–update cycles of belief-state maintenance in the kindergarten vacuum world with local sensing.

Leftmost Cycle: The process starts with a belief state containing two possible physical states (1 and 3). The agent takes the action "Suck," leading to a new predicted belief state containing states 5 and 7. The agent then receives the percept [L,Clean], which is consistent with both states. The updated belief state remains states 5 and 7.

Rightmost Cycle: The agent then takes the action "Right." This action is nondeterministic, so the agent's belief state expands to include four possible new physical states (2, 6, 4, and 8). The agent then receives the percept [R,Dirty]. This percept is only consistent with two of the predicted states (2 and 6). The agent updates its belief state to reflect this new information, and the set of possible states is reduced to {2, 6}. This illustrates how the agent uses a sequence of actions and percepts to reduce its uncertainty about the environment.

2.7 Constraint Satisfaction Problems

Constraint Satisfaction Problems (CSPs) offer a different way to think about and solve search problems. Instead of viewing states as single, indivisible "black boxes," CSPs represent a state as a set of variables, each with an assigned value. The goal is to find a set of values for all variables that satisfies a given set of constraints.

Defining a CSP

A CSP is defined by three main components:

- Variables (X): A set of variables that need to be assigned a value.
- Domains (D): A set of possible values for each variable.
- Constraints (C): A set of rules that specify which combinations of values are allowed for the variables.

A constraint is made up of two parts: its scope, which is the set of variables it applies to, and its relation, which defines the allowable combinations of values for those variables. For example, in a problem with variables X1 and X2, a constraint might be X1>X2.

Solving a CSP

The process involves finding a valid assignment of values to variables.

- Consistent/Legal Assignment: An assignment that does not violate any of the constraints.
- Complete Assignment: An assignment where every variable has been given a value.
- Solution: A consistent and complete assignment.

A partial assignment is one where some variables are still unassigned. The goal of a CSP search algorithm is to efficiently find a complete and consistent assignment. These algorithms are powerful because they can eliminate large portions of the search space by identifying and avoiding variable/value combinations that violate constraints early on.

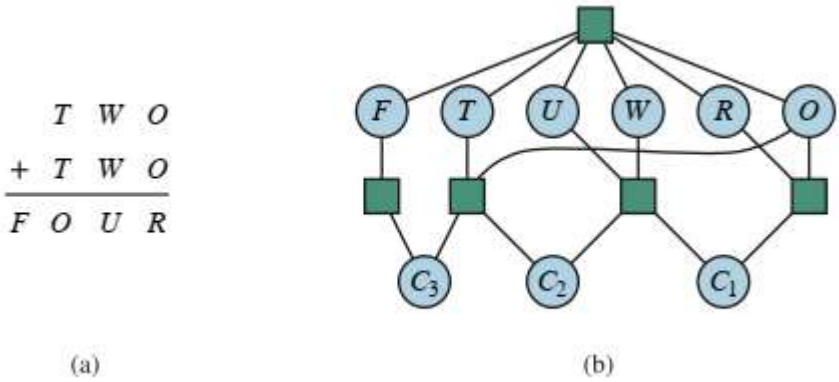


Figure 2.11 (a) A cryptarithmetic puzzle in which each letter corresponds to a unique digit. The objective is to assign digits to letters so that the resulting sum is mathematically correct, with the additional restriction that no leading digit can be zero.
(b) The corresponding constraint hypergraph representation. The top square denotes the Alldiff constraint, ensuring all letters take distinct values, while the four middle squares represent the column-wise addition constraints. The auxiliary variables C1,C2,C3 indicate the carry values for the units, tens, and hundreds columns, respectively.
A cryptarithmetic puzzle is a type of mathematical puzzle where:

- Each letter represents a unique digit (0–9).
- No two letters can represent the same digit.
- The assignment must satisfy the arithmetic operation exactly.
- Leading letters cannot be zero (e.g., T and F here).

TWO+TWO=FOUR

Constraints: AllDiff(T,W,O,F,U,R)
→ all letters must map to different digits.
Column-wise addition constraints:
From right to left:

- **Units place:** O+O=R+10·C1
- **Tens place:** C1+W+W=U+10·C2
- **Hundreds place:** C2+T+T=O+10·C3
- **Thousands place:** C3=F where C1,C2,C3 are carry digits.
- Since **FOUR has more digits than TWO**, F must be **1** (the carry into the thousands place).
- F=1 The word **FOUR** is approximately twice **TWO**.

T=7, W=6, O=5, F=1, U=3, R=0
765+765=1530
TWO+TWO=FOUR

1 . Constraint Propagation: Inference in CSPs

Constraint Satisfaction Problems (CSPs) provide a framework where solutions are discovered by assigning values to variables while satisfying given constraints. Instead of brute-force searching through all possibilities, inference techniques—particularly **constraint propagation**—help reduce the search space significantly.
Constraint propagation is an inference technique used in CSPs.

- Instead of guessing values for variables blindly, we use the constraints to eliminate impossible values from variable domains.
- This reduces the search space and often makes the problem easier—or sometimes solves it completely.

Example

Sudoku is a **constraint satisfaction puzzle** played on a 9×9 grid. The rules are:

1. Each row must contain the digits **1–9** without repetition.
2. Each column must contain the digits **1–9** without repetition.

3. Each 3×3 subgrid must also contain the digits **1–9** without repetition.

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

(b)

Figure 2.12 (a) A Sudoku puzzle and (b) its solution.

2. Backtracking Search for CSPs

Constraint propagation alone is often not enough to solve a CSP completely—some variables may still have multiple possible values. In such cases, we need search. The most widely used search method for CSPs is backtracking search, a depth-first search that builds partial assignments step by step.

Example: Map Coloring of Australia

Figure 2.13 Map Coloring Problem

We want to color the regions of Australia with three colors (red, green, blue) so that no two adjacent regions share the same color.

Step 1: Assign WA = red

- WA (Western Australia) is given red.
- Using forward checking, we eliminate red from NT and SA (its neighbors).

Step 2: Assign NT = green

- NT can only be green now.
- This forces Q (Queensland) to avoid green.

Step 3: Assign Q = blue

- Q is colored blue.
- Now SA (South Australia) is left with only one option: blue.

Step 4: Conflict occurs

- But SA already neighbors Q (blue). This is a contradiction.

Step 5: Backtrack

- We backtrack and try another value for Q.
- Assign Q = red instead of blue.
- This leads to a consistent coloring:
  WA = red, NT = green, Q = red, SA = blue, NSW = green, V = red, T = any color.

2.8 Game Playing

- Game Playing is an important domain of artificial intelligence. Games don't require much knowledge; the only knowledge we need to provide is the rules, legal moves and the conditions of winning or losing the game.
- Both players try to win the game. So, both of them try to make the best move possible at each turn. Searching techniques like BFS(Breadth First Search) are not accurate for this as the branching factor is very high, so searching will take a lot of time. So, we need another search procedures that improve –
    - **Generate procedure** so that only good moves are generated.
    - **Test procedure** so that the best move can be explored first.

The most common search technique in game playing is Minimax search procedure. It is depth-first depth-limited search procedure. It is used for games like chess and tic-tac-toe.

Minimax algorithm uses two functions:

MOVEGEN : It generates all the possible moves that can be generated from the current position.

STATIC EVALUATION : It returns a value depending upon the goodness from the viewpoint of two-player

This algorithm is a two player game, so we call the first player as PLAYER1 and second player as PLAYER2. The value of each node is backed-up from its children. For PLAYER1 the backed-up value is the maximum value of its children and for PLAYER2 the backed-up value is the minimum value of its children. It provides most promising move to PLAYER1, assuming that the PLAYER2 has make the best move. It is a recursive algorithm, as same procedure occurs at each level.

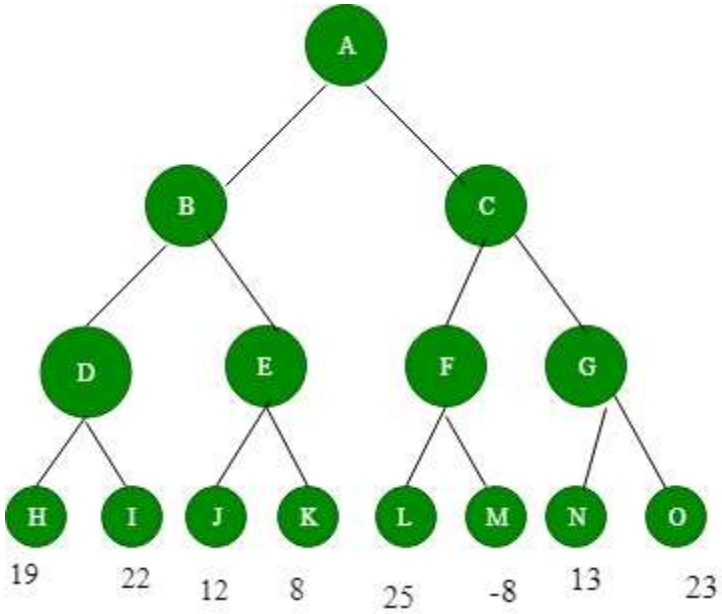


Figure 2.14 Before backing-up of values

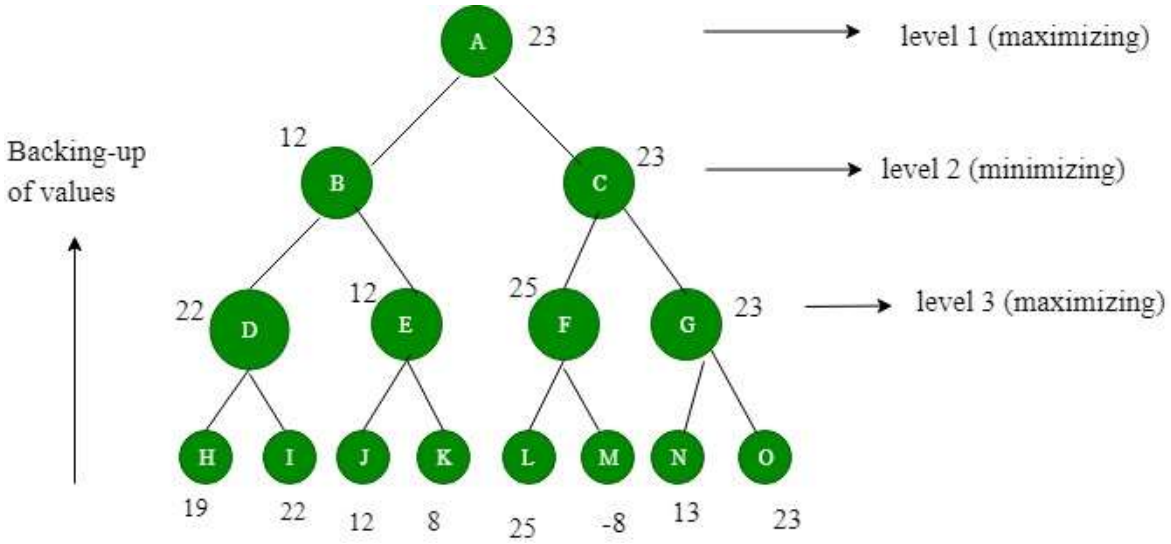


Figure 2.15 After backing-up of values

We assume that PLAYER1 will start the game. 4 levels are generated. The value to nodes H, I, J, K, L, M, N, O is provided by STATIC EVALUATION function. Level 3 is maximizing level, so all nodes of level 3 will take maximum values of their children. Level 2 is minimizing level, so all its nodes will take minimum values of their children. This process continues. The value of A is 23. That means A should choose C move to win.

2.9 Optimal Decisions in Games

Game trees Minimax algorithm

- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory. It provides an optimal move for the player assuming that opponent is also playing optimally.
- Mini-Max algorithm uses recursion to search through the game-tree.
- Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, go, and various tow-players game. This Algorithm computes the minimax decision for the current state.
- In this algorithm two players play the game, one is called MAX and other is called MIN.
- Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
- The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree.
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

Pseudo-code for MinMax Algorithm:

```
function minimax(node, depth, maximizingPlayer) is
if depth ==0 or node is a terminal node then
return static evaluation of node
if MaximizingPlayer then      // for Maximizer Player
maxEva= -infinity
for each child of node do
```

```
eva= minimax(child, depth-1, false)
maxEva= max(maxEva,eva)        //gives Maximum of the values
return maxEva
else                    // for Minimizer player
minEva= +infinity
for each child of node do
eva= minimax(child, depth-1, true)
minEva= min(minEva, eva)        //gives minimum of the values
return minEva
```

Initial call:
Minimax(node, 3, true)

Working of Min-Max Algorithm:

- The working of the minimax algorithm can be easily described using an example. Below we have taken an example of game-tree which is representing the two-player game.
- In this example, there are two players one is called Maximizer and other is called Minimizer.
- Maximizer will try to get the Maximum possible score, and Minimizer will try to get the minimum possible score.
- This algorithm applies DFS, so in this game-tree, we have to go all the way through the leaves to reach the terminal nodes.
- At the terminal node, the terminal values are given so we will compare those value and backtrack the tree until the initial state occurs. Following are the main steps involved in solving the two-player game tree:

Step-1: In the first step, the algorithm generates the entire game-tree and apply the utility function to get the utility values for the terminal states. In the below tree diagram, let's take A is the initial state of the tree. Suppose maximizer takes first turn which has worst-case initial value =- infinity, and minimizer will take next turn which has worst-case initial value = +infinity.

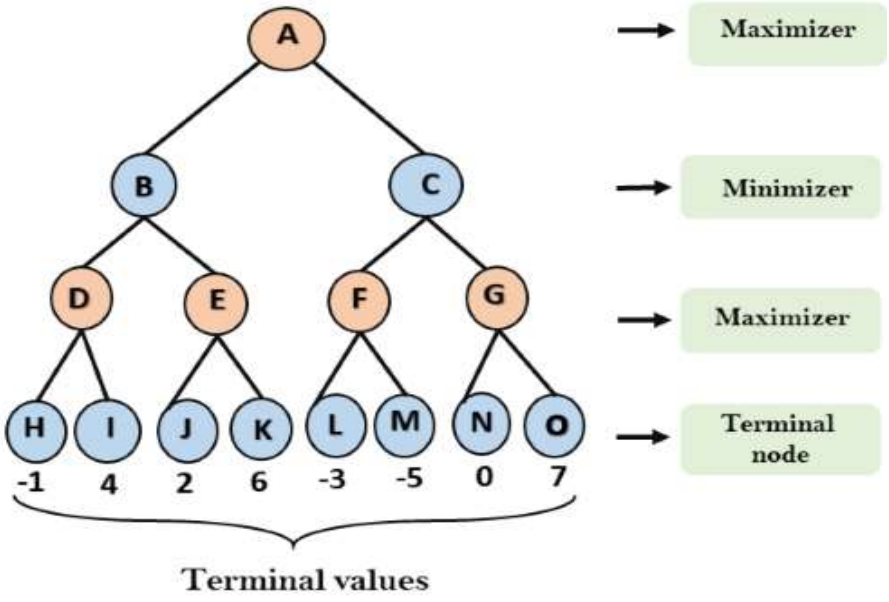


Figure 2.16 Tree with Terminal Values

Step 2: Now, first we find the utilities value for the Maximizer, its initial value is -∞, so we will compare each value in terminal state with initial value of Maximizer and determines the higher nodes values. It will find the maximum among the all.

For node D max(-1,- -∞) => max(-1,4)= 4

- For Node E max(2, -∞) => max(2, 6)= 6
- For Node F max(-3, -∞) => max(-3,-5) = -3
- For node G max(0, -∞) = max(0, 7) = 7

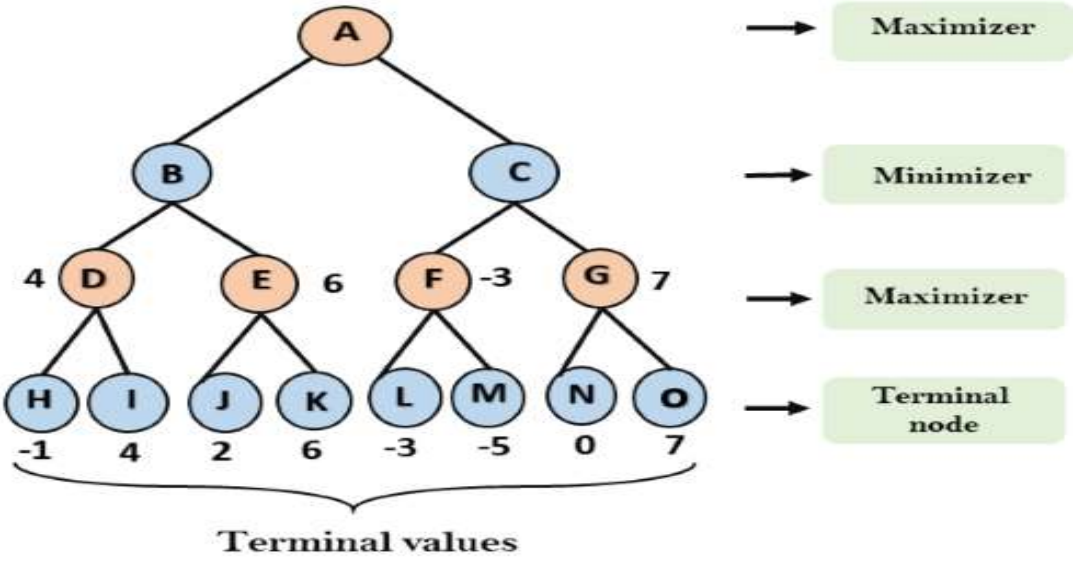


Figure 2.17 Tree with root values and Terminal Values

Step 3: In the next step, it's a turn for minimizer, so it will compare all nodes value with +∞, and will find the 3rd layer node values.

- For node B= min(4,6) = 4
- For node C= min (-3, 7) = -3

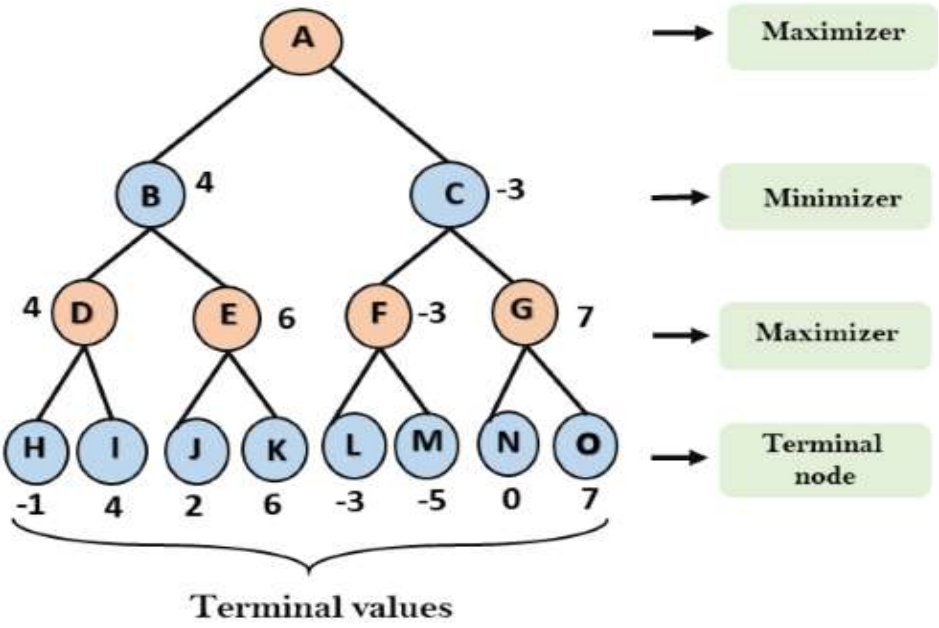


Figure 2.18 Minimizer Tree with root and Terminal Values

Step 4: Now it's a turn for Maximizer, and it will again choose the maximum of all nodes value and find the maximum value for the root node. In this game tree, there are only 4 layers, hence we reach immediately to the root node, but in real games, there will be more than 4 layers.

- For node A max(4, -3)= 4

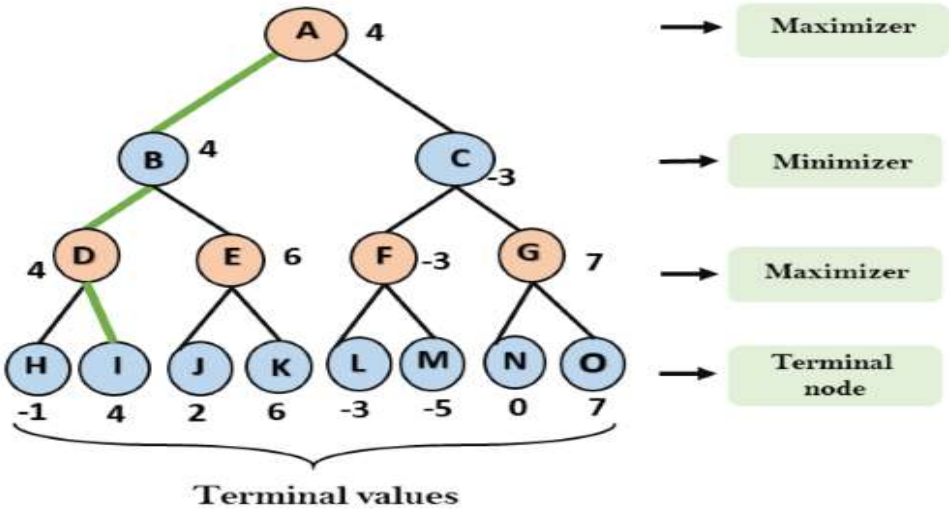


Figure 2.19 Complete Tree with root values and terminal Values

That was the complete workflow of the minimax two player game.

Properties of Mini-Max algorithm:

- **Complete-** Min-Max algorithm is Complete. It will definitely find a solution (if exist), in the finite search tree.
- **Optimal-** Min-Max algorithm is optimal if both opponents are playing optimally.
- **Time complexity-** As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is **O(b^m)**, where b is branching factor of the game-tree, and m is the maximum depth of the tree.
- **Space Complexity-** Space complexity of Mini-max algorithm is also similar to DFS which is **O(bm)**.

Limitation of the minimax Algorithm:

The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess, go, etc. This type of games has a huge branching factor, and the player has lots of choices to decide. This limitation of the minimax algorithm can be improved from **alpha-beta pruning** which we have discussed in the next topic.

2.10 Alpha-Beta Pruning

- Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.
- As we have seen in the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree. Since we cannot eliminate the exponent, but we can cut it to half. Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called pruning. This involves two threshold parameter Alpha and beta for future expansion, so it is called alpha-beta pruning. It is also called as Alpha-Beta Algorithm**.**
- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree.

The two-parameter can be defined as:

1. **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is -∞.

2. **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is +∞.

- The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.

Condition for Alpha-beta pruning:

The main condition which required for alpha-beta pruning is:

1. α>=β

Key points about alpha-beta pruning:

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes.

Pseudo-code for Alpha-beta Pruning:

```
function minimax(node, depth, alpha, beta, maximizingPlayer) is
if depth ==0 or node is a terminal node then
return static evaluation of node
if MaximizingPlayer then      // for Maximizer Player
   maxEva= -infinity
   for each child of node do
   eva= minimax(child, depth-1, alpha, beta, False)
  maxEva= max(maxEva, eva)
  alpha= max(alpha, maxEva)
   if beta<=alpha
 break
 return maxEva
else                    // for Minimizer player
   minEva= +infinity
   for each child of node do
   eva= minimax(child, depth-1, alpha, beta, true)
   minEva= min(minEva, eva)
   beta= min(beta, eva)
    if beta<=alpha
  break
 return minEva
```

Working of Alpha-Beta Pruning:

Let's take an example of two-player search tree to understand the working of Alpha-beta pruning

Step 1: At the first step the, Max player will start first move from node A where α= -∞ and β= +∞, these value of alpha and beta passed down to node B where again α= -∞ and β= +∞, and Node B passes the same value to its child D.

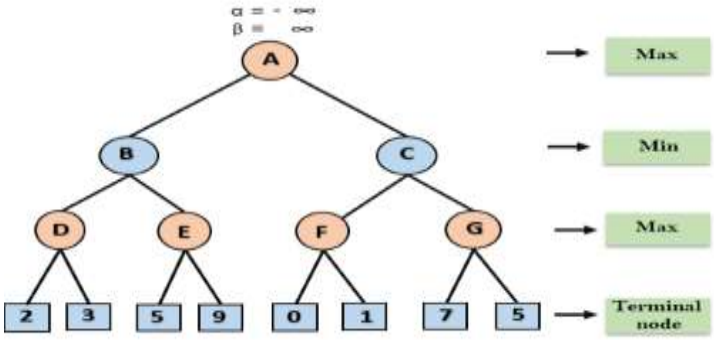


Figure 2.20 Working of Alpha-Beta Pruning

Step 2: At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and node value will also 3.
Step 3: Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now β= +∞, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3, hence at node B now α= -∞, and β= 3.

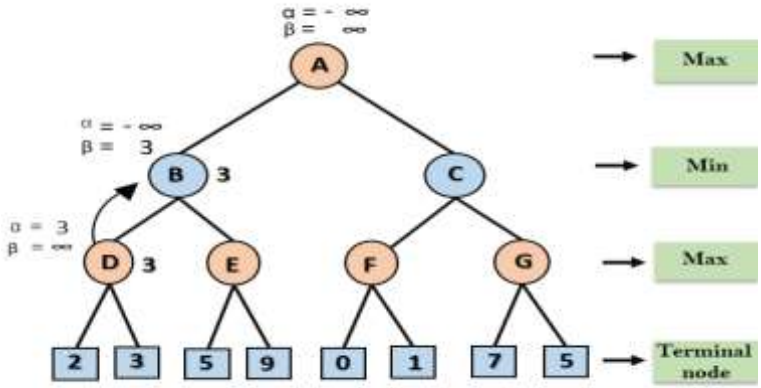


Figure 2.21 Working of Alpha-Beta Pruning with min max terminal nodes

In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α= -∞, and β= 3 will also be passed.
Step 4: At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so max (-∞, 5) = 5, hence at node E α= 5 and β= 3, where α>=β, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.

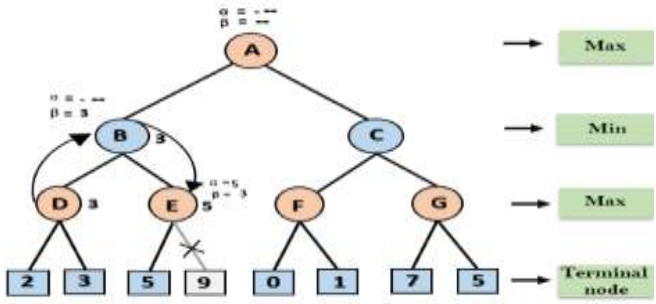


Figure 2.22 left root Min-Max available value

Step 5: At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as max (-∞, 3)= 3, and β= +∞, these two values now passes to right successor of A which is Node C.
At node C, α=3 and β= +∞, and the same values will be passed on to node F.
Step 6: At node F, again the value of α will be compared with left child which is 0, and max(3,0)= 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.

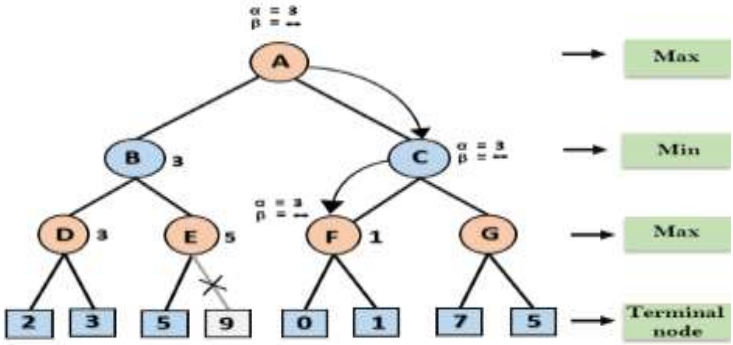


Figure 2.23 right root Min-Max available value

Step 7: Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed, it will compare with 1 so min (∞, 1) = 1. Now at C, α=3 and β= 1, and again it satisfies the condition α>=β, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.

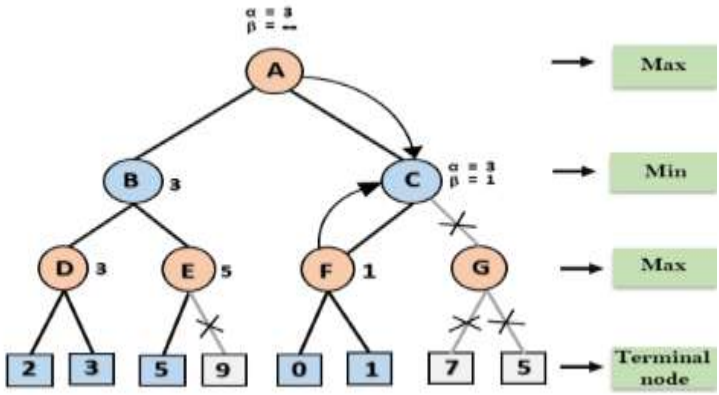


Figure 2.24 left and right root Min-Max available value

Step 8: C now returns the value of 1 to A here the best value for A is max (3, 1) = 3. Following is the final game tree which is the showing the nodes which are computed and nodes which has never computed. Hence the optimal value for the maximizer is 3 for this example.

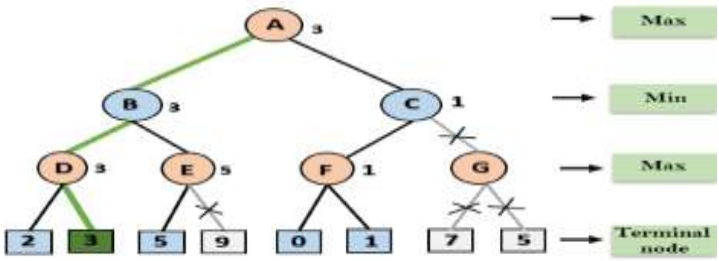


Figure 2.25 Alpha-Beta pruning

Move Ordering in Alpha-Beta pruning:

The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined. Move order is an important aspect of alpha-beta pruning.

It can be of two types:

- **Worst ordering:** In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm. In this case, it also consumes more time because of alpha-beta factors, such a move of pruning is called worst ordering. In this case, the best move occurs on the right side of the tree. The time complexity for such an order is $O(b^m)$.
- **Ideal ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree. We apply DFS hence it first search left of the tree and go deep twice as minimax algorithm in the same amount of time. Complexity in ideal ordering is $O(b^{m/2})$.

Rules to find good ordering:

Following are some rules to find good ordering in alpha-beta pruning:

- Occur the best move from the shallowest node.
- Order the nodes in the tree such that the best nodes are checked first.
- Use domain knowledge while finding the best move. Ex: for Chess, try order: captures first, then threats, then forward moves, backward moves.
- We can bookkeep the states, as there is a possibility that states may repeat.

UNIT – III KNOWLEDGE REPRESENTATION

3.1 PREDICATE LOGIC

Predicate logic, also known as first-order logic (FOL), extends propositional logic by allowing AI systems to represent relationships between objects and their properties. This makes predicate logic a powerful tool for knowledge representation and reasoning. It enables AI systems to understand relationships like "John is the father of Mary" or "All humans are mortal."

Role of Predicate Logic in AI

It provides a structure for representing complex facts about objects and their relationships in a system.

AI systems use predicate logic to infer new information from existing facts, making it suitable for decision-making tasks.

Example:

"John is the father of Mary" can be represented as: Father(John,Mary)Father(John, Mary)Father(John,Mary)

This expression tells us that there is a **relationship** (Father) between two objects (John and Mary). Predicate logic enables us to model and reason about such relationships effectively.

Components of Predicate Logic:

Predicate logic involves several key components that allow it to represent relationships and properties of objects in a structured way.

1. Predicates:

A predicate is a function that returns either true or false based on the relationship between its arguments.

Example:

IsHungry(John)IsHungry(John)IsHungry(John)

This predicate represents whether John is hungry, returning true if he is and false if not.

2. Variables:

Variables are placeholders for objects within a domain. They allow us to represent general statements that apply to multiple objects.

Example*: In IsHungry(x),* the variable x can represent any person.

3. Constants:

Constants represent specific objects or entities in the domain.

Example: John is a constant in the predicate IsHungry(John).

Structure of Predicates

Predicate consists of two key elements: the predicate symbol and arguments. Predicates are enhanced with quantifiers to specify the scope of variables involved.

1. Predicate Symbol

The predicate symbol defines the property or relationship being described.

Example:

IsHungry(x) represents whether a person (x) is hungry.

Married(x, y) denotes that person x is married to person y.

Predicates are named based on the relationship or property they represent. The symbol is followed by arguments enclosed in parentheses.

2. Arguments and Arity

Arguments refer to the specific objects that the predicate is applied to.

The arity of a predicate refers to the number of arguments it takes.

Examples:

IsHungry(x): A predicate with 1 argument (arity = 1).

Married(x, y): A predicate with 2 arguments (arity = 2).

X(a, b, c): A predicate with 3 arguments (arity = 3), representing something like "a + b + c = 0."

3. Quantifiers in Predicate Logic

Quantifiers allow us to specify the scope of variables. There are two main types:

Existential Quantifier (∃)

Meaning: There exists at least one object that satisfies the given condition.

Example: ∃x IsHungry(x)

This statement means that at least one person is hungry.

Negation: The negation of the existential quantifier means that no such object exists.

¬∃x IsHungry(x)

This means that no one is hungry.

Universal Quantifier (∀)

Meaning: The given condition holds for all objects in the domain.

Example: ∀x (IsHuman(x)→IsMortal(x))

This means that all humans are mortal.

Negation: The negation of the universal quantifier means there is at least one exception.

$\neg\forall x\ IsHuman(x) \rightarrow IsMortal(x)$

This implies that at least one human is not mortal.

Examples of Predicate Logic

1. Simple Predicate Example:

Predicate: IsHungry(John)

This predicate represents the state of whether John is hungry. It takes one argument (John) and returns true if John is hungry, otherwise false.

In NLP-based chatbots, predicates like this could help infer the user's intent. For example, if a chatbot detects that the user is hungry, it could suggest nearby restaurants.

2. Equality Predicate Example:

Predicate: $E(x,y) \equiv (x=y)$

This predicate denotes that x is equal to y. It returns true if the two objects are identical.

AI-based reasoning systems use equality predicates to match objects. For example, in a robot warehouse, a robot may use this predicate to determine if an object picked matches the one requested (e.g., E(Package1, RequestedItem)).

3. Mathematical Predicate Example:

Predicate: $X(a,b,c) \equiv (a+b+c=0)$

This predicate checks whether the sum of a, b, and c equals zero. It returns true if the equation holds, otherwise false.

In optimization problems, AI models might use mathematical predicates to check if constraints are satisfied. For example, in scheduling systems, such predicates can validate if certain conditions are met (e.g., X(shiftA, shiftB, totalTime) checks if the total shift hours are balanced).

4. Relationship Predicate Example:

Predicate: $M(x,y) \equiv$ x is married to y

This predicate expresses a relationship between two objects, indicating that x is married to y.

In family tree AI systems, relationship predicates are used to infer relationships among family members. For instance, if M(John, Mary) is true, the system can infer that John is Mary's spouse.

5. Universal Quantification Example:

Expression: $\forall x\ (IsHuman(x) \rightarrow IsMortal(x))$

This statement reads as "For all x, if x is human, then x is mortal." It applies to every object in the domain of humans. If an object is found to be human, it must also be mortal for the statement to hold true.

Knowledge-based systems use such rules to infer properties about objects. For example, in medical diagnosis systems, rules like "All viruses can spread infections" (∀x IsVirus(x) → CanSpreadInfection(x)) help the system reason about diseases.

6. Existential Quantification Example:

Expression: ∃x IsHungry(x)

This reads as "There exists at least one x such that x is hungry." It indicates that at least one object in the domain satisfies the condition of being hungry.

In robot planning, a robot could use existential quantifiers to plan actions. For example, "There exists a task that requires charging" (∃x TaskRequires(x, Charging)) might guide the robot to prioritize charging tasks.

7. Compound Example with Multiple Quantifiers:

Expression: ∀x∃y (Parent(x,y))

This statement means "For every person x, there exists a person y such that x is the parent of y." It shows how multiple quantifiers can be used together to represent complex relationships.

Example: Order of Quantifiers Matters

∀x∃y (Parent(x,y))

Meaning: For every person x, there exists at least one person y such that x is the parent of y.

Example in AI: In a family tree system, this could represent the rule that every parent must have at least one child.

Examples of AI Applications Using Predicate Logic

Natural Language Processing (NLP): NLP systems use predicate logic to understand relationships in text and answer complex queries. For example, in a chatbot, predicate logic helps infer the user's intent.

Robot Planning Systems: In a warehouse, robots use predicate logic to plan tasks based on conditions like:

∃x TaskRequires(x,Charging)

This ensures robots prioritize tasks that require charging.

Expert Systems:

Predicate logic is used in expert systems (like medical diagnosis tools) to store if-then rules and infer new facts about diseases based on symptoms.

3.2 PROPOSITIONAL LOGIC

Propositional Logic (PL) is a branch of logic that focuses on statements (propositions) that can be either true or false. It is also known as Boolean logic since the truth values are binary—either True (1) or False (0).

In AI, propositional logic forms the foundation for logical reasoning, allowing systems to represent facts and rules about a problem domain. These rules help the system infer new information or make decisions based on the given inputs. Propositional logic simplifies knowledge representation by breaking down reasoning into atomic statements or propositions.

Example:

An AI system used in home automation might have propositions such as:

P: “The light is on.”

Q: “The window is open.”

Using logical connectives, the system can combine these propositions to represent more complex statements like:

“If the light is on and the window is open, turn off the light.”

Basic Facts about Propositional Logic

1. Propositions are Declarative Statements:

In propositional logic, each statement, known as a proposition, is either True or False.

Example:

P: “It is raining.” (True or False)

Q: “The heater is on.” (True or False)

2. Atomic Propositions:

These are simple, indivisible statements that cannot be broken down further. Each atomic proposition represents a basic fact or condition.

Example: “The door is closed.”

3. Compound Propositions:

Multiple atomic propositions can be combined using logical connectives (like AND, OR, NOT) to create compound propositions.

Example: “The door is closed AND the heater is on.”

4. Binary Truth Values:

Every proposition has a binary truth value: it can only be True (1) or False (0). There are no intermediate states. This simplicity makes propositional logic ideal for clear-cut decisions.

5. Logical Connectives Combine Propositions:

Logical connectives such as AND, OR, NOT, IF-THEN, and IF AND ONLY IF allow us to create more complex propositions from simple ones.

Syntax:

The syntax of propositional logic defines the rules for creating valid propositions. In propositional logic, we combine atomic propositions using logical connectives to form more complex statements, known as compound propositions.

Building Blocks:

Atomic Propositions:

These are basic statements that represent individual facts or conditions.

Example:

P: “It is raining.”

Q: “The heater is on.”

Logical Connectives:

Connectives are used to combine atomic propositions to form compound propositions.

AND ($\wedge$): True if both propositions are true.

OR ($\vee$): True if at least one proposition is true.

NOT ($\neg$): Negates the truth value of a proposition.

IF-THEN ($\rightarrow$): True unless the first proposition is true and the second is false.

IF AND ONLY IF ($\leftrightarrow$): True if both propositions have the same truth value.

Compound Propositions:

These are more complex statements formed by connecting atomic propositions using logical connectives.

Example:

“If it is raining and the heater is on, then the room will be warm.”

This can be written in propositional logic syntax as: $(P \wedge Q) \rightarrow R$

Where:

P: “It is raining.”

Q: “The heater is on.”

R: “The room is warm.”

Example:

Scenario:

P: “It is hot outside.”

Q: “The windows are open.”

R: “Turn on the air conditioner.”

Using propositional logic, we can represent the system’s decision-making with the following compound proposition:

$(P \wedge \neg Q) \rightarrow R$

This logic reads as:

“If it is hot outside AND the windows are not open, then turn on the air conditioner.”

Common Logical Connectives:

Connective	**Symbol**	**Meaning**	**Example**

AND	∧	True if both propositions are true.	(P ∧ Q): "It is raining AND cold."
OR	∨	True if at least one proposition is true.	(P ∨ Q): "It is raining OR cold."
NOT	¬	Negates the truth value of a proposition.	¬P: "It is not raining."
IF-THEN	→	True unless the first is true and second is false.	(P → Q): "If it rains, then it will flood."
IF AND ONLY IF	↔	True if both propositions are either true or false.	(P ↔ Q): "It rains if and only if it is cloudy."

3.3 AGENTS BASED ON PROFESSIONAL LOGIC

Propositional logic, also known as propositional calculus, is a branch of formal logic that deals with propositions and their relationships. A proposition is a declarative statement that can be evaluated as true or false but not both.

For example, "The sky is blue" is a proposition because it can be classified as either true or false depending on the context.

In propositional logic, each proposition is assigned a truth value: **true (T)** or **false (F)**. This binary nature allows for the construction of complex logical expressions using logical operators such as AND (∧∧), OR (∨∨), NOT (¬¬), IMPLIES ( ), and BICONDITIONAL ( ).

Syntax of Propositional Logic

- **Propositions**: Basic statements (e.g., P*P*: "It is raining").
- **Logical Connectives**: Operators that connect propositions to form compound statements.

Operator	Symbol	Description
AND	∧	True if both propositions are true
OR	∨	True if at least one proposition is true
NOT	¬	True if the proposition is false
IMPLIES	→	True unless a true proposition implies a false one
BICONDITIONAL	↔	True if both propositions are either true or false

Truth Tables

Truth tables are used to represent all possible truth values of propositions and their combinations. For

instance, consider two propositions PP and QQ:

This table shows how the truth values of PP and QQ affect the compound statements formed using logical operators.

P	Q	$P \wedge Q$	$P \vee Q$	$\neg P$
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

Applications

Knowledge Representation

One of the primary applications of propositional logic in AI is knowledge representation. Propositional logic provides a formal framework for representing facts and rules about a specific domain. This is particularly useful in expert systems, where domain-specific knowledge is encoded as a set of propositions. For example, consider a medical diagnosis system that uses propositional logic to represent rules such as:

- **P**: "The patient has a fever."
- **Q**: "The patient has a cough."
- **R**: "The patient might have the flu."

These can be expressed in propositional logic as:

(P∧Q)→R($P \wedge Q$)→R

This statement means that if both propositions P and Q are true, then R is also likely to be true. Such logical representations allow AI systems to infer new information from existing knowledge efficiently.

Reasoning and Decision-Making

Propositional logic is instrumental in enabling reasoning and decision-making processes within AI systems. By applying logical rules, AI agents can evaluate conditions and make decisions based on predefined criteria.

For instance, in a home automation system, an AI might use the following propositions:

- **P**: "It is hot outside."

- **Q**: “The windows are open.”
- **R**: “Turn on the air conditioner.”

The decision-making process can be represented as:

$(P \land \neg Q) \rightarrow R$ $(P \land \neg Q) \rightarrow R$

This translates to: “If it is hot outside AND the windows are not open, then turn on the air conditioner.” Such logical constructs enable AI systems to automate responses based on environmental conditions.

Natural Language Processing (NLP)

In the realm of Natural Language Processing, propositional logic plays a vital role in understanding and interpreting human language. It allows AI systems to convert natural language sentences into logical representations that can be processed and reasoned about.

For example, consider the sentence: “If it rains tomorrow, I will take an umbrella.” This can be represented in propositional logic as:

$P \rightarrow Q$ $P \rightarrow Q$

Where:

- **P**: “It rains tomorrow.”
- **Q**: “I will take an umbrella.”

By using propositional logic, chatbots and virtual assistants can better understand user queries and respond logically based on the information provided.

Game Playing

Propositional logic also applied in game-playing AI, where it helps agents make strategic decisions based on the current state of the game. For example, in chess, an AI might use logical expressions to evaluate potential moves and their outcomes.

Consider the proposition:

- **P**: “The opponent’s king is in check.”
- **Q**: “I can win if I move my queen.”

The AI can reason through these propositions to determine its best course of action:

$P \rightarrow Q$ $P \rightarrow Q$

This means that if the opponent’s king is in check, then moving the queen could lead to victory.

Problem Solving and Planning

Propositional logic is fundamental in problem-solving and planning within AI systems. It enables

planners to represent actions, states, and goals logically. For instance, the STRIPS (Stanford Research Institute Problem Solver) planning system employs propositional logic to represent preconditions and effects of actions.

In a scenario where an AI needs to plan a route for delivery, it might use propositions like:

- **P**: "The package is ready."
- **Q**: "The delivery vehicle is available."
- **R**: "Start the delivery."

The planning can be expressed as:

$(P \wedge Q) \rightarrow R$ $(P \wedge Q) \rightarrow R$
This indicates that if both P and Q are true, then the action R (starting the delivery) should executed.

Game Theory and Multi-Agent Systems

In multi-agent systems and game theory applications, propositional logic helps model the beliefs and actions of different agents. By using logical frameworks, AI can predict agent behaviour and design strategies for interaction.
For instance, consider two competing agents where each agent has propositions regarding their actions:

- Agent A believes: **P**: "If I attack first, I will win."
- Agent B believes: **Q**: "If I defend well, I will survive."

These beliefs can represented logically to help each agent strategise their moves based on expected outcomes.

3.4 FIRST ORDER LOGIC

First-Order Logic (FOL) is a powerful knowledge representation method used in Artificial Intelligence (AI) for reasoning and making inferences. Unlike propositional logic, which deals with true or false values, FOL extends logical capabilities by allowing the representation of objects, relationships, and quantifiers. This makes it more suitable for AI applications that require deeper insights into relationships between entities.

FOL is critical in intelligent systems as it provides the foundation for building knowledge-based AI systems. With the ability to express complex statements and conditions, FOL plays an essential role in areas such as natural language processing, automated reasoning, and knowledge representation.

The building blocks of First-Order Logic include:

- **Constants:** Specific objects in the domain (e.g., John, Earth).
- **Variables:** Placeholders for objects (e.g., x, y).
- **Functions:** Maps objects to other objects (e.g., father(John) = Mike).
- **Predicates:** Represent relationships between objects (e.g., isStudent(John)).

Quantifiers in First-Order Logic (FOL) allow the expression of statements involving multiple objects. They extend logic beyond individual propositions by enabling statements about entire groups or subsets within a domain.

1. Universal Quantifier (∀)

The universal quantifier (∀) denotes that a statement applies to all objects in a given domain. It is expressed as:

∀x P(x) meaning "For all x, P(x) holds."

Example:

∀x (isMammal(x) → hasHair(x))

This sentence states that every mammal has hair. Universal quantification is commonly used in AI to create general rules that apply to all entities, such as "all students must attend class."

By using the universal quantifier, AI systems can formulate rules that apply broadly across various scenarios, supporting tasks like inference generation.

2. Existential Quantifier (∃)

The existential quantifier (∃) is used when a statement applies to at least one object in the domain. It is expressed as:

∃x P(x) meaning "There exists an x such that P(x) holds."

Example:

∃x (teaches(John, x))

This sentence asserts that John teaches at least one person. Existential quantification is useful for expressing conditions where only a single instance or occurrence is required to satisfy the logic, such as "there exists a student who completed the project."

This quantifier plays a vital role in systems that search for specific solutions or instances within a problem space.

3. Properties of Quantifiers

Quantifiers in FOL possess important properties such as:

Commutativity: ∀x ∀y P(x, y) is logically equivalent to ∀y ∀x P(x, y).

Distributivity: Quantifiers can be distributed across logical operations, such as:

∀x (P(x) ∨ Q(x)) ≡ (∀x P(x)) ∨ (∀x Q(x)).

These properties ensure logical consistency when constructing complex rules. For instance, distributivity allows the creation of nested logical conditions, which are essential in AI for handling multiple scenarios simultaneously.

Free and Bound Variables

In First-Order Logic (FOL), variables are categorized as free or bound.

- **Free variables** are not associated with any quantifier and can take any value from the domain. For example, in the expression P(x), x is a free variable.
- **Bound variables** are associated with a quantifier (∀ or ∃) and are limited to the scope of that quantifier. In ∀x P(x), x is bound.

The distinction between free and bound variables is crucial because only statements with bound variables are valid logical expressions for reasoning. Free variables must be quantified for the statement to be meaningful in AI applications.

3.5 UNIFICATION

Unification in AI is the process of making two logical expressions identical by determining a suitable substitution of variables. It is a key operation in first-order logic and is widely used in automated reasoning, inference engines, and logic programming to resolve logical statements systematically.

Example:

Consider two logical expressions in predicate logic:

1. **Parent(X, Mary)**
2. **Parent(John, Mary)**

To unify these expressions, we find a substitution that makes them identical. Here, substituting X = John results in:

Parent(John, Mary) = Parent(John, Mary)

Since the expressions are now identical, unification is successful. This process allows AI systems to infer new knowledge and establish logical relationships, making it fundamental in knowledge-based reasoning and automated decision-making.

Importance of Unification in AI

Unification is essential for simplifying logical reasoning and theorem proving by matching logical expressions and ensuring consistency in automated deduction. It helps AI systems derive conclusions by resolving symbolic relationships efficiently.

In rule-based AI systems like Prolog and expert systems, unification enables pattern matching, allowing the system to apply rules and infer new facts dynamically. This makes it crucial for logic programming, automated reasoning, and knowledge representation.

Conditions for Unification

For unification to be successful, certain conditions must be met. These conditions ensure that logical expressions can be made identical without contradictions or infinite loops.

1. Identical Constants Must Match

Two constants can only be unified if they are exactly the same. If the constants are different, unification fails.

Example:

"apple" = "apple" (valid unification)

"apple" ≠ "banana" (invalid unification)

2. Variables Can Be Replaced with Constants or Other Variables

A variable can take the value of a constant or another variable to achieve unification.

Example:

If X = "red", then color(X) becomes color("red"), making the expressions identical.

3. Function Terms Must Match

When unifying functions, the function names and the number of arguments must be the same. Variables within the function can be substituted to complete the unification.

Example:

f(a, X) = f(a, b) is unified with X = b, making both expressions identical.

4. Cannot Create Cycles in Substitutions

Unification should not lead to infinite recursion by assigning a variable to an expression that contains itself.

Example:

X = f(X) creates an infinite cycle and is not allowed in unification.

By following these conditions, unification ensures logical consistency, enabling AI systems to perform reasoning, inference, and symbolic computation effectively.

Step-by-Step Breakdown of the Unification Algorithm

Step 1: Compare Terms

- Identify whether the terms are constants, variables, or functions.
- If both terms are constants, they must be identical for unification to proceed.
- If both terms are functions, they must have the same name and number of arguments.

Step 2: Apply Substitution

- If a variable is found, it can be replaced with a matching constant or another variable.
- Example: Unifying X = Alice in loves(X, Y) = loves(Alice, Bob) results in X → Alice.

Step 3: Recursively Check Sub-terms

- If the terms involve functions, recursively apply the unification process to each argument.

- Example: Unifying f(a, X) = f(a, b) requires checking both a = a (valid) and X = b.

Step 4: Resolve Conflicts

- If contradictions arise, such as mismatched constants or cyclic substitutions, unification fails.
- Example: Trying to unify X = f(X) creates an infinite loop, causing failure.

Python Implementation of Unification Algorithm

```
def unify(term1, term2, substitutions={}):
    if term1 == term2:
        return substitutions  # Terms are already identical
    if isinstance(term1, str) and term1.islower():  # If term1 is a variable
        return unify_var(term1, term2, substitutions)
    if isinstance(term2, str) and term2.islower():  # If term2 is a variable
        return unify_var(term2, term1, substitutions)
    if isinstance(term1, tuple) and isinstance(term2, tuple) and len(term1) == len(term2):
        for t1, t2 in zip(term1, term2):
            substitutions = unify(t1, t2, substitutions)
            if substitutions is None:
                return None
        return substitutions
    return None  # Unification fails


def unify_var(var, value, substitutions):
    if var in substitutions:
        return unify(substitutions[var], value, substitutions)
    elif value in substitutions:
        return unify(var, substitutions[value], substitutions)
    else:
```

```
    substitutions[var] = value
    return substitutions
# Example Usage:
expr1 = ("loves", "X", "Y")
expr2 = ("loves", "Alice", "Bob")
result = unify(expr1, expr2)
print("Unification Result:", result)
```

Applications

Automated Theorem Proving

Unification is essential in logic solvers and automated theorem proving. It is used in Resolution Theorem Proving, where logical statements are unified to derive conclusions in formal logic systems. This helps in AI-driven problem-solving, such as proving mathematical theorems and verifying logical consistencies.

Natural Language Processing (NLP)

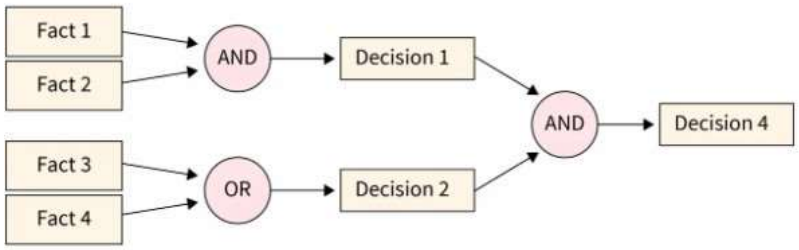
Unification is applied in feature unification for grammar parsing, ensuring consistency in syntactic structures. It is used in natural language understanding, speech recognition, and machine translation, where linguistic rules must be unified to generate meaningful sentence structures.

Knowledge Representation and Reasoning

In first-order logic (FOL) and rule-based AI systems, unification helps match logical facts with inference rules. Expert systems and AI-driven reasoning engines use unification to deduce new facts by applying predefined logical rules, making it a key component of AI knowledge representation.

3.5 FORWARD CHAINING

Forward chaining is a data-driven reasoning strategy in AI. It starts with known facts and applies rules to generate new facts or reach a conclusion. The process continues until no more new facts can be inferred or a goal is achieved. This approach is often used in expert systems for tasks such as troubleshooting and diagnostics.



Properties:

Data-Driven: The reasoning starts from available data (facts) and works toward a goal.

Bottom-Up Approach: It builds knowledge from facts, gradually moving towards conclusions.

Breadth-First Search Strategy: The inference engine explores multiple rules simultaneously, applying them step by step.

Possibility of Irrelevant Rules: Forward chaining may explore rules that do not contribute to the final solution, making it less efficient in some cases.

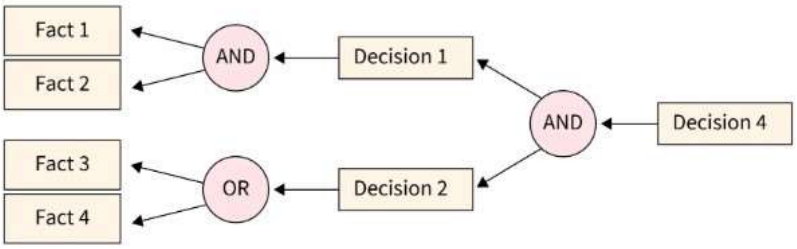
Example:

In forward chaining, facts are often represented in First-Order Logic (FOL) to apply rules systematically.

1. **Fact 1:** The patient has a fever.
2. **Fact 2:** The patient has a sore throat.
3. **Rule:** If the patient has a fever and sore throat, they might have the flu.

3.6 BACKWARD CHAINING

Backward chaining is a goal-driven reasoning strategy used in AI. It starts with a goal or hypothesis and works backward to determine if the available facts support the goal. The process continues by recursively breaking down the goal into smaller sub-goals until either all facts are verified or no more supporting data is found.



Properties:

- **Goal-Driven:** Reasoning begins with a desired goal and searches for evidence to support it.
- **Top-Down Approach:** The system starts from the goal and works back to find relevant facts.
- **Depth-First Search Strategy:** The inference engine follows a path deeply before exploring other possibilities, prioritizing each goal or sub-goal in sequence.
- **Possibility of Infinite Loops:** If not handled properly, backward chaining may get stuck in

loops while looking for evidence to support the goal.

Example

Consider the same medical diagnosis system with the goal of determining if a patient has the flu.

1. **Goal:** Does the patient have the flu?
2. **Rule:** If the patient has a fever and sore throat, they might have the flu.
3. **Sub-goals:**
   - Verify if the patient has a fever.
   - Verify if the patient has a sore throat.

The system works backward from the goal (flu) and checks if the patient's symptoms (fever and sore throat) match. If all sub-goals are verified, the system concludes that the patient likely has the flu.

3.7 RESOLUTION

Resolution is a fundamental inference rule and a key technique used in automated theorem proving and logic programming within the field of artificial intelligence. It is particularly associated with the resolution refutation method, which aims to prove the validity or satisfiability of logical formulas through a process of negation and resolution. The resolution rule allows us to derive new clauses from existing ones by eliminating complementary literals.

Example 1:

Let's consider a simple example with propositional logic. Suppose we want to prove that the following statements are contradictory:

P OR Q

NOT P OR R

NOT Q OR R

To prove contradiction, we can use the resolution rule:

Resolve clauses 1 and 2: (P OR Q) AND (NOT P OR R) => Q OR R

Resolve the result with clause 3: (Q OR R) AND (NOT Q OR R) => R

Since we have derived a clause that contains both R and NOT R (a contradiction), we conclude that the original statements are contradictory.

Example 2:

Let's consider a more complex example using first-order logic. Suppose we want to prove the statement: "All humans are mortal."

For all x, Human(x) => Mortal(x)

We'll assume that our knowledge base includes the following premises:

A. Human(Socrates)

B. NOT Mortal(Socrates)

To prove the statement using resolution:

Convert premise 1 to its negation: Human(S) AND NOT Mortal(S)

Resolve with premise A: (Human(S) AND NOT Mortal(S)) AND Human(S) => NOT Mortal(S)

Resolve the result with premise B: NOT Mortal(S) AND NOT Mortal(S) => Contradiction

Since we've derived a contradiction, we can conclude that our original statement "All humans are mortal" is valid.

3.8 KNOWLEDGE REPRESENTATION

Knowledge representation in AI refers to the methods used to encode information about the world into a format that an AI system can understand and use to make decisions. It is a crucial component of AI that bridges the gap between raw data and meaningful reasoning. By representing knowledge in a structured way, AI systems can interpret data, draw inferences, and apply reasoning techniques to solve problems.

Different Types of Knowledge

1. Declarative Knowledge

What it is: This type of knowledge refers to facts or statements that describe the world, often in the form of “knowing what.” It is static and doesn’t involve actions or procedures.

Example: “Paris is the capital of France” is a piece of declarative knowledge.

2. Procedural Knowledge

What it is: Procedural knowledge describes “how” to perform tasks or solve problems. It is dynamic and action-oriented, focusing on processes and methods.

3. Structural Knowledge

What it is: This type of knowledge deals with the relationships between entities or concepts, organizing knowledge into structures like hierarchies or networks.

4. Meta Knowledge

What it is: Meta knowledge is knowledge about knowledge. It refers to understanding which knowledge to use in a particular situation or context.

5. Heuristic Knowledge

What it is: Heuristic knowledge consists of experience-based rules of thumb or best practices that help in making decisions quickly without complete information.

Cycle of Knowledge Representation

The cycle of knowledge representation in AI refers to the iterative process through which AI systems perceive, learn, represent, and apply knowledge to make informed decisions. This cycle is essential for building intelligent systems capable of reasoning and interacting with their environment.

1. Perception

The process begins with AI systems perceiving data from their environment. This data could come from sensors, cameras, user inputs, or databases.

2. Learning

After perceiving data, the system learns by identifying patterns, relationships, and trends from the data. This can involve supervised learning, unsupervised learning, or reinforcement learning techniques.

3. Knowledge Representation & Reasoning

In this stage, the system organizes the learned data into structured knowledge, allowing it to reason and draw conclusions. This is where the AI uses knowledge representation techniques to encode the information it has learned.

4. Planning

The system uses the represented knowledge to plan actions based on goals and constraints. Planning involves selecting the best course of action based on the available knowledge.

5. Execution

Finally, the system executes the planned actions, completing the cycle. After execution, the AI system can receive feedback and adjust its knowledge and actions accordingly.

3.9 ONTOLOGICAL ENGINEERING

Ontological engineering is a crucial part of Artificial Intelligence (AI) that focuses on creating structured frameworks to represent knowledge. It helps AI systems understand, manage, and connect complex information efficiently.

Importance of Ontologies:

- **Knowledge Representation:** Ontologies provide a standardized way to represent knowledge.
- **Semantic Web:** Helps machines understand and connect web data.
- **Natural Language Processing (NLP):** Enhances language understanding by structuring relationships.
- **Decision-Making:** Supports AI systems in reasoning and making informed decisions.

Types of Ontology

Ontologies can be categorized based on the type of knowledge they represent and the purpose they serve in a given domain. These models help structure and define relationships between concepts, supporting various AI applications.

Domain Ontology

A domain ontology describes concepts and relationships specific to a particular field or domain. It represents the knowledge and terminology used within that area, helping AI systems understand domain-specific data.

Example:

Healthcare Ontology: Defines medical concepts such as diseases, symptoms, treatments, and their relationships.

E-commerce Ontology: Represents concepts like products, categories, prices, and customer preferences.

Task Ontology

A task ontology focuses on the processes and actions required to achieve specific tasks within a domain. It is designed to guide AI systems in understanding and performing tasks.

Application Ontology

Application ontologies are tailored for specific applications and focus on solving problems in particular industries, such as healthcare, finance, or education. They define concepts that are directly related to the application's needs.

Upper Ontology

An upper ontology provides a high-level framework for knowledge representation across various domains. It defines general concepts like "object," "event," and "process," which are then further detailed in domain-specific ontologies.

Components of Ontology

An ontology is made up of several core components that help structure knowledge and define relationships between entities. These components allow AI systems to organize, interpret, and reason with data effectively.

Individuals

Individuals represent the most basic and specific objects or entities in an ontology. These are the instances of concepts defined within the ontology.

Classes

Classes are groups or sets of individuals that share common characteristics. They represent concepts or categories in an ontology and help organize individuals meaningfully.

Attributes

Attributes describe the properties or characteristics of individuals. They provide additional information about the entities within the ontology.

Relations

Relations define the connections between individuals, classes, and attributes in an ontology. They establish meaningful relationships within the knowledge base.

Different Ontology Languages

Ontology languages are formal tools used to represent, share, and manage ontologies in Artificial Intelligence. These languages allow AI systems to understand and process knowledge systematically,

enabling clear communication between humans and machines.

CycL

CycL is a specialized ontology language used in the Cyc Project, a large-scale AI initiative aimed at creating a common-sense knowledge base.

Rule Interchange Format (RIF)

The Rule Interchange Format (RIF) is a W3C standard designed to enable the exchange of rules between different systems.

Open Biomedical Ontologies (OBO)

OBO is a collection of shared ontologies specifically designed for the biomedical and healthcare domains.

Web Ontology Language (OWL)

The Web Ontology Language (OWL) is one of the most widely used languages for creating, sharing, and managing ontologies. It is a W3C standard, designed for the semantic web.

3.10 CATEGORIES AND OBJECTS

Categories:

Categories are grouping of objects that share common attributes, properties and characteristics. They serve as conceptual frameworks for organizing and understanding the world around us.

Categories can range from simple concepts like “animals” or “colors” to more complex ones like “types of transportation” or “genres of music”.

Objects:

Objects are individual entities or instances that belong to one or more categories.

They represent perceptible or abstract things in the world that have distinct identities, properties and relationships.

Objects can be anything from physical objects like “cars” or “books” to abstract concept like “happiness” or “justice”.

Example

Category: Animals Objects: Dog, Cat, Elephant and Lion

Category: Colors Objects: Red, Blue, Green and Yellow

AI representing categories and objects allow systems to organize, store and manipulate knowledge in a structured and meaningful way, enabling tasks such as reasoning, decision-making, and problem-solving.

3.11 EVENTS

Situations denote the states resulting from executing actions. This approach is called situation calculus. Situations are logical terms consisting of the initial situation and all situations that are generated by

applying an action to a situation.

Fluents are functions and predicates that vary from one situation to the next, such as the location the agent.

Discrete events have a definite structure. For example, Shankar's trip has a beginning, middle and end.

Categories of events with continuously happening property are called process categories or liquid event categories. Any process that happens over an interval also happens over any subinterval: (e € Processes) Happens (e,(t1, t4)) (t1<t2<t3<t4) => Happens(e,(t2, t3))

For example, the predicate T, represents time. Here it is some subinterval of time lunch hour.

T (Working (Start), TodayLunchHour)

This liquid event is called temporal substance, whereas substance like butter is special substance.

3.12 MENTAL EVENTS AND MENTAL OBJECTS

For a single- agent domains, knowledge about one's own knowledge and reasoning processes is useful for controlling inference. In a multiagent domains, it becomes important for an agent to reason about the mental states of the other agents.

Example

Bob and Alice scenario

It requires a model of the mental objects in someone's head(knowledge base) and the processes that manipulate these mental objects. Relationships between agents and mental objects: believes, knows, wants, intends ... are called propositional attitudes.

Lois knows that Superman can fly:

– Knows (Lois , CanFly (Superman))

If Superman is Clark, then we must conclude that Lois knows that Clark can fly:

– (Superman = Clark) ! Knows (Lois , CanFly (Superman)) |= Knows(Lois, CanFly(Clark))

"Can Clark fly?" – No. Need descriptions

If an agent knows that 2 + 2 = 4 and 4 < 5, then we want an agent to know that 2 + 2 < 5. This property is called referential transparency.

For propositional attitudes like believes and knows, we would like to have referential opacity Modal logic address this problem. Modal logic includes special modal operators that take sentences (rather than terms) as arguments.

Example: *"A knows P"* is represented with the notation $K_A P$,

- where K is the modal operator for knowledge,
- agent A (written as the subscript) and a sentence.

Possible Worlds and Accessibility Relations

In modal logic, consider both the possibility that Superman's secret identity is Clark and that it isn't.

Clark=Superman and Clark! =Superman

Build a model, that consists of a collection of possible worlds rather than just one true world and the worlds are connected in a graph by accessibility relations

"Bond knows that someone is a spy" is ambiguous

$\exists K_{Bond}Spy(x))$ which in modal logic means that there is an x that, in all accessible worlds, Bond knows to be a spy. Bond just knows that there is at least one spy: $K_{Bond}\ \exists XSpy(x)$. The modal logic interpretation is that in each accessible world there is an x that is a spy, but it need not be the same x in each world.

Example:

Eating food necessary for living, so we eat.

Gambling to gain money.

Probability. Believe we will win. Knows (a, p) – agent a knows that proposition p is true. Lois knows whether Clark can fly if she either knows that Clark can fly or knows that he cannot – Knowswhether(a, p) $\Leftrightarrow$knows(a, p) v knows(a, ¬ p)

Belief is also something which can change over time.
For example:

Lois believes Superman flies today

T(Believes(Lois, flies(Superman)),Today)

The same will not be true in 100 years when he died of old age.

3.13 REASONING SYSTEMS FOR CATEGORIES

Reasoning is the process by which machines simulate human-like decision-making, problem-solving and predictions. Just as humans use logic to navigate the world, AI systems rely on different types of reasoning to draw conclusions, interpret data and adapt to new situations. These reasoning techniques ranges from simple rule-based systems to more advanced, learning-driven models which helps AI handle dynamic and unpredictable environments.

Deductive Reasoning

Deductive reasoning starts with general principles and applies them to specific cases to arrive at certain conclusions. If the premises are true, the conclusion must also be true.

Example: If all humans are mortal and Socrates is a human, then Socrates is mortal.

Application: It is used in expert systems. These systems apply predefined rules such as "if-then" statements to specific problems to derive solutions. For example, in a medical diagnosis system, if the presence of certain symptoms matches a known pattern a diagnosis is made.

Inductive Reasoning

Inductive reasoning works by drawing general conclusions from specific observations. The conclusions reached are not certain but are based on probability and patterns observed in the data.

Example: If we observe that the sun rises in the east every day, we might infer that it will rise in the east tomorrow.

Application: It is fundamental to machine learning. AI systems, in supervised learning, identify patterns in data and use them to make predictions about new, unseen data. For example, Netflix's recommendation engine uses inductive reasoning to suggest movies based on past viewing habits.

Abductive Reasoning

Abductive reasoning starts with incomplete observations and seeks the most likely explanation. It's about making educated guesses based on available data, even if not all facts are known.

Example: If a patient has a fever and cough, a doctor might hypothesize that they have the flu even though other illnesses could cause similar symptoms.

Application: It is used in diagnostic AI systems like those in healthcare or fault detection systems. For example, an AI tool for diagnosing diseases can suggest the most likely diagnosis based on a set of symptoms even if it doesn't have all the information.

Analogical Reasoning

It involves comparing two situations that are similar and using knowledge from one to solve problems in another. It helps AI systems solve problems in new domains by applying solutions from related areas.

Example: If flying a helicopter is similar to flying a drone, knowledge from piloting a helicopter can be transferred to flying a drone.

Applications: This type of reasoning is used in robotics and cognitive systems where AI can transfer knowledge from one task like navigating a robot to a similar task of piloting a drone.

Common Sense Reasoning

It allows AI to handle situations based on everyday knowledge something that humans use naturally. It involves making judgments about the world that are obvious to humans but difficult for machines to

understand such as predicting outcomes in familiar situations.

Example: If it rains, we can expect the ground to get wet even without explicitly stating it.

Application: It is important in conversational AI such as Siri, Alexa which allows AI to respond to user queries in a logical, intuitive manner. It's also used in autonomous vehicles where AI must anticipate and react to everyday scenarios like pedestrians crossing the road.

Monotonic Reasoning

It refers to a form of reasoning where conclusions once drawn, cannot be reversed even if new information becomes available. This ensures that conclusions remain consistent regardless of updates to the knowledge base.

Example: The statement "The Sahara is a desert" remains true even if more information about the world's deserts is introduced.

Applications: It is used in systems requiring consistency such as formal verification tools in AI. These tools ensure that AI systems behave as expected and that conclusions do not change unless deliberately altered.

Nonmonotonic Reasoning

In contrast to monotonic reasoning, nonmonotonic reasoning allows AI systems to revise their conclusions based on new information. It’s important for decision-making in dynamic and unpredictable environments.

Example: Initially, we might think that all birds can fly, but we revise this conclusion after learning about penguins, which cannot fly.

Application: It is important for adaptive AI systems that need to change their decisions based on new data such as in real-time traffic management or autonomous vehicles that adjust their routes depending on traffic conditions.

Fuzzy Reasoning

Fuzzy reasoning deals with uncertainty by allowing for degrees of truth rather than binary true/false values. This is useful when data is vague or incomplete.

Example: The statement "It’s warm outside" is vague. Fuzzy reasoning might assign a value like 0.7, to represent how warm it is.

Application in AI: It is used in control systems such as those found in smart appliances like air conditioners or washing machines where precise values may not always be available. It’s also used in autonomous vehicles to interpret sensor data when conditions are uncertain (e.g fog or poor visibility).

3.14 REASONING WITH DEFAULT INFORMATION

Default reasoning in AI is like making educated guesses when you don't have all the information you need. Default reasoning in artificial intelligence (AI) allows systems to make plausible assumptions and conclusions in situations with incomplete or uncertain information.

By relying on typical or "default" knowledge, such as assuming birds can fly unless specified otherwise, AI systems can function effectively in the real world where perfect information is rare. This type of reasoning is non-monotonic, meaning new information can change previously drawn conclusions, and it employs methods like circumscription and default logic to manage assumptions.

Default reasoning is crucial for handling uncertainty, enhancing flexibility, and improving efficiency in AI, though it faces challenges like conflict resolution and integrating with other reasoning methods.

Key aspects of Default Reasoning are:

Default Rules: These are common-sense assumptions you usually make. For example, "Most birds can fly."

Exceptions: Sometimes, there are special cases that don't follow the usual rules. For example, penguins are birds, but they can't fly.

Changing Your Mind: If you learn something new that contradicts your assumption, you change your guess. For instance, if you first assume a bird can fly but then find out it's a penguin, you update your guess to say it can't fly.

Handling Missing Information: Even if you don't know everything, you can still make a good guess based on what you typically know. So, if you see a bird and don't know what kind it is, you might guess it can fly because most birds do.

UNIT – IV PLANNING AND REASONING

4.1 CLASSICAL PLANNING

Classical Planning is a fundamental concept in Artificial Intelligence (AI) that involves forming a sequence of actions to achieve a specific goal. It plays an important role in domains such as robotics, automated problem-solving and game AI where actions must be taken to transition from an initial state to a goal state while sticking to certain constraints.

It differs from other problem-solving techniques because it focuses on predetermined actions with a clear understanding of the current environment and the effects of actions on that environment.

Key Concepts in Classical Planning

- State: A state is a configuration of the world at a given point in time. It represents all the conditions that are true at that time.
- Action: An action is a transition from one state to another. It consists of preconditions that must be satisfied for the action to be applicable and effects that describe how the action changes the state.
- Goal: It is the desired end state that we aim to reach by performing a sequence of actions.
- Plan: A plan is a sequence of actions that will take us from the starting state to the goal state, making sure to satisfy all the required conditions along the way.

Domain-Independent Planning

One of the strengths of Classical Planning is its **domain independence**. This means that the same planning techniques can be used in many different fields without needing to adjust the core methods. This makes classical planning highly flexible and widely applicable.

Domain-independent planning focuses on the key aspects of any planning problem which are:

- **State representations:** How to describe the current situation.
- **Action models:** How actions can change the state.
- **Search strategy:** How to systematically find the sequence of actions that lead to the goal.

By using general principles, rather than relying on detailed knowledge of a specific domain, these methods can be applied across many different fields including robotics, logistics, manufacturing and more.

Planning Domain Modelling Languages

The domain independence in classical planning can be further understood through the use of domain-independent modeling languages such as **PDDL** (Planning Domain Definition Language) and **STRIPS** (Stanford Research Institute Problem Solver). These languages allow planners to represent problems in a way that is not tied to any specific domain making them applicable to a variety of planning tasks.

1. **PDDL (Planning Domain Definition Language):** It is used for describing the objects, actions and goals in a planning problem. It provides a flexible framework to define both simple and complex planning scenarios.
1. **STRIPS (Stanford Research Institute Problem Solver):** It is one of the earliest planning languages that introduced a way to represent the state of the world, actions and the effects of those actions. It laid the foundation for many modern planning systems.

Classical Planning Techniques

Classical planning assumes a static and fully observable environment where the goal is to find a sequence of actions to move from the current state to the goal state while satisfying any constraints. There are two main categories of planning techniques:

1. State-Space Search

State-space search explores the space of possible states by applying actions to move from one state to another. It continues until the goal state is reached.

Key algorithms include:

- **Breadth-First Search:** Explores all states at one level before moving to the next. It guarantees finding the shortest solution but can be memory-heavy.
- **Depth-First Search:** It explores one branch deeply before backtracking. It uses less memory but doesn't guarantee the shortest path.
- **A* and Greedy Search:** It use heuristics to prioritize states, speeding up the search process.

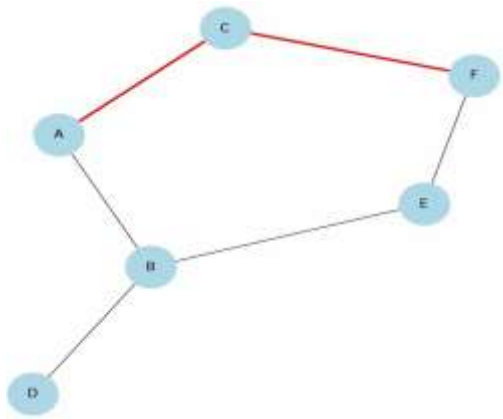
Example

```
import matplotlib.pyplot as plt
import networkx as nx
from collections import deque

def breadth_first_search(start, goal, graph):
    queue = deque([[start]])
    visited = set()
    while queue:
        path = queue.popleft()
        node = path[-1]
```

```
        if node == goal:
            return path
        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                new_path = list(path)
                new_path.append(neighbor)
                queue.append(new_path)
    return None
def visualize_bfs(graph, path):
    G = nx.Graph(graph)
    pos = nx.spring_layout(G)
    plt.figure(figsize=(8, 8))
    nx.draw(G, pos, with_labels=True, node_size=3000, node_color="lightblue", font_size=15)
    path_edges = [(path[i], path[i + 1]) for i in range(len(path) - 1)]
    nx.draw_networkx_edges(G, pos, edgelist=path_edges, edge_color="red", width=3)
    plt.title(f"BFS Path from {path[0]} to {path[-1]}")
    plt.show()
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}
path = breadth_first_search('A', 'F', graph)
if path:
    visualize_bfs(graph,  path)
```

OUTPUT



Plan-Space Search

Plan-space search starts with a partial plan and incrementally refines it until it's complete and consistent. This approach is useful for problems with complex dependencies between actions.

Techniques in this category include:

- **Partial-Order Planning:** It allows flexibility in the order of actions only specifying necessary dependencies.
- **Hierarchical Task Network (HTN) Planning:** It breaks tasks into smaller sub-tasks in a hierarchical structure.
- **UCPOP and VHPOP:** Algorithms that refine partial plans by resolving inconsistencies.

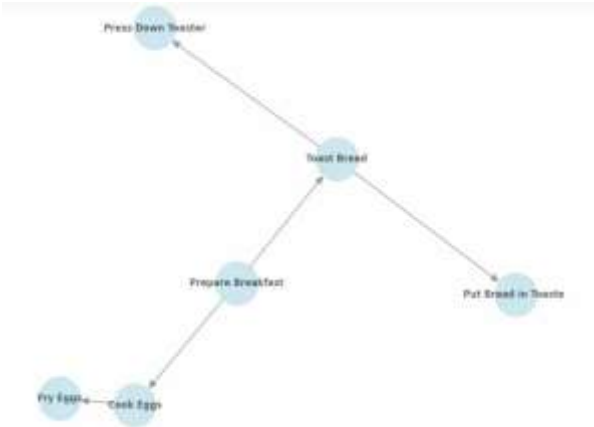
Example of Hierarchical Task Network

```
import matplotlib.pyplot as plt
import networkx as nx

htn_tasks = {
    "Prepare Breakfast": ["Cook Eggs", "Toast Bread"],
    "Cook Eggs": ["Fry Eggs"],
    "Toast Bread": ["Put Bread in Toaster", "Press Down Toaster"]
}
def htn_planning(high_level_task):
    plan = []
    tasks_to_process = [high_level_task]
    while tasks_to_process:
        current_task = tasks_to_process.pop()
        if current_task in htn_tasks:
            plan.append(current_task)
            tasks_to_process.extend(htn_tasks[current_task])
        else:
            plan.append(current_task)
    return plan
```

```
def visualize_htn(htn_tasks):
    G = nx.DiGraph()
    for task, sub_tasks in htn_tasks.items():
        for sub_task in sub_tasks:
            G.add_edge(task, sub_task)
    plt.figure(figsize=(10, 8))
    pos = nx.spring_layout(G, seed=42)
        nx.draw(G, pos, with_labels=True, node_size=3000, node_color="lightblue", font_size=12,
font_weight="bold", edge_color="gray", arrowsize=20, width=2, alpha=0.6)
    plt.title("Hierarchical Task Network (HTN) Planning")
    plt.show()
plan = htn_planning("Prepare Breakfast")
print(f"Generated Plan: {plan}")
visualize_htn(htn_tasks)
```

OUTPUT



Applications

1. **Robotics:** In robotics, it helps robots to plan a sequence of actions to achieve a specific task like picking up an object or navigating a room.
1. **Automated Scheduling:** It is used in industries such as manufacturing and logistics for scheduling tasks and optimizing resource allocation.
1. **Game AI:** These algorithms are used in game development for NPC (Non-Player Character) behavior, helping them plan a series of actions to complete specific missions.
1. **Space Missions:** In space exploration, it helps in planning the sequence of actions for a spacecraft to complete its mission objectives.

Advantages

1. **Optimality:** Classical planning algorithms are capable of finding the most efficient or optimal sequence of actions to achieve the desired goal.

2. **Simplicity:** The concepts behind classical planning are relatively easy to understand and implement making it accessible for a wide range of applications.
3. **Well-Defined Structure:** The problems in classical planning are clearly defined with specific initial and goal states. This structured approach simplifies the planning process.
4. **Domain Independence:** Classical planning techniques can be applied across various domains without needing significant adjustments to the core methods making it versatile and adaptable.

4.2 HEURISTIC PLANNING

Heuristic search is a strategy used in AI to optimize the search process by using a heuristic function to estimate the cost of reaching a goal. Instead of exhaustively exploring all possible paths, a heuristic search uses this estimate to prioritize the most promising paths, reducing computational complexity and speeding up decision-making.

The heuristic function in AI acts as a guide, providing approximate solutions when exact answers are computationally expensive or unnecessary. By leveraging domain-specific knowledge, heuristic searches can solve real-world problems more efficiently than traditional methods.

Heuristics are crucial for solving complex problems that involve large search spaces, such as pathfinding, decision-making, and optimization. In these cases, heuristics help narrow down the search space, reducing the time and computational power needed to arrive at a solution. The technique is widely applied in systems that require real-time decision-making, such as autonomous vehicles and robotics.

In AI, heuristic search techniques balance efficiency and accuracy, providing optimal or near-optimal solutions without the need for exhaustive computation. This makes them highly valuable for a wide range of applications in modern AI systems.

Significance of Heuristic Search in AI

Heuristic search techniques are integral to many AI systems due to their ability to enhance problem-solving efficiency. By using heuristic functions, AI systems can quickly determine which paths or actions are most likely to lead to a solution, significantly reducing the time required for decision-making.

1. **Speed and Efficiency**: Heuristic searches prioritize the most promising paths, making them faster than traditional search algorithms.
2. **Real-World Applications**: In tasks like search engines, autonomous systems, and route planning, heuristic search techniques improve performance by narrowing down search options.
3. **Scalability**: Heuristic searches are particularly valuable when dealing with large and complex datasets, where traditional search methods would be too computationally expensive.

Components

1. Heuristic Function

The heuristic function estimates the cost of reaching the goal from a given state. It guides the search by providing an informed guess about which path is most promising. A well-designed heuristic function can drastically reduce the time required to find a solution.

2. Search Algorithm

The search algorithm navigates through the possible solutions, using the heuristic function to prioritize paths. The algorithm's goal is to explore the most promising paths first, minimizing the total number of explored options.

3. Evaluation Function

The evaluation function combines the cost of the path taken so far with the estimated cost to reach the goal (provided by the heuristic function). This combined value helps the algorithm choose the most efficient path to the solution.

Types of Heuristic Search Techniques

1. A* Search Algorithm

The A* search algorithm is one of the most popular heuristic search techniques due to its ability to find optimal solutions efficiently. A* combines both the actual path cost and the heuristic estimate of the remaining cost to reach the goal.

- **Path Cost (g):** The actual cost from the start node to the current node.
- **Heuristic Estimate (h):** The estimated cost from the current node to the goal.

A* uses the evaluation function $f(n) = g(n) + h(n)$ to determine the best path. It's widely used in applications like game AI and route optimization.

2. Greedy Best-First Search

Greedy Best-First Search selects the path that appears to be the best at each step by using only the heuristic value (h). It prioritizes nodes with the lowest heuristic cost, focusing on reaching the goal as quickly as possible.

However, Greedy Best-First Search can be shortsighted, sometimes leading to suboptimal solutions because it doesn't consider the actual path cost. It's ideal for situations where a quick, approximate solution is needed, such as in real-time applications like robot navigation.

3. Hill Climbing

Hill Climbing is a local search technique that focuses on continuously moving towards a better solution based on the evaluation function. It evaluates neighboring solutions and moves towards the one with the highest score (closest to the goal).

The main drawback of Hill Climbing is that it can get stuck in local maxima, where no neighboring solution offers a better result, even though the global maximum is still far away. Variants like Stochastic Hill Climbing address this issue by occasionally allowing moves to worse solutions.

4. Simulated Annealing

Simulated Annealing is a heuristic search technique that explores the search space by occasionally accepting worse solutions to avoid getting stuck in local maxima. Inspired by the annealing process in metallurgy, this algorithm gradually reduces the probability of accepting worse solutions as the search progresses.

By allowing exploration of suboptimal solutions, simulated annealing can escape local maxima and find a better overall solution. It is commonly used in optimization problems where the search space is large and complex.

5. Beam Search

Beam Search is a variation of Best-First Search, where only a fixed number of the best paths are kept at each level of the search tree. This technique limits memory usage and computation by maintaining only a predetermined number of paths, or "beams."

Beam Search is commonly used in natural language processing (NLP) tasks and speech recognition, where the search space is vast, and computational efficiency is critical.

Applications

1. Autonomous Vehicles

In autonomous driving, heuristic search techniques are used for route planning and decision-making in real time. These systems evaluate various possible paths and choose the one that optimally balances factors like traffic, safety, and fuel efficiency.

2. Robotics

Heuristic search techniques are essential in robotics for tasks like pathfinding and obstacle navigation. Robots in dynamic environments use these techniques to find the most efficient routes while avoiding obstacles and ensuring safe operation.

3. Natural Language Processing (NLP)

In NLP, heuristic search techniques are used to optimize text processing tasks like sentence parsing and machine translation. By narrowing down the search space, these techniques allow NLP systems to provide faster and more accurate results.

4. Game Playing

Heuristic search plays a significant role in game AI, helping systems make decisions in games like chess, Go, and other strategy-based games. These algorithms evaluate possible moves and select the most promising ones, enabling AI to play at an advanced level.

4.3 HIERARCHICAL PLANNING

It is a problem-solving method where big tasks are broken down into smaller, manageable steps. It's like creating a to-do list for AI systems.

Imagine you're planning a birthday party. You'd start with big ideas like "get food" and "invite friends." Then, you'd break those down into smaller tasks like "buy cake" or "send invitations." That's

what hierarchical planning does for AI.

It helps AI systems tackle tricky problems by organizing them into levels, from big-picture goals to tiny, doable actions. This makes it easier for AI to solve problems and make smart choices.

Key Components of Hierarchical Planning

High-level goals

These are the big things the AI wants to achieve. Think of them as the main mission or the end result we're aiming for.

Sub-goals

These are smaller steps that help reach the high-level goals. They break down the big task into more manageable chunks.

Actions

These are the specific things the AI does to complete each sub-goal. They're like the tiny steps in a recipe.

Let's use a simple example: cleaning a room. The high-level goal is to "clean the room." Sub-goals might include "pick up toys," "vacuum floor," and "dust furniture." Actions for "pick up toys" could be "grab a toy," "walk to the toy box," and "put a toy in the box."

By organizing tasks this way, AI can tackle complex problems more easily. It starts with the big picture and works its way down to the little details, just like we do when we plan something big.

How Hierarchical Planning in Artificial Intelligence Works

Set the main goal

- The AI starts by identifying the big task it needs to complete.
- This is like deciding whether you want to "make a sandwich" or "clean the house.

Break it down

- The AI divides the main goal into smaller, more manageable tasks.
- For “make a sandwich,” this might include “get bread,” “add fillings,” and “put it together.

Create a detailed plan

- For each small task, the AI figures out the exact steps needed.
- Under “add fillings,” it might list “spread butter,” “add cheese,” and “add lettuce.

Put it all in order

- The AI arranges all these steps in the best order to reach the main goal.
- It’s like writing out a recipe, making sure each step leads to the next.

Stay flexible

- As the AI works through its plan, it can make changes if needed.
- If it runs out of cheese, it might switch to a different filling.

Benefits of Hierarchical Planning

Hierarchical planning offers several advantages for AI systems:

Easier problem-solving

- Big, complicated tasks become more manageable when broken into smaller parts.
- It’s like eating an elephant one bite at a time – suddenly, it doesn’t seem so impossible!

Faster work

- The AI can focus on one small task at a time, rather than getting stuck on the big picture.
- This helps it make progress more quickly and efficiently.

Handling complex tasks

- AI can take on more complicated jobs that have many steps or parts.
- It’s like using a map to navigate a big city – you can find your way even if you’ve never been there before.

Reusable plans

- Once the AI figures out how to do something, it can use that knowledge for similar tasks in the future.
- This saves time and helps the AI become smarter over time.

Better understanding

- Humans can more easily see how the AI is making decisions and planning its actions.

- This builds trust and helps us work better with AI systems.

The Future of Hierarchical Planning

AI could get better at making plans on its own, without much human input. This could lead to more autonomous systems in various fields. With improved planning skills, AI might help solve complex global issues. Think climate change, city planning, or even space exploration.

Multiple AI planners could work together, sharing knowledge and skills. This teamwork could lead to solving even trickier problems. We might see hierarchical planning in more parts of our daily lives. Smart homes could plan energy use, or AI assistants could help manage our schedules. As AI gets better at planning, it might find new solutions to old problems. This could lead to breakthroughs in science, medicine, and technology.

4.4 ANALYSIS OF PLANNING APPROACH UNCERTIANTY

Artificial intelligence (AI) uncertainty is when there's not enough information or ambiguity in data or decision-making. It is a fundamental concept in AI, as real-world data is often noisy and incomplete. AI systems must account for uncertainty to make informed decisions.

Sources of Uncertainty

Data Uncertainty: AI models are trained on data, and the quality and accuracy of the data can affect the performance of the model. Noisy or incomplete data can lead to uncertain predictions or decisions made by the AI system.

Model Uncertainty: AI models are complex and can have various parameters and hyperparameters that need to be tuned. The choice of model architecture, optimization algorithm, and hyperparameters can significantly impact the performance of the model, leading to uncertainty in the results.

Algorithmic Uncertainty: AI algorithms can be based on different mathematical formulations, leading to different results for the same problem. For example, different machine learning algorithms can produce different predictions for the same dataset.

Environmental Uncertainty: AI systems operate in dynamic environments, and changes in the environment can affect the performance of the system. For example, an autonomous vehicle may encounter unexpected weather conditions or road construction that can impact its ability to navigate safely.

Human Uncertainty: AI systems often interact with humans, either as users or as part of the decision-making process. Human behavior and preferences can be difficult to predict, leading to uncertainty in the use and adoption of AI systems.

Ethical Uncertainty: AI systems often raise ethical concerns, such as privacy, bias, and transparency. These concerns can lead to uncertainty in the development and deployment of AI systems, particularly in regulated industries.

Legal Uncertainty: AI systems must comply with laws and regulations, which can be ambiguous or unclear. Legal challenges and disputes can arise from the use of AI systems, leading to uncertainty in their adoption and implementation.

Uncertainty in AI Reasoning: AI systems use reasoning techniques to make decisions or predictions. However, these reasoning techniques can be uncertain due to the complexity of the problems they address or the limitations of the data used to train the models.

Uncertainty in AI Perception: AI systems perceive their environment through sensors and cameras, which can be subject to noise, occlusion, or other forms of interference. This can lead to uncertainty in the accuracy of the data used to train AI models or the effectiveness of AI systems in real-world applications.

Uncertainty in AI Communication: AI systems communicate with humans through natural language processing or computer vision. However, language and visual cues can be ambiguous or misunderstood, leading to uncertainty in the effective communication between humans and AI systems.

Types of Uncertainty

Uncertainty in artificial intelligence (AI) refers to the lack of complete information or the presence of variability in data and models. Understanding and modeling uncertainty is crucial for making informed decisions and improving the robustness of AI systems. There are several types of uncertainty in AI, including:

1. **Aleatoric Uncertainty:** This type of uncertainty arises from the inherent randomness or variability in data. It is often referred to as “data uncertainty.” For example, in a classification task, aleatoric uncertainty may arise from variations in sensor measurements or noisy labels.

2. **Epistemic Uncertainty:** Epistemic uncertainty is related to the lack of knowledge or information about a model. It represents uncertainty that can potentially be reduced with more data or better modeling techniques. It is also known as "model uncertainty" and arises from model limitations, such as simplifications or assumptions.
3. **Parameter Uncertainty:** This type of uncertainty is specific to probabilistic models, such as Bayesian neural networks. It reflects uncertainty about the values of model parameters and is characterized by probability distributions over those parameters.
4. **Uncertainty in Decision-Making:** Uncertainty in AI systems can affect the decision-making process. For instance, in reinforcement learning, agents often need to make decisions in environments with uncertain outcomes, leading to decision-making uncertainty.
5. **Uncertainty in Natural Language Understanding**: In natural language processing (NLP), understanding and generating human language can be inherently uncertain due to language ambiguity, polysemy (multiple meanings), and context-dependent interpretations.
6. **Uncertainty in Probabilistic Inference:** Bayesian methods and probabilistic graphical models are commonly used in AI to model uncertainty. Uncertainty can arise from the process of probabilistic inference itself, affecting the reliability of model predictions.
7. **Uncertainty in Reinforcement Learning:** In reinforcement learning, uncertainty may arise from the stochasticity of the environment or the exploration-exploitation trade-off. Agents must make decisions under uncertainty about the outcomes of their actions.
8. **Uncertainty in Autonomous Systems:** Autonomous systems, such as self-driving cars or drones, must navigate uncertain and dynamic environments. This uncertainty can pertain to the movement of other objects, sensor measurements, and control actions.
9. **Uncertainty in Safety-Critical Systems:** In applications where safety is paramount, such as healthcare or autonomous vehicles, managing uncertainty is critical. Failure to account for uncertainty can lead to dangerous consequences.
10. **Uncertainty in Transfer Learning:** When transferring a pre-trained AI model to a new domain or task, uncertainty can arise due to domain shift or differences in data distributions. Understanding this uncertainty is vital for adapting the model effectively.
11. **Uncertainty in Human-AI Interaction:** When AI systems interact with humans, there can be uncertainty in understanding and responding to human input, as well as uncertainty in predicting human behavior and preferences.

Techniques for Addressing Uncertainty

Probabilistic Logic Programming

Probabilistic logic programming (PLP) is a way to mix logic and probability to handle uncertainty in computer programs. This is useful for computer programmers when they are not completely sure about the facts and rules they are working with. PLP uses probabilities to help them make decisions and learn from data. They can use different techniques, like Bayesian logic programs or Markov logic networks, to put PLP into action. PLP is handy in various areas of artificial intelligence, like making guesses when we're not sure, planning when there are risks involved, and creating models with pictures and symbols.

Fuzzy Logic Programming

To deal with uncertainty in logic programming, there's a method called fuzzy logic programming (FLP). FLP combines regular logic with something called "fuzzy" logic. This helps programmers express things that are a bit unclear or not black and white. FLP also helps them make decisions and learn from this uncertain information.

Nonmonotonic Logic Programming

To deal with problems of inconsistency in logic programming, there's something called nonmonotonic logic programming (NMLP). This is a way of thinking in computer programming that doesn't strictly follow the rules of traditional logic. With NMLP, programmers can handle situations where things don't always go as expected. They can use techniques like negation as failure, default reasoning, and exceptions. NMLP also helps them make decisions and learn in situations that are not set in stone.

Paraconsistent Logic Programming

Paraconsistent logic programming (PLP) is a technique for dealing with conflicting information in logic programming. PLP adds a special type of logic called paraconsistent logic to the mix. This helps programmers work with contradictory facts and rules without creating chaos. It also lets them make sense of this contradictory information and learn from it.

Hybrid Logic Programming

Hybrid logic programming (HLP) is a way to handle situations where things are unclear or don't quite match up in logic programming. HLP brings together different styles of logic programming to help programmers work with complicated information. It lets them use various kinds of logic to express rules and complex facts and make sense of them. There are different methods to make HLP work, like using a mix of probabilities, fuzzy reasoning, and handling situations where things might not always follow the same rules. HLP is useful in various areas of artificial intelligence, like managing interactions between different computer systems and organizing information on the internet. It can also be used for creating structured knowledge systems.

Ways to Solve Problems with Uncertain Knowledge

Probability plays a central role in AI by providing a formal framework for handling uncertainty. AI systems use probabilistic models and reasoning to make informed decisions, assess risk, and quantify uncertainty, allowing them to operate effectively in complex and uncertain real-world scenarios. In probability, there are two ways to solve problems when we're not sure about the information:

- Bayes' rule
- Bayesian statistics

Bayes' Rule

Bayes' rule is an important tool in probability that lets us adjust our best guesses when we learn new stuff. It's a way to use what we already know and mix it with new information to make better guesses about what might happen. People use Bayes' rule a lot in artificial intelligence for things like sorting things into groups, making guesses about the future, and deciding what to do when things are uncertain. Mathematically, Bayes' theorem is expressed as follows:

$$P(A|B) = (P(B|A) * P(A)) / P(B)$$

Here,

The posterior probability, represented by P(A|B), is the chance of event A happening when event B has happened.

P(B|A) shows how likely event B is when event A has already happened.

The prior probability, P(A), is the initial chance of event A happening before any new information is considered.

P(B) is the probability of event B happening, whether or not event A has happened.

In AI, Bayes' theorem updates probabilities of hypotheses or predictions with new data or evidence. It is helpful for dealing with uncertainty and making decisions with incomplete or unclear information.

Bayesian Statistics

Bayesian statistics is a type of statistics that uses probability to analyze data. The framework helps us

make inferences and estimate probabilities using data and prior knowledge. Bayesian statistics has been used in different fields to handle uncertainty and make informed choices. It has been applied in environmental modeling, social sciences, and medical research.

Example:

Let's consider an example of a financial risk assessment system that utilizes probabilistic reasoning to handle uncertainty when deciding if loan applicants are creditworthy. This system is designed to determine whether an individual or a business is a suitable candidate for a loan based on various financial and personal factors, but these factors can be subject to uncertainty and ambiguity.

The system uses probabilistic reasoning techniques to address uncertainty in the following ways:

Prior Probabilities: The system assigns prior probabilities to different creditworthiness categories based on historical data and market conditions. These prior probabilities represent the initial beliefs about the likelihood of an applicant falling into each creditworthiness category before taking into account the applicant's specific financial and personal information.

Likelihoods: The system employs statistical models to estimate the likelihood of observing certain financial behaviors and personal characteristics given an applicant's creditworthiness category. For instance, it considers factors such as income, credit history, outstanding debt, and employment status. These likelihoods may be modeled with probabilistic distributions to account for the uncertainty inherent in the data.

Bayesian Updating: Bayes' rule is applied to update the probabilities of different creditworthiness categories based on the prior probabilities and the observed financial and personal information of the applicant. The updated probabilities, referred to as posterior probabilities, represent the revised beliefs about the likelihood of the applicant belonging to each creditworthiness category.

Decision-Making: The system uses the posterior probabilities of different creditworthiness categories to make a final lending decision. The decision can be based on a predetermined threshold, a decision-making rule, or a combination of factors. For example, if the posterior probability of an applicant being in the "low credit risk" category exceeds a certain threshold, the system may approve the loan. Alternatively, the system may generate a ranked list of creditworthiness categories, allowing the financial institution to decide the terms and conditions of the loan based on the level of risk they are willing to accept.

4.5 BASIC PROBABILITY NOTATION

Artificial Intelligence (AI) systems often operate under uncertainty. To represent and reason about uncertain events, probability theory provides a formal mathematical framework. In this chapter, we

introduce the basic probability notations and concepts that are fundamental to AI and Machine Learning.

Random Variables

A **random variable** is a variable whose values are outcomes of a random phenomenon.

- **Notation**:
  - Capital letters such as X,Y,ZX, Y, ZX,Y,Z denote random variables.
  - Lowercase letters such as x,y,zx, y, zx,y,z denote specific values taken by those variables.

Example:
Let XXX represent the weather. Possible values of XXX are x={sunny, rainy, cloudy}x = \{\text{sunny, rainy, cloudy}\}x={sunny, rainy, cloudy}.

Probability of Events
The probability of an event is a number between 0 and 1 that expresses how likely the event is to occur.

- **Notation**:
  P(X=x)P(X = x)P(X=x) represents the probability that the random variable XXX takes the value xxx.

Joint Probability

The joint probability distribution specifies the probability of two or more random variables taking specific values simultaneously.

- Notation:

  P(X=x,Y=y)P(X = x, Y = y)P(X=x,Y=y)

Marginal Probability

Marginal probability is derived from a joint distribution by summing (or integrating) over the other variables.

- **Notation**:

$$P(X = x) = \sum_{y} P(X = x, Y = y)$$

This allows us to focus on one variable by ignoring others.

Conditional Probability

Conditional probability expresses the likelihood of an event given that another event has already

occurred.

- **Notation**:

$$P(X = x \mid Y = y) = \frac{P(X = x, Y = y)}{P(Y = y)}$$

Example:
If the probability of playing outside is higher when it is sunny, conditional probability helps quantify this relationship.

Independence
Two variables are said to be **independent** if the occurrence of one does not affect the probability of the other.

- **Notation**:

$$P(X, Y) = P(X) \cdot P(Y) \quad \text{if } X \perp Y$$

Independence assumptions are widely used in AI models such as the Naïve Bayes classifier.

Bayes' Theorem

Bayes' theorem provides a way to update prior beliefs based on new evidence. It is a cornerstone of probabilistic reasoning in AI.

$$P(X \mid Y) = \frac{P(Y \mid X)\, P(X)}{P(Y)}$$

Expectation and Variance
Probabilistic reasoning often involves summarizing random variables using their expected value and variance.

Expectation (Mean):

$$E[X] = \sum_x x \cdot P(X = x)$$

Variance:

$$Var(X) = E[(X - E[X])^2]$$

BAYES RULE
Bayes Rule provides a mathematical way to update beliefs based on new evidence, forming the backbone of various AI methods, including machine learning models, robotics, natural language processing, and expert systems.

Key Terms in Bayes Rule
Prior Probability (P(H)): The initial belief or probability of a hypothesis (H) before observing new

evidence.

Example: Probability that an email is spam before reading it.

Likelihood (P(E|H)): The probability of observing evidence (E) assuming the hypothesis (H) is true.

Example: Probability of finding the word "discount" in a spam email.

Posterior Probability (P(H|E)): The updated probability of the hypothesis (H) after observing the evidence (E).

Example: Adjusted probability that an email is spam after seeing "discount" in the message.

Evidence (P(E)): The overall probability of observing the evidence across all hypotheses.

Mathematical Formula and Derivation of Bayes' Rule

$$P(H|E) = \frac{P(E|H) \times P(H)}{P(E)}$$

Where

P(H | E) = Posterior probability

P(E | H) = Likelihood

P(H) = Prior probability

P(E) = Marginal probability of evidence

Derivation (simplified):

From conditional probability rules:

$$P(H \cap E) = P(E|H) \times P(H) = P(H|E) \times P(E)$$

Rearranging gives Bayes' Rule.

Simple example:

Suppose 5% of emails are spam (P(Spam)=0.05P(\text{Spam}) = 0.05P(Spam)=0.05). If "discount" appears 90% of the time in spam emails but only 10% in non-spam, Bayes' Theorem helps update the probability that an email containing "discount" is spam.

Why Bayes' Theorem is Relevant in AI?

In artificial intelligence, uncertainty is a constant. AI systems often make decisions with incomplete, noisy, or ambiguous data. **Bayes' Theorem** provides a principled way for these systems to **reason probabilistically**, allowing them to update their understanding of the world as new evidence becomes available.

Probabilistic thinking is essential in AI because many real-world environments are dynamic and unpredictable. For example, a self-driving car must predict pedestrian behavior, a medical AI must diagnose based on partial symptoms, and a recommendation system must guess preferences from

limited user interactions. Bayes' Theorem enables AI to manage these uncertainties intelligently, rather than making rigid, deterministic decisions.

Moreover, **Bayesian reasoning closely mirrors human decision-making**. Humans continuously revise their beliefs based on new observations—whether diagnosing an illness, adjusting expectations in a conversation, or anticipating weather changes. Similarly, Bayesian AI models update probabilities to make **adaptive, context-aware decisions**.

In **supervised learning**, Bayesian techniques are used to estimate model parameters and make predictions with associated confidence levels. In **probabilistic models** like **Bayesian networks** and **Hidden Markov Models**, Bayes' Theorem forms the mathematical backbone, allowing AI to infer hidden states, forecast outcomes, and model complex systems under uncertainty.

Applications of Bayes Rule

1. Spam Detection

Spam filters in email systems commonly use Bayesian reasoning to classify incoming messages as "spam" or "not spam." The model starts with a prior probability of a message being spam. As it analyzes words or phrases within the email (evidence), it updates its belief based on the likelihood of those terms appearing in known spam or legitimate messages.

For example, if an email contains words like "free," "winner," or "discount," the probability of it being spam increases. Bayesian spam filters dynamically adapt over time, improving their accuracy as they learn from more emails.

2. Medical Diagnosis Systems

Medical AI systems leverage Bayes' Theorem to predict diseases based on observed symptoms. Doctors and diagnostic tools start with prior probabilities (e.g., prevalence of a disease in a population). When symptoms like fever, cough, or chest pain are observed (evidence), Bayes' Rule helps update the probability that a patient has a specific condition, such as pneumonia or COVID-19. This probabilistic reasoning allows for dynamic, evidence-based diagnosis, even when symptoms are overlapping or ambiguous. It reduces misdiagnosis and supports clinical decision-making with calculated risk assessments.

3. Machine Learning Algorithms (Naïve Bayes Classifier)

The Naïve Bayes Classifier is one of the simplest yet powerful machine learning algorithms, especially for text classification tasks like spam detection, sentiment analysis, and document categorization. It assumes that all features are independent given the class label—a strong assumption, but one that makes computation fast and scalable.

Using Bayes' Theorem, the model calculates the posterior probability of a text belonging to a certain

class and selects the class with the highest probability. Despite its simplicity, Naïve Bayes often achieves surprisingly high accuracy in real-world AI tasks.

4. Robotics and Perception

In robotics, Bayes' Theorem underpins probabilistic perception techniques like sensor fusion, simultaneous localization and mapping (SLAM), and robot navigation. Robots integrate uncertain sensor data (e.g., from cameras, LiDAR) using Bayesian updating to build maps, localize themselves, and navigate dynamically changing environments with resilience to noise and error.

Example:

Applying Bayes Rule in a Simple AI Problem

Problem Statement

Suppose a patient is tested for a rare disease that affects **1% of the population.**
The test is **95% accurate**, meaning:

- If a person has the disease, the test correctly identifies it 95% of the time (True Positive Rate).
- If a person does not have the disease, the test correctly returns negative 95% of the time (True Negative Rate).

Now, if a patient's test result comes back positive, what is the probability that the patient actually has the disease?

Given Data:
P(Disease) = 0.01 (Prior probability of disease)

P(No Disease) = 0.99

P(Positive | Disease) = 0.95 (Test sensitivity)

P(Positive | No Disease) = 0.05 (False positive rate)

We need to find: P(Disease | Positive) (Posterior probability).

Step-by-Step Calculation Using Bayes' Rule

Bayes' Theorem formula:

$$P(\text{Disease} \mid \text{Positive}) = \frac{P(\text{Positive} \mid \text{Disease}) \times P(\text{Disease})}{P(\text{Positive})}$$

First calculate P(Positive) (total probability of a positive test):

P(Positive) = P(Positive | Disease) × P(Disease) + P(Positive | No Disease) × P(No Disease)

P(Positive) = (0.95 × 0.01) + (0.05 × 0.99) = 0.0095 + 0.0495 = 0.059

Now, applying Bayes' Theorem:

$$P(\text{Disease} \mid \text{Positive}) = \frac{0.95 \times 0.01}{} = \frac{0.0095}{} \approx 0.161$$

0.059 0.059

Thus, even after a positive test, the probability that the patient actually has the disease is only about 16.1%.

Tree Diagram

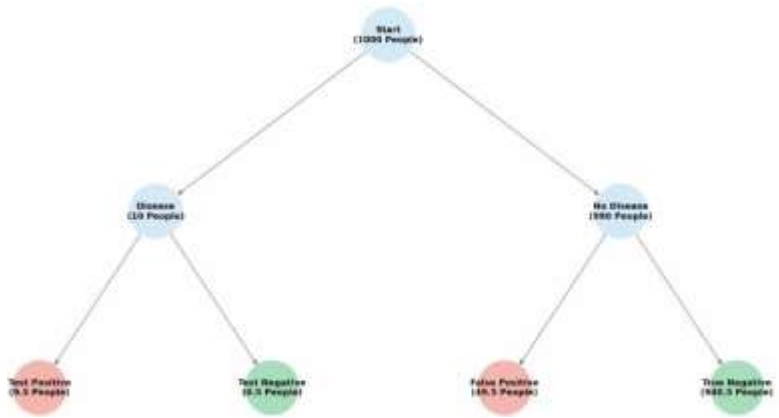


Fig : Enhanced Visual Tree Diagram for disease Diagnosis(Bayes' Rule)

Imagine a tree:

Start with 1000 people.

10 people have the disease (1%).

9.5 test positive (95% accuracy).

990 people do not have the disease.

49.5 still test positive (5% false positive rate).

Total positive tests = 9.5 + 49.5 = 59. Chance of actual disease among positives = 9.5 / 59 ≈ 16.1%.

Challenges and Limitations of Bayes' Rule:

1. **Requires Accurate Prior Probabilities**: If the initial prior assumptions are inaccurate or subjective, the resulting predictions can be significantly biased, leading to poor decision-making.
2. **Computationally Expensive for Complex Problems**: In high-dimensional spaces or large datasets, calculating exact posterior probabilities can become computationally intensive, sometimes requiring approximation methods like Monte Carlo simulations.
3. **Sensitive to Biased or Incomplete Data**: Bayesian models heavily rely on the quality and representativeness of both prior information and observed data. Biases or missing data can distorrt outcomes and reduce model reliability.

4.6 NAIVE BAYES' RULE

Naive Bayes is a machine learning classification algorithm that predicts the category of a data point using probability. It assumes that all features are independent of each other. Naive Bayes performs well in many real-world applications such as spam filtering, document categorization and sentiment analysis.

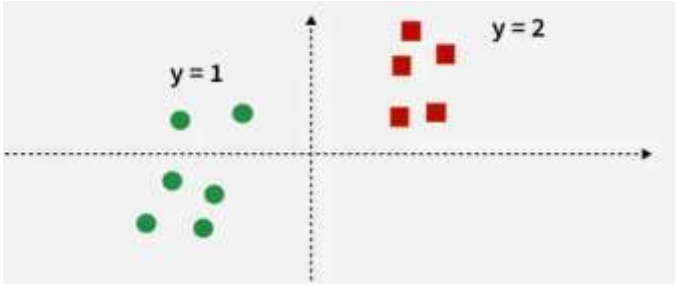


Fig : Original Data

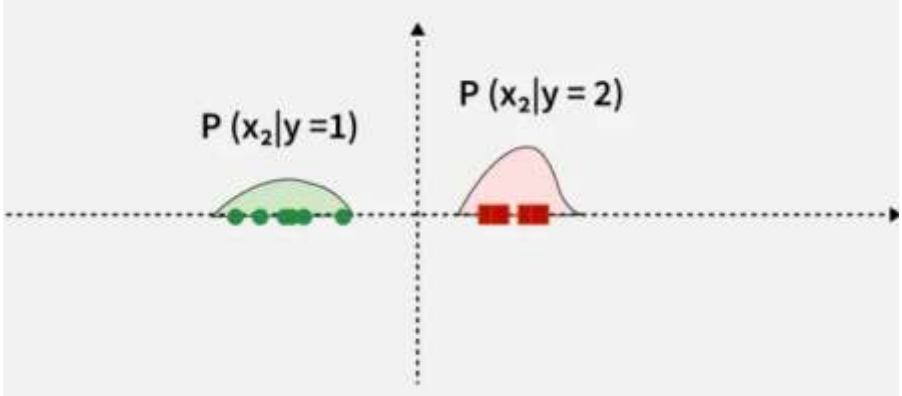


Fig : Estimation of First Dimension

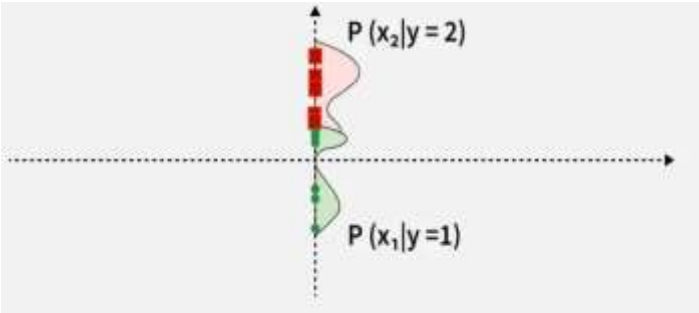


Fig : Estimation of Second Dimension

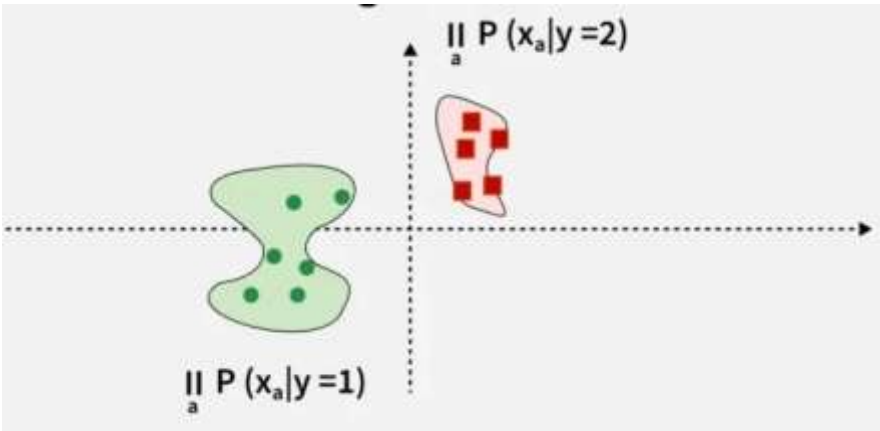


Fig : Resulting Data Distribution

Key Features of Naive Bayes:

- The Naive Bayes Classifier is a simple probabilistic classifier and it has very few number of parameters which are used to build the ML models that can predict at a faster speed than other classification algorithms.
- It is a probabilistic classifier because it assumes that one feature in the model is independent of existence of another feature. In other words, each feature contributes to the predictions with no relation between each other.
- Naive Bayes Algorithm is used in spam filtration, Sentimental analysis, classifying articles and many more.

Why it is Called Naive Bayes

It is named as "Naive" because it assumes the presence of one feature does not affect other features. The "Bayes" part of the name refers to its basis in Bayes' Theorem.

Consider a fictional dataset that describes the weather conditions for playing a game of golf. Given the weather conditions, each tuple classifies the conditions as fit("Yes") or unfit("No") for playing golf. Here is a tabular representation of our dataset.

	Outlook	Temperature	Humidity	Windy	Play Golf
0	Rainy	Hot	High	False	No
1	Rainy	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Sunny	Mild	High	False	Yes
4	Sunny	Cool	Normal	False	Yes
5	Sunny	Cool	Normal	True	No
6	Overcast	Cool	Normal	True	Yes
7	Rainy	Mild	High	False	No
8	Rainy	Cool	Normal	False	Yes

9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Overcast	Hot	Normal	False	Yes
13	Sunny	Mild	High	True	No

The dataset is divided into two parts i.e feature matrix and the response vector.

- Feature matrix contains all the vectors(rows) of dataset in which each vector consists of the value of dependent features. In above dataset, features are 'Outlook', 'Temperature', 'Humidity' and 'Windy'.
- Response vector contains the value of class variable (prediction or output) for each row of feature matrix. In above dataset, the class variable name is 'Play golf'.

Naive Bayes Working

1. Terminology

Consider a classification problem (like predicting if someone plays golf based on weather). Then:

- y*y* is the class label (e.g. "Yes" or "No" for playing golf)
- X=(x1,x2,...,xn)*X*=(*x*1,*x*2,...,*xn*) is the feature vector (e.g. Outlook, Temperature, Humidity, Wind)

A sample row from the dataset:
X=(Rainy, Hot, High, False),y=NoX=(Rainy, Hot, High, False),y=No
This represents:
What is the probability that someone will not play golf given that the weather is Rainy, Hot, High humidity, and No wind?

2. The Naive Assumption

The "naive" in Naive Bayes comes from the assumption that all features are independent given the class. That is:

$$P(x_1, x_2, ..., x_n|y) = P(x_1|y) \cdot P(x_2|y) \cdots P(x_n|y)$$

Thus, Bayes' theorem becomes:

$$P(y|x_1, ..., x_n) = \frac{P(y) \cdot \prod_{i=1}^{n} P(x_i|y)}{P(x_1)P(x_2)...P(x_n)}$$

Since the denominator is constant for a given input, we can write:

$$P(y|x_1, ..., x_n) \propto P(y) \cdot \prod_{i=1}^{n} P(x_i|y)$$

3. Constructing the Naive Bayes Classifier

We compute the posterior for each class *yy* and choose the class with the highest probability:

$$\hat{y} = \arg\max_y P(y) \cdot \prod_{i=1}^{n} P(x_i|y)$$

This becomes our Naive Bayes classifier.

4. Example: Weather Dataset

Let's take a dataset used for predicting if golf is played based on:

- **Outlook**: Sunny, Rainy, Overcast
- **Temperature**: Hot, Mild, Cool
- **Humidity**: High, Normal
- **Wind**: True, False

	Yes	No	P(yes)	P(no)
Sunny	3	2	3/10	2/4
Overcast	4	0	4/10	0/4
Rainy	3	2	3/10	2/4
Total	10	4	100%	100%

Fig : Outlook

	Yes	No	P(yes)	P(no)
Hot	2	2	2/9	2/5
Mild	4	2	4/9	2/5
Cool	3	1	3/9	1/5
Total	9	5	100%	100%

Fig : Temperature

	Yes	No	P(yes)	P(no)
Hot	3	4	3/9	4/5
Normal	6	1	6/9	1/5
Total	9	5	100%	100%

Fig : Humidity

	Yes	No	P(yes)	P(no)
False	6	2	6/9	2/5
True	3	3	3/9	3/5
Total	9	5	100%	100%

Fig : Wind

Play		P(Yes)/P(No)
Yes	9	9/14
No	5	5/14
Total	14	100%

Fig : Final Goal

Example Input: X=(Sunny,Hot,Normal,False)X=($Sunny$,Hot,$Normal$,$False$)
Goal: Predict if golf will be played (Yes or No).

5. Pre-computation from Dataset
Class Probabilities:
From dataset of 14 rows:

- P(Yes)=914P(Yes)=149
- P(No)=514P(No)=145

Conditional Probabilities

Feature	Value	P (Value | Yes)	P (Value | No)
Outlook	Sunny	2/9	3/5
Temperature	Hot	2/9	2/5
Humidity	Normal	6/9	1/5
Wind	False	6/9	2/5

6. Calculate Posterior Probabilities

For Class = Yes:
P(Yes | today)∝29·29·69·69·914P(Yes | today)∝92·92·96·96·149 P(Yes | today)≈0.02116P(Yes | today)≈0.02116

For Class = No:
P(No | today)∝35·25·15·25·514P(No | today)∝53·52·51·52·145 P(No | today)≈0.0068P(No | today)≈0.0068

7. Normalize Probabilities
To compare:

$$P(\text{Yes} \mid \text{today}) = \frac{0.02116}{0.02116+0.0068} \approx 0.756$$

$$P(\text{No} \mid \text{today}) = \frac{0.0068}{}$$

$$0.02116+0.0068 \approx 0.244$$

8. Final Prediction

Since:

P(Yes | today)>P(No | today)P(Yes | today)>P(No | today)

The model predicts: **Yes (Play Golf)**

Types of Naive Bayes Model

There are three types of Naive Bayes Model:

1. Gaussian Naive Bayes

In Gaussian Naive Bayes, continuous values associated with each feature are assumed to be distributed according to a Gaussian distribution. A Gaussian distribution is also called Normal distribution. When plotted, it gives a bell shaped curve which is symmetric about the mean of the feature values as shown below:

2. Multinomial Naive Bayes

Multinomial Naive Bayesis used when features represent the frequency of terms (such as word counts) in a document. It is commonly applied in text classification, where term frequencies are important.

3. Bernoulli Naive Bayes

Bernoulli Naive Bayes deals with binary features, where each feature indicates whether a word appears or not in a document. It is suited for scenarios where the presence or absence of terms is more relevant than their frequency. Both models are widely used in document classification tasks.

4.7 THE SEMANTICS OF BAYESIAN NETWORK

A Bayesian network is a type of graphical model that uses probability to determine the occurrence of an event. It is also known as a belief network or a causal network. It consists of directed cyclic graphs (DCGs) and a table of conditional probabilities to find out the probability of an event happening. It contains nodes and edges, where edges connect the nodes. The graph is acyclic - meaning there is no direct path where one node can reach another. The table of probability, on the other hand, shows the likelihood that a random variable will take on certain values.

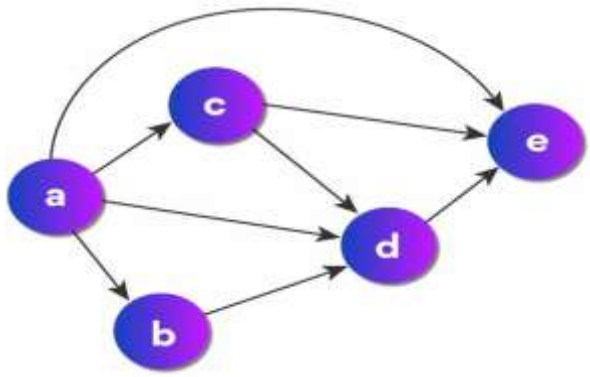


Fig : Directed Acyclic Graph

The above image illustrates a directed acyclic graph. There are five nodes, namely, a, b, c, d, and e. From the above graph, we can get the following information:

- Node a is the parent of node b, c, and e, and node b, c, and e are the child nodes of node a.
- Node b and c are the parent nodes of d.
- Node e is the child node of nodes d, c, and a.

It is important to note the relationships between the nodes. Bayesian networks fall under probabilistic graphical techniques; hence, probability plays a crucial role in defining the relationship among these nodes.

Types of Probabilities

Joint probability

Joint probability is a probability of two or more events happening together. For example, the joint probability of two events A and B is the probability that both events occur, $P(A \cap B)$.

Conditional Probability

Conditional probability defines the probability that event B will occur, given that event A has already occurred. There are two ways joint probability can be represented:

If A and B are dependent events P(B/A) = P(A and B) / P(A)

If A and B are independent events : P(B/A) = P(B)

The conditional probability distribution of each node is represented by a table called the "node table". It contains two columns, one for each possible state of the parent node (or "parent random variable")

and one for each possible state of the child node (or "child random variable").

Problem:

Based on the evidence on who will or will not call, find the probability of a burglary occurring in the house. In a Bayesian network, we can see nodes as random variables.

There are five nodes:

- **Burglary (B)**
- **Earthquake (E)**
- **Alarm (A)**
- **Chris calls (C)**
- **Martin calls (M)**

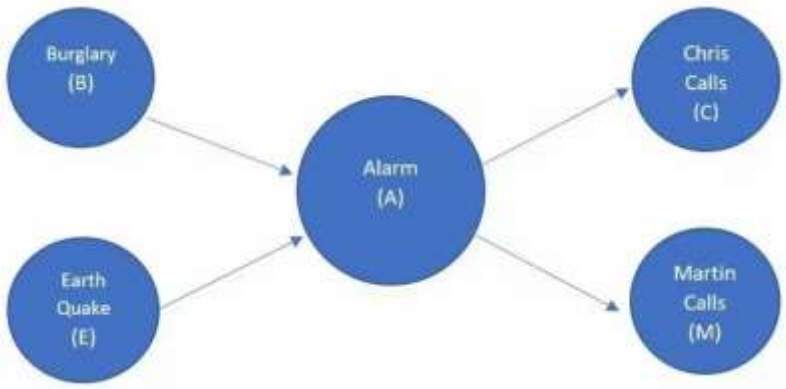


Fig : Casual dependencies

Links act as causal dependencies that define the relationship between the nodes. Both Chris and Martin call when there is an alarm. Let's write the probability distribution function formula for the above five nodes.

P[B ,E , A , C, M] = P[C | A] P[M | A] P[A | B , E] P[B | E] P[E]

In the above probability distribution formula P[B,E,A,C,M] represents probability distribution of all the 5 nodes.

Now, let's look at the observed values for each of the nodes with the table of probabilities:

Node B:

True	0.015
False	0.985

Node E:

True	0.010
False	0.990

Node A:

B	E	P(A = True)	P(A = False)
True	True	0.90	0.10
True	False	0.95	0.05
False	False	0.85	0.75
False	True	0.99	0.99

Node C:

Alarm	P(C = True)	P(C= False)
True	0.90	0.10
False	0.10	0.90

Node M:

Alarm	P(M = True)	P(M= False)
True	0.85	0.15
False	0.05	0.95

Based on the above observed values, the conditional values can be derived and, therefore, the probability distribution can be calculated.

Conditional values for the above nodes are:

B	E	P(A = True)	P(A = False)
True	True	0.90	0.10
True	False	0.95	0.05

False	False	0.50	0.75
False	True	0.05	0.95

Alarm	P(C = True)	P(C= False)
True	0.90	0.10
False	0.10	0.90

Alarm	P(M = True)	P(M= False)
True	0.85	0.15
False	0.05	0.95

To calculate the joint distribution, we use the formula:

P[B,E,A,C,M] = P[C | A] P[M | A] P[A | B, E] P[B | E] P[E]

Applications of Bayesian networks

Fig : Applications of Bayesian networks

Spam filtering: A spam filter is a program that helps in detecting unsolicited and spam mails. Bayesian spam filters check whether a mail is spam or not. They use filtering to learn from spam and ham messages.

Biomonitoring: This involves the use of indicators to quantify the concentration of chemicals in the human body. Blood or urine is used to measure the same.

Information retrieval: Bayesian networks assist in information retrieval for research, which is a constant process of extracting information from databases. It works in a loop. Hence, we have to continuously reconsider and redefine our research problem to avoid data overload.

Image processing: A form of signal processing, image processing uses mathematical operations to convert images into digital format. Once images are converted, their quality can be enhanced with

more operations. The input image doesn't necessarily have to be in the form of an image; it could be a photograph or a video frame.

Gene regulatory network: A Bayesian network is an algorithm that can be applied to gene regulatory networks in order to make predictions about the effects of genetic variations on cellular phenotypes. Gene regulatory networks are a set of mathematical equations that describe the interactions between genes, proteins, and metabolites. They are used to study how genetic variations affect the development of a cell or organism.

Turbo code: Turbo codes are a type of error correction code capable of achieving very high data rates and long distances between error correcting nodes in a communications system. They have been used in satellites, space probes, deep-space missions, military communications systems, and civilian wireless communication systems, including WiFi and 4G LTE cellular telephone systems.

Document classification: This is a problem often encountered in computer science and information science. Here, the main issue is to assign a document multiple classes. The task can be achieved manually and algorithmically. Since manual effort takes too much time, algorithmic documentation is done to complete it quickly and effectively.

4.8 EXACT INFERENCE IN BAYESIAN NETWORK

4.8.1 Probabilistic Inference and Learning

In practice, exact inference is not used widely, and most probabilistic inference algorithms are approximate. Nevertheless, it is important to understand exact inference and its limitations.

There are two typical tasks with graphical models: inference and learning. Given a graphical model M that describes a unique probability distribution P, inference means answering queries about P_M, e.g., $P_M(X|Y)$. Learning means obtaining a point estimate of model M from data D. However, in statistics, both inference and learning are commonly referred to as either inference or estimation.

From the Bayesian perspective, for example, learning p(M|D) is actually an inference problem. When not all variables are observable, computing point estimates of M needs inference to impute the missing data.

4.8.1.1 Likelihood

One of the simplest queries one may ask is likelihood estimation. The likelihood estimation of a probability distribution is to compute the probability of the given evidence, where evidence is an assignment of values to a subset of variables. Likelihood estimation involves marginalization of the other variables. Formally, let e and x denote evidence and the remaining variables, respectively, the likelihood of e is P(e) = P(x1 x1 xk xk e)

$$P(e) = \sum_{x1} .. \sum_{xk}^{n} P(x1, \dots . xk, e).$$

From the perspective of computational statistics, calculating this is computationally expensive because the computation involves exploring exponentially many con gurations.

1.2 Conditional Probability Another type of queries is the conditional probability of variables given evidence. The probability of variables X given evidence e is

$$P(X|e) = \frac{P(X,e)}{P(e)} = \frac{P(X,e)}{\sum_x P(X=x,e)}.$$

This is called a posteriori belief in X given evidence e. As in likelihood estimation, calculating a conditional probability involves marginalization of variables that are not of our interest (i.e., the summation part in the equation). We will be covering e cient algorithms for marginalization later in this class. Calculating a conditional probability is called in di erent ways depending on the query nodes. When the query node is a terminal variable in a directed graphical model, the inference process is called prediction. But if the query node is an ancestor of the evidence, the inference process is called diagnosis; e.g., the probability of disease/fault given evidence symptoms. For instance, in the deep belief network, a restricted Boltzmann machine with multiple hidden layers, the hidden layers are estimated given data

y1	**y2**	**P(y1,y2)**
0	0	0.35
0	1	0.05
1	0	0.3
1	1	0.3

Table : MPA

Most Probable Assignment

We may query the most probable assignment (MPA) for a subset of variables in the domain. Given evidence e, query variables Y, and the other variables Z, the MPA of Y is

$$\mathrm{MPA}(Y|e) = \arg\max_y P(y|e) = \arg\max_y \sum_z P(y,z|e).$$

The MPA of a variable depends on its context. For example, in Table 1, the MPA of Y1 is 1 because P(Y1 = 0) = 035+005 = 04 and P(Y1 = 1) = 03+03 = 06. However, when we include Y2 as context, the MPA of (Y1

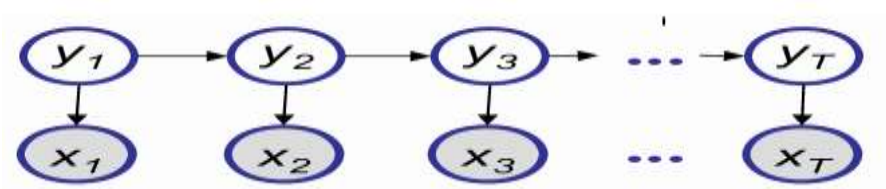
Y2) is (00) because P(Y1 = 0Y2 = 0) = 035 is the highest value in the table.

This is related to multi-task learning in machine learning. Multi-task learning indicates solving multiple learning tasks jointly instead of learning task-specific models separately ignoring context. For example, we may improve the accuracy of a classifier of having lunch and that of a classifier of having coffee if we jointly model having lunch and having coffee together, compared to making the two classifiers separately.

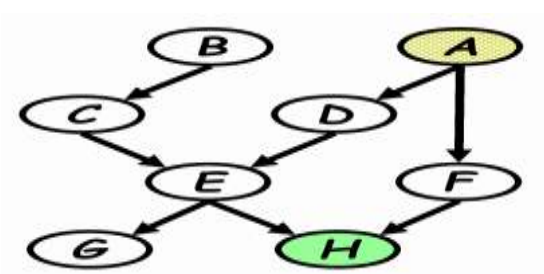
Approaches to Inference

There are two types of inference techniques: exact inference and approximate inference. Exact inference algorithms calculate the exact value of probability P(X|Y). Algorithms in this class include the elimination algorithm, the message-passing algorithm (sum-product, belief propagation), and the junction tree algorithms. We will not cover the junction tree algorithms in the class because they are outdated and confusing. The time complexity of exact inference on arbitrary graphical models is NP-hard. However, we can improve efficiency for particular families of graphical models. Approximate inference techniques include stochastic simulation and sampling methods, Markov chain Monte Carlo methods, and variational algorithms.

(a) Chain



(b) Conditional Random Field



(c) Hidden Morkov Model

(d) General Directed Graphical Model

Elimination

3.1 Chains

Let's first consider inference on the simple chain in Figure1a.The probability P(E=e) can be calculated as

$$P(e) = \sum_d \sum_c \sum_b \sum_a P(a, b, c, d, e).$$

This naïve summation enumerates over exponentially many configurations of the variables and thus is in efficient. But if we use the chain structure, the marginal probability can be calculated as

$$\begin{aligned}
P(e) &= \sum_d \sum_c \sum_b \sum_a P(a)P(b|a)P(c|b)P(d|c)P(e|d) \\
&= \sum_d P(e|d) \sum_c P(d|c) \sum_b P(c|b) \sum_a P(a)P(b|a) \\
&= \sum_d P(e|d) \sum_c P(d|c) \sum_b P(c|b)P(b) \\
&= \sum_d P(e|d) \sum_c P(d|c)P(c) \\
&= \sum_d P(e|d)P(d).
\end{aligned}$$

The summation in each line involves enumerating over only two variables, and there are four such summations. Therefore, in general, the time complexity of the elimination algorithm is $O(nk^2)$,where n is the number of variables and k is the possible values of each variable. That is, for simple chains, exact inference can be done in polynomial time as opposed to the exponential time for the naïve approach.

Hidden Markov Model

Let's consider the hidden Markov model in Figure1c. The conditional probability of variable Yi given X can be calculated as

In fact, m(x1 y2) is equal to the marginal probability P(x1 y2).

$$\begin{aligned}
P(y_i|x_1, \cdots, x_T) &= \sum_{y_1} \cdots \sum_{y_{i-1}} \sum_{y_{i+1}} \cdots \sum_{y_T} P(y_1, \cdots, y_T, x_1, \cdots, x_T) \\
&= \sum_{y_1} \cdots \sum_{y_{i-1}} \sum_{y_{i+1}} \cdots \sum_{y_T} P(y_1)P(x_1|y_1) \cdots P(y_T|y_{T-1})P(x_T|y_T) \\
&= \sum_{y_2} \cdots \sum_{y_{i-1}} \sum_{y_{i+1}} \cdots \sum_{y_T} P(x_2|y_2) \cdots P(y_T|y_{T-1})P(x_T|y_T) \sum_{y_1} P(y_1)P(x_1|y_1)P(y_2|y_1) \\
&= \sum_{y_2} \cdots \sum_{y_{i-1}} \sum_{y_{i+1}} \cdots \sum_{y_T} P(x_2|y_2) \cdots P(y_T|y_{T-1})P(x_T|y_T)m(x_1, y_2) \\
&= \sum_{y_3} \cdots \sum_{y_{i-1}} \sum_{y_{i+1}} \cdots \sum_{y_T} P(x_3|y_3) \cdots P(y_T|y_{T-1})P(x_T|y_T)m(x_1, x_2, y_3) \\
&= \vdots
\end{aligned}$$

Undirected Chains

Let's consider the undirected chain in Figure1b.With the elimination algorithm, the marginal probability P(E=e) can be calculated similarly as

$$
\begin{aligned}
P(e) &= \sum_d \sum_c \sum_b \sum_a \frac{1}{Z} \phi(b,a)\phi(c,b)\phi(d,c)\phi(e,d) \\
&= \frac{1}{Z} \sum_d \sum_c \sum_b \phi(c,b)\phi(d,c)\phi(e,d) \sum_a \phi(b,a) \\
&= \vdots
\end{aligned}
$$

In general, we can view the task at hand as that of computing the value of an expression of the form

$$\sum_z \prod_{\phi \in F} \phi,$$

Where F is a set of factors. This task is called the sum-product inference.

4.9 CAUSAL NEWORKS

Causal AI is a branch of Artificial Intelligence (AI) that focuses on understanding and modeling cause-and-effect relationships, rather than just identifying patterns or correlations as other traditional Machine Learning systems do.

Causal AI vs traditional AI: Key differences

Traditional AI systems focus on identifying patterns, correlations, and making predictions based on data. To do so, they rely on statistical and Machine Learning (ML) techniques to find associations in data.

Finally, while traditional (non-causal) AI methods have many applications, the key takeaway from this section is: Correlation does not imply causation.

- Correlation occurs when two variables move together, but this does not prove that one causes the other.
- Causation means that one variable directly influences the other.

Returning to the ice cream versus shark attacks example from the introduction, we can see that ice cream sales and shark attack incidents are correlated (both increase in summer), but ice cream does not cause shark attacks.

The core of Causal AI is the goal of establishing cause-and-effect relationships between variables. However, there are also two fundamental concepts essential for understanding and manipulating these relationships within a system:

Counterfactual reasoning

Counterfactual reasoning involves considering hypothetical scenarios to understand how changes to certain variables can alter outcomes. I like to see Counterfactual Reasoning as asking questions such as: “What would have happened if variable X had taken a different value?”. That is a “What If” question.

In our ice cream versus shark attacks example, we could ask ourselves:“What if ice cream sales had been lower? Would there have been fewer shark attacks?” In this counterfactual scenario, we imagine an alternate version of reality where ice cream sales decrease while everything else remains the same. Since both ice cream sales and shark attacks increase in summer due to warmer weather (a common cause), reducing ice cream sales would not affect shark attacks. This demonstrates correlation, not causation.

Interventions

Interventions refer to the deliberate manipulation of variables within a system to observe changes in other variables. In this case, interventions involve actively changing a variable to a specific value, allowing us to study its direct effects on the system.

In our example, an intervention could be:

“Let’s ban ice cream sales on hot days (setting ice cream sales to 0). Will this reduce shark attacks?”

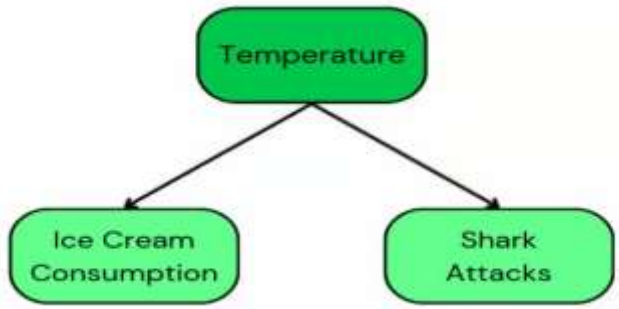
Here, we are deliberately altering the system by removing ice cream sales. The most likely outcome is no change in shark attacks since banning ice cream does not stop people from swimming. This simple example illustrates that intervening in ice cream sales has no causal effect on shark attacks.

Causal AI models

Directed Acyclic Graphs

Directed Acyclic Graphs (DAGs) encode the directional relationships between variables without cycles, meaning there are no loops where a variable can indirectly cause itself. They are used to depict causal relations in a binary way: Either there is a cause-effect relation between two variables, in which case there is a directed edge from cause to effect in the graph, or there is none, and therefore there is also no edge.

For example, the DAG of our original example of ice cream consumption and shark attacks shows the following causal pathways:



This DAG tells us that temperature is a common cause for both ice cream consumption and shark attacks. In addition, it shows that there is no causality between ice cream consumption and shark attacks. DAGs are simple representations, but they can become more complex by adding more information to the pathways.

Structural Causal Models

Structural Causal Models (SCMs) include equations that model how variables are influenced by others. To move from a DAG to an SCM, we need to specify a system of structural equations that quantitatively describe how each variable is generated.

Concretely, a structural equation in this context is a mathematical equation that expresses one variable (outcome) as a function of one or more other variables (causes) plus a disturbance term representing all other influences on that outcome.

Example

$$\begin{aligned} t &= U_t \\ IceCream &= a \times t + U_i \\ SharkAttacks &= b \times t + U_s \end{aligned}$$

Since temperature (t) is an exogenous variable, meaning that it is not caused by the other two, we assume it is determined by external factors. In the equations above, both ice cream consumption and shark attacks follow a linear model on the temperature, showing how a change in temperature propagates to both outcomes.

UNIT V

AI REAL-WORLD APPLICATIONS

Fraud Detection in Banking and Finance - AI-Powered Voice Assistants - Object Detection in Autonomous Vehicles - AI in Self-Driving Cars - Automated Grading and Feedback Systems - Al in Music Composition and Creativity - Traffic Management and Smart Transportation -Case Study on AI - Assisted X-Ray Analysis.

5.1 Fraud Detection in Banking and Finance

The banking and finance industry plays a crucial role in global economic stability, but it is also a prime target for fraudulent activities such as credit card fraud, identity theft, money laundering, and phishing. Fraud not only leads to direct financial losses but also damages trust between customers and institutions. Traditional rule-based systems often fail to detect sophisticated fraud because fraudsters continuously adapt their methods. In this context, Artificial Intelligence (AI) has emerged as a powerful tool for fraud detection. By leveraging machine learning (ML), natural language processing (NLP), and deep learning, financial institutions can identify anomalies, predict fraudulent transactions, and protect both customers and organizations.

Types of Fraud in Banking and Finance

1. Credit Card Fraud – Unauthorized use of card details for purchases.
2. Identity Theft – Impersonating a customer to access accounts or loans.
3. Phishing and Social Engineering – Trick customers into revealing sensitive data.
4. Money Laundering – Concealing illegally obtained money by passing it through complex transactions.
5. Loan Fraud – Using false documents or fake identities to secure loans.
6. AI is applied across all these categories to enhance detection and prevention.

Role of Artificial Intelligence in Fraud Detection

AI models are capable of:

Analysing large-scale data from millions of transactions in real time.

Identifying unusual patterns that deviate from normal customer behaviour.

Learning and adapting as fraudsters change their strategies.

Reducing false positives, which are common in rule-based systems.

Machine Learning Models

Supervised Learning – Models are trained on historical labeled data (fraud vs. non-fraud transactions). Algorithms such as logistic regression, decision trees, and support vector machines are common.

Unsupervised Learning – Used when labelled data is limited. Clustering and anomaly detection techniques flag suspicious transactions.

Deep Learning – Neural networks (e.g., CNNs, RNNs, LSTMs) capture complex relationships in transaction sequences.

Natural Language Processing (NLP)

Detects phishing emails and fraudulent communication.

Analyzes unstructured text in applications, claims, or customer support logs.

Real-Time Monitoring Systems

AI enables **real-time fraud detection** by instantly analyzing transaction details such as location, amount, time, and device used. For example, if a customer's credit card is used simultaneously in two different countries, the AI system can block the transaction immediately.

Case Studies and Applications

PayPal – Uses AI-driven deep learning to analyze billions of transactions per year, detecting unusual spending patterns.

JPMorgan Chase – Employs machine learning models to detect money laundering activities.

Mastercard Decision Intelligence – An AI system that evaluates each transaction in real time, reducing false declines.

These examples show how leading financial institutions rely on AI to protect customers and minimize losses.

Benefits of AI in Fraud Detection

Accuracy: Reduces false positives and improves fraud detection rates.

Scalability: Can handle millions of transactions per second.

Proactiveness: Detects and prevents fraud before it impacts the customer.

Adaptive Learning: Continuously updates models with new fraud patterns.

Challenges and Limitations

Data Privacy Issues: Financial data is highly sensitive, requiring strict compliance (e.g., GDPR, RBI, PCI DSS).

Bias in Algorithms: Poorly trained models may unfairly flag certain demographics.

High Implementation Cost: Developing AI systems requires infrastructure, expertise, and investment.

Evolving Fraud Techniques: Criminals are constantly innovating, demanding frequent model retraining.

Future of AI in Fraud Detection

Explainable AI (XAI): Enhancing transparency so financial institutions understand why transactions are flagged.

Federated Learning: Training models across multiple banks without sharing sensitive customer data.

Blockchain Integration: Combining blockchain's immutability with AI's analytical power.

Advanced Biometric Security: Voice recognition, facial recognition, and behavioral biometrics.

AI is expected to become even more central to fraud prevention as banks embrace

Predictive, proactive, and autonomous systems.

Fraud detection in banking and finance is a complex challenge due to the constantly evolving nature of fraudulent techniques. Traditional systems are no longer sufficient to safeguard financial institutions and customers. Artificial Intelligence provides robust, scalable, and adaptive solutions for detecting anomalies, analysing vast amounts of data in real time, and preventing financial crimes. While challenges such as bias, privacy, and cost remain, AI's role in fraud detection will only expand, ensuring safer and more secure banking experiences in the digital age.

5.2 AI-Powered Voice Assistants

Voice assistants are one of the most common applications of Artificial Intelligence (AI) in everyday life. Examples include Apple's Siri, Amazon Alexa, Google Assistant, and Microsoft Cortana. These systems enable users to interact with devices using natural speech rather than traditional input methods like typing or clicking.

The core technologies behind voice assistants are:

Automatic Speech Recognition (ASR): Converts spoken words into text.

Natural Language Processing (NLP): Understands the meaning and intent of the text.

Text-to-Speech (TTS): Converts the system's response back into spoken words.

How Voice Assistants Work

A voice assistant typically follows these steps:

Wake Word Detection:

The assistant stays in a passive listening mode, activated by a wake word (e.g., “Hey Siri” or “Ok Google”).

Lightweight neural networks are used to detect the wake word with minimal battery consumption.

Speech Recognition (ASR): Captures the user’s speech and converts it into text.

Algorithms: Hidden Markov Models (HMM), Deep Neural Networks (DNN), Recurrent Neural Networks (RNN).

Modern systems use end-to-end deep learning models for higher accuracy.

Natural Language Understanding (NLU):

Processes the transcribed text to extract meaning.

Tasks include: intent recognition (e.g., “set an alarm”), entity extraction (e.g., “7 a.m.”).

Uses machine learning models such as transformers (BERT, GPT-based models).

Action/Response Generation:

The system decides what action to perform (e.g., play music, provide weather updates).

Uses dialogue management systems and knowledge bases.

Text-to-Speech (TTS):

Converts the assistant’s response back into natural-sounding speech.

Old systems used rule-based synthesis; modern systems use neural TTS models like WaveNet.

Applications of Voice Assistants

Smart Homes: Controlling IoT devices like lights, thermostats, and appliances.

Customer Service: Automated support for queries, reducing human workload.

Healthcare: Helping patients with reminders for medicine and providing health tips.

Education: Assisting students in learning by answering questions and explaining concepts.

Banking & Finance: Voice-enabled transactions and account inquiries.

Entertainment: Playing music, audiobooks, and managing media.

Advantages of AI-Powered Voice Assistants

Hands-free and convenient interaction.

Personalization based on user preferences and history.

Multi-tasking support (e.g., asking for directions while driving).

Accessibility for visually impaired or disabled users.

Integration with multiple services and applications.

Challenges and Limitations

Accuracy Issues: Misinterpretation of accents, dialects, or background noise.

Privacy Concerns: Voice data often stored on cloud servers, leading to potential misuse.

Security Risks: Vulnerable to spoofing attacks or unauthorized access.

Limited Context Understanding: Struggles with multi-turn, complex conversations.

Dependence on Internet: Most assistants require connectivity to function effectively.

Technologies Behind Voice Assistants

ASR Tools: Google Speech-to-Text, CMU Sphinx, DeepSpeech.

NLP Frameworks: spaCy, NLTK, Hugging Face Transformers.

TTS Systems: Amazon Polly, Google WaveNet, Tacotron.

Dialogue Management: Rasa, Microsoft Bot Framework, Dialogflow.

Future of Voice Assistants

More Human-like Conversations: With advanced generative AI (GPT-based models), assistants will engage in natural dialogues.

Multimodal Interaction: Combining speech with gesture, vision, and context awareness.

On-Device AI: More processing will shift to edge devices for faster, private interactions.

Integration with AR/VR: Voice commands will become central in immersive experiences.

Emotional Intelligence: Assistants may detect user emotions and respond empathetically.

Case Studies

Amazon Alexa: Dominates smart home applications, supports over 100,000 skills.

Google Assistant: Strong contextual understanding and integration with Google services.

Apple Siri: Focuses on privacy and device-based processing.

Healthcare Example: Mayo Clinic's Alexa skill providing first-aid guidance.

AI-powered voice assistants are transforming how humans interact with technology by making communication natural, intuitive, and accessible. With rapid progress in deep learning, NLP, and edge AI, future assistants will be even more intelligent, secure, and human-like. However, challenges around privacy, security, and ethical usage must be carefully managed.

5.3 Object Detection in Autonomous Vehicles

Autonomous vehicles, also known as self-driving cars, rely heavily on artificial intelligence (AI) and computer vision to understand and interact with their surroundings. Among the most crucial tasks is object detection, which involves identifying and classifying various objects such as pedestrians, vehicles, traffic signs, cyclists, and obstacles on the road. Reliable object detection ensures that the vehicle can make safe driving decisions, avoid collisions, and comply with traffic regulations.

The challenge lies in achieving real-time, highly accurate detection under varying environmental conditions such as different lighting, weather, and traffic scenarios. This section explores the theoretical foundations of object detection in autonomous vehicles, the underlying technologies, common algorithms, and existing challenges.

Importance of Object Detection in Autonomous Vehicles

Object detection plays a central role in the overall perception system of self-driving cars. Without accurate detection, higher-level decision-making processes such as path planning and motion control cannot function safely.

1. Safety – Detecting pedestrians and other vehicles prevents accidents.
2. Navigation – Recognizing traffic signs, lanes, and signals guides decision-making.
3. Regulatory Compliance – Ensures vehicles adhere to road rules by interpreting signs and signals.
4. Efficiency – Helps optimize lane changes, overtaking, and adaptive cruise control.

Technologies Used in Object Detection

Autonomous vehicles use a combination of sensors and AI algorithms for object detection.

1. Sensor Technologies

- Cameras – Capture high-resolution images for detecting traffic lights, signs, and lane markings.
- LiDAR (Light Detection and Ranging) – Provides 3D mapping of the environment by measuring the time taken for light pulses to reflect from surfaces.
- Radar – Detects objects' distance and speed, particularly effective in adverse weather.
- Ultrasonic Sensors – Used for close-range detection, such as parking assistance.

2. Sensor Fusion

Since no single sensor is perfect, autonomous vehicles rely on sensor fusion, which integrates data from multiple sensors to create a comprehensive understanding of the environment. For example, cameras provide visual detail, while LiDAR ensures depth accuracy.

Object Detection Algorithms

Over the years, computer vision techniques for object detection have evolved significantly, from traditional image processing methods to deep learning-based approaches.

1. Traditional Computer Vision Approaches

- Haar Cascades and HOG (Histogram of Oriented Gradients) – Earlier methods that detected objects using handcrafted features.
- Limitations – Struggled with variations in lighting, angles, and occlusions.

2. Deep Learning-Based Approaches

Deep learning revolutionized object detection by enabling systems to learn features directly from data. Convolutional Neural Networks (CNNs) form the backbone of most modern methods.

a. Region-Based Convolutional Neural Networks (R-CNN family)

- R-CNN – Generates region proposals and applies CNN for classification.
- Fast R-CNN and Faster R-CNN – Improved efficiency with shared convolutional layers and Region Proposal Networks (RPNs).
- Strengths – High accuracy.
- Limitations – Slower, making them less suitable for real-time applications.

b. Single Shot Detectors (SSD)

- Predicts bounding boxes and class probabilities in a single pass.
- Faster than R-CNN but sometimes less accurate.

c. You Only Look Once (YOLO)

- Processes the entire image in one go, making it extremely fast.
- Popular in real-time autonomous vehicle applications.
- Variants like YOLOv5 and YOLOv7 achieve a balance between speed and accuracy.

d. Transformer-Based Models (DETR)

- Recent methods use transformers for end-to-end object detection.
- Improve contextual understanding and reduce dependency on handcrafted region proposals.

Challenges in Object Detection for Autonomous Vehicles

Despite advancements, several challenges persist:

1. Environmental Variability – Poor lighting, fog, rain, or snow can reduce sensor accuracy.
2. Real-Time Processing – Object detection must be both fast and accurate to support safe driving decisions.
3. Occlusion and Crowded Scenes – Pedestrians or vehicles partially hidden behind objects make detection harder.
4. Class Imbalance – Some objects (e.g., pedestrians) appear less frequently in datasets compared to vehicles, causing bias in detection.
5. Adversarial Attacks – Deliberate manipulations, such as stickers on stop signs, can fool AI models.

Case Studies and Applications

1. Tesla Autopilot – Primarily camera-based system using deep learning for lane detection, sign recognition, and object tracking.

2. Waymo – Uses LiDAR, radar, and cameras combined with deep learning models for robust perception.
3. NVIDIA DRIVE – Offers AI platforms with pretrained detection models optimized for automotive use.

Future Directions

- Improved Sensor Fusion – Combining multiple modalities with AI for more reliable detection.
- Edge AI Processing – Deploying lightweight, high-speed models optimized for in-car hardware.
- Explainable AI (XAI) – Making detection systems more interpretable to ensure trust and accountability.
- 5G and V2X Integration – Sharing detection data across vehicles and infrastructure for enhanced situational awareness.

Object detection is a cornerstone of autonomous vehicle technology, enabling safe navigation, compliance with traffic laws, and efficient driving. The combination of advanced sensors, deep learning algorithms, and sensor fusion techniques has brought significant progress, yet challenges remain in achieving robustness across diverse environments. With ongoing innovations in AI models, sensor technologies, and computing power, the future of object detection in self-driving cars promises to be both safer and more efficient.

5.4 AI in Self-Driving Cars

Artificial Intelligence (AI) has become the backbone of self-driving cars, enabling vehicles to perceive their surroundings, make decisions, and navigate without direct human intervention. A self-driving car, also known as an autonomous vehicle, relies on AI technologies such as computer vision, machine learning, deep learning, and sensor fusion to function safely and efficiently. The ultimate goal is to reduce accidents caused by human error, improve traffic management, and provide more accessible transportation solutions.

Core Components of AI in Self-Driving Cars

Perception System

The perception system is responsible for enabling the car to “see” and interpret the environment.

- **Sensors used:** Cameras, LiDAR, radar, GPS, and ultrasonic sensors.
- **AI Role:** Computer vision algorithms analyze visual input to detect objects such as vehicles, pedestrians, lane markings, and traffic signs.

- **Techniques:**
    - Convolutional Neural Networks (CNNs) for object detection.
    - Semantic segmentation to identify road boundaries and lanes.
    - Sensor fusion to combine data from multiple sensors for more accurate perception.

Decision-Making System

Once the environment is understood, the AI must decide how the vehicle should act.

- **Path Planning:** Determines the safest and most efficient route.
- **Behavioural Planning:** Decides whether to accelerate, slow down, stop, or overtake another vehicle.
- **AI Techniques:**
    - Reinforcement learning for trial-and-error decision making.
    - Rule-based systems for traffic law compliance.
    - Probabilistic models to handle uncertainty.

Control System

The control system converts AI decisions into actual vehicle movements.

- **Steering, acceleration, and braking** are managed using control algorithms.
- **AI ensures smooth execution** by predicting vehicle dynamics in real time.
- **Example Techniques:** Proportional-Integral-Derivative (PID) controllers, Model Predictive Control (MPC).

Levels of Autonomy

The Society of Automotive Engineers (SAE) defines six levels of automation:

- **Level 0:** No automation (human driver fully in control).
- **Level 1:** Driver assistance (adaptive cruise control).
- **Level 2:** Partial automation (Tesla Autopilot, driver must supervise).
- **Level 3:** Conditional automation (car can drive itself in some conditions).
- **Level 4:** High automation (no driver needed in specific areas).
- **Level 5:** Full automation (completely driverless everywhere).

  AI plays a crucial role in progressing from Level 2 to Level 5 autonomy.

Key AI Technologies in Self-Driving Cars

1. **Computer Vision:**
    - Object detection and classification.
    - Lane and traffic sign recognition.
    - Pedestrian detection.
2. **Machine Learning & Deep Learning:**

    - Training models on large datasets of driving scenarios.
    - Predicting pedestrian or vehicle movement.
3. **Reinforcement Learning:**
    - Teaching cars to learn optimal driving strategies through simulations.
4. **Simulations and Digital Twins:**
    - Training AI in virtual environments such as CARLA or NVIDIA DRIVE Sim.
    - Reduces real-world risks during development.

Challenges in AI-Powered Self-Driving Cars

- **Safety and Reliability:** AI must achieve near-perfect accuracy to avoid accidents.
- **Ethical Concerns:** Decision-making in unavoidable crash scenarios.
- **Data Requirements:** Training requires massive datasets.
- **Adverse Conditions:** AI struggles in poor weather, fog, or unusual traffic situations.
- **Cybersecurity Risks:** Vulnerability to hacking and system manipulation.

Real-World Case Studies

- **Tesla Autopilot:** Uses computer vision and radar for semi-autonomous driving.
- **Waymo (Alphabet/Google):** Focuses on Level 4 automation using LiDAR and deep learning.
- **Uber ATG (now Aurora):** Experimented with mixed AI systems but faced challenges after real-world accidents.

Future Directions

AI in self-driving cars is advancing rapidly, with ongoing research in:

- **V2X (Vehicle-to-Everything) Communication:** Cars exchanging data with each other and with infrastructure.
- **Edge AI:** Running AI computations directly on the car's onboard systems for faster response.
- **Hybrid AI Models:** Combining symbolic AI with deep learning for more transparent decision-making.
- **Regulation and Standardization:** Ensuring safety, ethics, and legal frameworks keep up with technology.

AI is the foundation of self-driving cars, enabling perception, decision-making, and control in complex traffic environments. While significant progress has been made, full autonomy (Level 5) remains a challenge due to safety, ethical, and technical barriers. With continuous improvements in deep learning, sensor technology, and reinforcement learning, AI-powered self-driving cars are moving closer to widespread adoption.

5.5 Automated Grading and Feedback Systems

Education has always relied on assessment as a critical tool for measuring student learning, evaluating progress, and providing feedback for improvement. Traditionally, grading has been carried out manually by teachers, which can be time-consuming, subjective, and inconsistent. With the growth of digital education platforms and advances in Artificial Intelligence (AI) and Natural Language Processing (NLP), **Automated Grading and Feedback Systems (AGFS)** have emerged as a transformative technology in modern education.

Automated grading systems refer to computer-based tools that can evaluate student responses — such as multiple-choice questions, short answers, essays, or even programming assignments — without human intervention. Feedback systems, on the other hand, extend grading by offering students personalized comments, explanations, or suggestions to guide learning. Together, these systems not only reduce the workload of teachers but also enable faster, more consistent, and scalable assessments in diverse learning environments.

Evolution of Automated Grading

The development of AGFS can be traced through different phases:

1. **Rule-Based Grading Systems (1960s–1990s):**
    - Early approaches were based on strict predefined rules.
    - Multiple-choice answer sheets were processed using optical mark recognition (OMR).
    - Essay grading relied on counting word frequency or checking keyword matches.
    - Limitations: lacked flexibility and failed to assess deeper understanding.
2. **Machine Learning–Based Systems (2000s–2010s):**
    - Introduction of ML algorithms enabled pattern recognition in student responses.
    - Natural Language Processing (NLP) allowed evaluation of essays beyond word counts.
    - Examples: e-rater (ETS, used in TOEFL exams).
3. **AI-Enhanced Systems (2015–present):**
    - Deep learning, transformers, and large language models (LLMs) revolutionized grading accuracy.

- These systems can now assess creativity, coherence, argumentation, and writing style.
- They also provide **personalized feedback**, making them interactive learning tools rather than just evaluators.

Types of Automated Grading Systems

Automated grading can be broadly categorized into the following types:

(a) Objective-Type Grading Systems

- Used for multiple-choice, true/false, and matching-type questions.
- Highly accurate because answers are definitive.
- Example: Online quizzes in Moodle, Blackboard, and Google Forms.

(b) Short-Answer Grading Systems

- Evaluate student responses of one or two sentences.
- Use NLP techniques to match semantic similarity rather than just keywords.
- Example: Intelligent Tutoring Systems for science and language learning.

(c) Essay Grading Systems

- The most challenging and advanced type of AGFS.
- Systems analyze structure, coherence, grammar, vocabulary, and argumentation.
- Example: ETS's e-rater, Pearson's Intelligent Essay Assessor.

(d) Programming/Code Grading Systems

- Evaluate student submissions in programming courses.
- Use test cases, code execution, and style analysis.
- Example: HackerRank, Codeforces, and university coding platforms.

Technologies Behind Automated Grading and Feedback

Automated grading relies on a combination of advanced technologies:

1. **Natural Language Processing (NLP):**
    - Tokenization, part-of-speech tagging, syntactic parsing.
    - Semantic similarity models (e.g., word embeddings, BERT).
2. **Machine Learning (ML):**
    - Supervised models trained on large datasets of graded answers.
    - Regression/classification used to predict scores.
3. **Deep Learning:**
    - CNNs and RNNs for text classification.
    - Transformers (BERT, GPT-based models) for essay evaluation.
4. **Automated Feedback Generation:**

    - Uses rule-based hints combined with AI explanations.
    - Personalized recommendations based on errors or performance trends.

Advantages of Automated Grading and Feedback Systems

1. **Efficiency and Scalability:**
    - Can handle thousands of submissions in minutes.
    - Particularly useful for large online courses (MOOCs).
2. **Consistency and Objectivity:**
    - Eliminates human bias and fatigue in grading.
    - Ensures fairness in evaluation.
3. **Instant Feedback to Students:**
    - Students can immediately see their strengths and weaknesses.
    - Encourages continuous learning and self-improvement.
4. **Reduced Teacher Workload:**
    - Teachers can focus more on personalized instruction and mentoring.
5. **Data-Driven Insights:**
    - Systems generate analytics about student performance.
    - Helps in curriculum design and targeted interventions.

Challenges and Limitations

1. **Accuracy Concerns:**
    - AI systems may misinterpret creative or unconventional answers.
    - High reliance on training data — biased data can lead to unfair grading.
2. **Lack of Human Judgment:**
    - Struggles with assessing creativity, critical thinking, and originality.
3. **Ethical Issues:**
    - Concerns about transparency, accountability, and fairness in automated decisions.
4. **Student Adaptation:**
    - Some students may "game" the system by writing long or keyword-heavy responses.
5. **Privacy and Data Security:**
    - Storing large volumes of student data raises concerns of misuse.

Real-World Applications and Case Studies

- **ETS e-rater:** Used in TOEFL exams for automated essay scoring.
- **Pearson's Intelligent Essay Assessor:** Evaluates essays using latent semantic analysis.
- **Coursera & edX:** Use AI systems for large-scale grading of online learners.

- **University Coding Platforms:** Automated test case evaluation in computer science courses.

Automated Grading and Feedback Systems represent a significant step forward in education technology. They enhance efficiency, scalability, and objectivity, making them particularly useful in digital learning environments. However, they also come with challenges, including accuracy limitations, ethical concerns, and the absence of human judgment. As AI technologies evolve, these systems will become more reliable, transparent, and integrated into personalized learning platforms. Ultimately, they will not replace teachers but will act as powerful tools to support educators and improve student learning experiences.

5.6 Al in Music Composition and Creativity

Artificial Intelligence (AI) has expanded beyond technical fields and entered creative domains such as art, literature, and music. Traditionally, music composition was regarded as an inherently human activity that required imagination, emotional depth, and artistic intuition. However, recent advancements in machine learning (ML), deep learning, and generative algorithms have enabled AI systems to not only analyze music but also compose original pieces, generate melodies, and assist human composers in creative workflows.

AI-driven music composition is part of a broader field known as Computational Creativity, which studies how machines can mimic, augment, or inspire human creativity. The question of whether AI can truly "create" music raises philosophical and ethical debates, but its practical applications in education, entertainment, and industry are undeniable.

Foundations of AI in Music

AI systems that generate music typically rely on a combination of music theory, data-driven models, and generative algorithms. Some of the key foundations include:

Representation of Music in AI

- Symbolic Representation: Music is represented in structured formats like MIDI files or musical notation. This allows AI to analyze pitch, rhythm, harmony, and structure.
- Audio Representation: Raw audio signals are analyzed using spectrograms or waveforms to capture timbre and sound textures.

Machine Learning Approaches

- Rule-Based Systems: Early AI music systems followed pre-defined rules of harmony and counterpoint.

- Markov Models: These probabilistic models generate sequences of notes based on statistical patterns.
- Deep Learning Models: Neural networks, especially Recurrent Neural Networks (RNNs) and Transformers, are widely used to learn long-term musical structures.
- Generative Adversarial Networks (GANs): Used for creating novel audio textures and compositions that resemble human-made music.

Applications of AI in Music Composition

Autonomous Music Composition

AI tools can independently generate original pieces in different genres, from classical symphonies to modern pop songs. For example, OpenAI's MuseNet can compose multi-instrumental music across diverse styles.

Human–AI Collaboration

Instead of replacing composers, many AI systems act as assistive tools. A composer can input a melody, and the AI generates harmonization or variations, serving as a creative partner.

Adaptive and Personalized Music

In video games, AI can generate music that dynamically adapts to player actions. Similarly, in wellness applications, AI can generate personalized music for relaxation, focus, or therapy.

Music Education and Practice

AI can help students learn composition by suggesting chord progressions, correcting harmonic inconsistencies, or generating practice exercises.

Commercial and Entertainment Industry

Advertising agencies and streaming services use AI to create royalty-free background music tailored for specific moods, audiences, or events.

Case Studies and Tools

Google Magenta

An open-source project that uses deep learning for music generation. Its Piano Genie allows non-musicians to improvise on the piano with AI assistance.

AIVA (Artificial Intelligence Virtual Artist)

A professional AI composer used in film, video games, and advertising. AIVA is trained on thousands of classical pieces to generate emotionally rich compositions.

Jukedeck & Amper Music

Platforms that allow users to generate custom soundtracks for videos, podcasts, or games without requiring musical expertise.

Benefits of AI in Music Creativity

- Efficiency: AI can produce multiple variations of a melody within seconds.
- Accessibility: Allows non-musicians to compose music with minimal training.
- Inspiration: Provides new ideas and directions for professional composers.
- Scalability: Enables large-scale production of background music for media.

Challenges and Ethical Considerations

- Originality vs. Imitation: AI often mimics existing styles, raising questions about whether the output is truly creative.
- Copyright Issues: If an AI-generated piece closely resembles existing works, legal ownership becomes unclear.
- Human Value of Art: Some argue that music created by machines lacks emotional depth compared to human compositions.
- Bias in Training Data: If the training dataset is limited to certain genres, the AI may fail to generate diverse music.

Future of AI in Music

The future will likely see more hybrid collaborations, where human composers work with AI systems to explore musical possibilities beyond human imagination. Developments in multimodal AI may also integrate music with visual art, storytelling, and immersive virtual environments. AI will not replace human creativity but will augment it as a co-creator, expanding the horizons of artistic expression.

AI in music composition and creativity is transforming how music is made, consumed, and understood. From autonomous music generation to collaborative tools, AI is helping musicians push boundaries and democratize access to music creation. While challenges remain in terms of originality, ethics, and artistic value, the ongoing fusion of AI and music highlights the synergy between technology and creativity, opening up new dimensions for the future of art.

5.7 Traffic Management and Smart Transportation

Traffic congestion and inefficient transportation systems are among the most pressing challenges faced by modern cities. Rapid urbanization, population growth, and increased private vehicle ownership have placed immense pressure on road networks, leading to longer travel times, higher fuel consumption, environmental pollution, and increased risk of accidents. Traditional traffic

management systems, which rely on static signals and manual monitoring, often fail to adapt to the dynamic nature of urban traffic.

The emergence of **Artificial Intelligence (AI), Internet of Things (IoT), and data-driven technologies** has transformed the way cities approach transportation. **Smart transportation systems** integrate AI, big data, and real-time analytics to optimize traffic flow, reduce congestion, enhance road safety, and improve overall mobility. These advancements form the foundation of **intelligent traffic management** in smart cities.

Concept of Smart Transportation

Smart transportation refers to the integration of **digital technologies and intelligent decision-making systems** into existing transportation networks. The key goal is to make transportation safer, more efficient, and environmentally sustainable.

Key features of smart transportation systems include:

- **Real-time traffic monitoring** using cameras, sensors, and GPS data.
- **Predictive analysis** to anticipate congestion patterns.
- **Adaptive traffic signals** that respond to actual traffic demand.
- **Integration with public transport systems** to encourage sustainable travel.
- **Use of AI and machine learning** to continuously improve traffic management.

Traffic Management Using AI and IoT

Data Collection

- **Sensors and IoT devices:** Installed on roads, vehicles, and traffic signals to collect data on vehicle speed, density, and flow.
- **CCTV cameras and drones:** Used for monitoring and real-time traffic analysis.
- **GPS and mobile data:** Collected from smartphones and navigation systems to track movement patterns.

Data Processing

Collected data is analyzed using **AI algorithms and big data analytics**. For instance:

- **Machine learning models** detect traffic anomalies (e.g., sudden congestion or accidents).
- **Predictive analytics** forecast peak traffic hours.
- **Optimization algorithms** suggest alternate routes.

Implementation of AI Solutions

- **Adaptive traffic signals:** Traffic lights that automatically adjust their timing based on current traffic conditions.
- **Incident detection systems:** AI systems that detect accidents and notify emergency services instantly.

- **Smart toll systems:** AI-powered tolling reduces delays and improves traffic flow on highways.

Case Studies of Smart Traffic Systems

Singapore

Singapore has one of the world's most advanced smart transportation systems. It uses **Electronic Road Pricing (ERP)**, AI-powered traffic monitoring, and real-time public transport updates. This has significantly reduced congestion in peak hours.

China (Hangzhou City)

Alibaba's **City Brain Project** in Hangzhou uses AI to control traffic signals, reducing congestion by nearly **15–20%**. The system integrates cameras, sensors, and cloud computing to manage real-time traffic across the city.

India

Indian cities like Delhi and Bangalore are deploying **AI-based adaptive traffic signals** to handle unpredictable traffic surges. Pilot projects have shown improved travel times and reduced waiting at intersections.

Benefits of Smart Transportation Systems

1. **Reduced Congestion:** AI-driven traffic lights and predictive routing minimize delays.
2. **Improved Road Safety:** Accident detection and emergency response are faster.
3. **Environmental Benefits:** Reduced fuel consumption lowers carbon emissions.
4. **Economic Efficiency:** Less time wasted in traffic improves productivity.
5. **Enhanced Public Transport:** Real-time tracking and scheduling make public transport more reliable.

Challenges in Implementing Smart Traffic Management

- **High Infrastructure Cost:** Sensors, IoT devices, and AI systems require significant investment.
- **Data Privacy Concerns:** Real-time tracking of vehicles and people raises privacy issues.
- **Integration with Legacy Systems:** Older traffic infrastructure may not support new technologies.
- **Cybersecurity Risks:** Smart transportation systems are vulnerable to hacking and data breaches.
- **Unequal Accessibility:** Advanced systems may be limited to developed cities, leaving rural areas behind.

Future Directions

The future of smart transportation will rely heavily on emerging technologies such as:

- **5G Networks:** Enabling faster communication between vehicles and traffic systems.
- **Autonomous Vehicles:** Self-driving cars will integrate directly with smart traffic systems.
- **Blockchain Technology:** For secure data sharing in transportation systems.
- **Green Transportation:** Electric and shared mobility solutions integrated with AI for sustainable cities.

Traffic management and smart transportation are central to the development of sustainable and intelligent cities. By leveraging AI, IoT, and big data, urban areas can overcome congestion challenges, improve road safety, and create efficient mobility solutions. Although challenges such as high costs and privacy concerns exist, continuous advancements in technology promise a future where transportation is seamless, eco-friendly, and highly intelligent.

5.8 Case Study on AI - Assisted X-Ray Analysis

Medical imaging has become an essential tool in modern healthcare, enabling physicians to diagnose and monitor diseases with greater accuracy. Among different imaging techniques, **X-rays** remain one of the most widely used methods due to their cost-effectiveness, speed, and ability to detect abnormalities in bones, lungs, and soft tissues.

However, traditional X-ray interpretation requires highly trained radiologists, and human error may occur due to fatigue, subjectivity, or workload pressure. To address these challenges, **Artificial Intelligence (AI)**, particularly **deep learning**, is being integrated into X-ray analysis. AI systems can learn from large datasets of labeled medical images, identify subtle patterns, and assist doctors in making faster and more accurate diagnoses.

This case study explores how AI-assisted X-ray analysis is transforming healthcare, discussing its working principles, applications, benefits, challenges, and future potential.

Background and Need for AI in Radiology

Radiologists are often required to analyze hundreds of images per day. Misinterpretation of X-rays can lead to delayed treatment, unnecessary procedures, or missed diagnoses. The growing demand for medical imaging has also outpaced the number of trained specialists worldwide.

AI provides a solution by:

- **Automating routine tasks** such as detecting fractures, nodules, or pneumonia.
- **Improving diagnostic accuracy** through pattern recognition beyond human perception.

- **Reducing workload** for healthcare professionals.
- **Enhancing accessibility** in regions with limited radiology expertise.

How AI Works in X-Ray Analysis

Data Collection and Preprocessing

AI models are trained using large datasets of labeled X-rays. Preprocessing includes noise reduction, contrast enhancement, and image normalization.

Machine Learning and Deep Learning Models

- **Convolutional Neural Networks (CNNs):** Specialized for image recognition and widely used in X-ray analysis.
- **Transfer Learning:** Pre-trained models like ResNet, VGG, or Inception can be adapted for medical imaging tasks.
- **Ensemble Models:** Combining multiple models for higher accuracy.

Diagnostic Process

1. Input X-ray is fed into the AI model.
2. The system detects abnormalities (e.g., lung infection, bone fracture).
3. The AI provides a probability score and highlights affected regions using heatmaps (Grad-CAM visualization).
4. A radiologist reviews the results for final decision-making.

Applications of AI in X-Ray Analysis

Pneumonia and Lung Diseases

AI systems trained on chest X-ray datasets (e.g., NIH ChestX-ray14) can detect pneumonia, tuberculosis, and COVID-19.

Bone Fracture Detection

AI models identify fractures in wrists, hips, and ribs, which are sometimes missed in busy emergency departments.

Cancer Detection

Early detection of lung nodules and breast abnormalities via AI enhances survival rates through timely intervention.

Triage and Workflow Optimization

AI tools prioritize urgent cases (e.g., pneumothorax) for radiologists, reducing diagnosis delays.

Benefits of AI-Assisted X-Ray Analysis

- **Higher Accuracy:** AI can reduce false negatives and false positives.
- **Faster Diagnosis:** Real-time analysis speeds up emergency care.
- **Scalability:** AI can handle large volumes of data with consistency.

- **Global Healthcare Access:** Assists hospitals in low-resource regions lacking specialists.
- **Support for Radiologists:** Works as a "second opinion," improving confidence in diagnosis.

Challenges and Limitations

- **Data Quality:** AI performance depends on diverse, high-quality training datasets.
- **Bias and Generalization:** Models may underperform on underrepresented patient groups.
- **Interpretability:** "Black box" nature of deep learning makes it difficult to explain decisions.
- **Regulatory and Legal Issues:** Approval from medical authorities (FDA, CE) is required.
- **Ethical Concerns:** Privacy, data protection, and patient consent.

Case Study Examples

- **Google Health's AI for Chest X-rays:** Demonstrated human-level performance in detecting lung conditions.
- **Stanford's CheXNet Model:** Achieved expert radiologist-level accuracy in pneumonia detection using CNNs.
- **Qure.ai (India):** Developed AI solutions for tuberculosis and head CT scan interpretation in low-resource settings.

Future Directions

- **Explainable AI (XAI):** Making AI decisions transparent for clinicians.
- **Integration with Other Modalities:** Combining X-ray analysis with CT scans, MRIs, and lab reports.
- **Continuous Learning Systems:** Models that update with new data to maintain relevance.
- **Global Deployment:** AI can bridge healthcare gaps in rural and developing regions.

AI-assisted X-ray analysis represents a significant advancement in healthcare technology. By combining human expertise with machine intelligence, these systems enhance diagnostic accuracy, reduce workload, and improve patient outcomes. While challenges such as bias, data limitations, and regulatory concerns remain, the future of AI in radiology looks promising.

This case study highlights that AI should not replace radiologists but serve as a **collaborative tool** that strengthens medical decision-making.